

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Hệ thống giám sát chất lượng không khí đô thị sử dụng mạng cảm biến không dây LoRa

NGUYỄN ĐỨC THẮM
tham.nd225395@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: ThS. Lê Bá Vui

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Hệ thống giám sát chất lượng không khí đô thị sử dụng mạng cảm biến không dây LoRa

NGUYỄN ĐỨC THẮM
tham.nd225395@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: ThS. Lê Bá Vui _____

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Lê Bá Vui — giảng viên hướng dẫn đồ án tốt nghiệp. Thầy đã tận tình định hướng, góp ý chuyên môn và đồng hành cùng em trong suốt quá trình thực hiện đồ án.

Em xin cảm ơn các thầy cô trong Khoa Kỹ thuật máy tính, Trường Công nghệ thông tin và Truyền thông, Đại học Bách khoa Hà Nội đã truyền đạt những kiến thức nền tảng quý báu, giúp em có đủ năng lực để hoàn thành đồ án này.

Em cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn ủng hộ, động viên em trong suốt thời gian học tập và thực hiện đồ án.

Mặc dù đã nỗ lực hết sức, đồ án không thể tránh khỏi những thiếu sót. Em rất mong nhận được ý kiến đóng góp từ quý thầy cô và bạn đọc để hoàn thiện hơn.

Cam kết về việc sử dụng công cụ AI

Trong quá trình thực hiện đồ án, em có sử dụng công cụ trí tuệ nhân tạo Google Antigravity làm phương tiện hỗ trợ. Phạm vi sử dụng bao gồm: (i) hỗ trợ viết và gỡ lỗi mã nguồn firmware, backend, frontend; (ii) hỗ trợ soạn thảo và chỉnh sửa nội dung báo cáo; (iii) tra cứu tài liệu kỹ thuật và giải đáp thắc mắc trong quá trình phát triển. Công cụ này được sử dụng thường xuyên trong suốt quá trình thực hiện đồ án.

Tuy nhiên, toàn bộ ý tưởng thiết kế, quyết định kiến trúc, lựa chọn công nghệ, và đánh giá kết quả đều do em tự thực hiện. Công cụ AI chỉ đóng vai trò hỗ trợ kỹ thuật, không thay thế quá trình tư duy và ra quyết định. Em chịu trách nhiệm hoàn toàn về nội dung và tính chính xác của đồ án.

Hà Nội, tháng 6 năm 2026

Sinh viên thực hiện

Nguyễn Đức Thắm

TÓM TẮT NỘI DUNG ĐỒ ÁN

Ô nhiễm không khí là mối đe dọa sức khỏe cộng đồng hàng đầu toàn cầu, đặc biệt nghiêm trọng tại Việt Nam khi nồng độ PM2.5 trung bình năm cao gấp gần sáu lần ngưỡng khuyến nghị của WHO. Hệ thống quan trắc quốc gia chỉ có khoảng 35 trạm đo tự động, không đủ phản ánh chất lượng không khí ở mức vi mô cho từng khu vực đô thị. Các giải pháp thương mại hiện có như PAM Air, IQAir và PurpleAir đều yêu cầu kết nối WiFi hoặc 4G tại mỗi điểm đo, chi phí thiết bị cao và phần mềm mã nguồn đóng. Không giải pháp nào đồng thời đáp ứng bốn tiêu chí: đo đa thông số, truyền dữ liệu tầm xa không cần WiFi, mã nguồn mở và chi phí thấp. Trên cơ sở đó, đồ án lựa chọn hướng tiếp cận xây dựng hệ thống IoT bốn tầng sử dụng cảm biến chi phí thấp kết hợp giao thức truyền thông LoRa tầm xa, loại bỏ yêu cầu hạ tầng mạng tại mỗi điểm đo.

Hệ thống gồm Sensor Node (vi điều khiển ESP32 kết hợp cảm biến PMS7003, CCS811, AHT10) đo sáu thông số môi trường, đóng gói thành gói tin nhị phân 18 byte và gửi qua LoRa đến Gateway. Gateway chuyển tiếp dữ liệu lên Backend Server (Node.js, TimescaleDB), nơi tính chỉ số AQI theo chuẩn US EPA và phát dữ liệu thời gian thực qua Socket.IO đến giao diện web React. Đồ án có bốn đóng góp chính: (i) hệ thống IoT đa tầng end-to-end hoạt động ổn định trong môi trường production với tổng quy mô 14.036 dòng mã, (ii) giao thức LoRa binary 18 byte giảm thời gian chiếm dụng kênh truyền khoảng 11 lần so với JSON kèm cơ chế chống trùng lặp dữ liệu, (iii) pipeline xử lý telemetry trên Backend đảm bảo không mất dữ liệu đo với toàn bộ 44 kịch bản kiểm thử đạt pass, và (iv) chi phí phần cứng khoảng 875.000 VNĐ mỗi Sensor Node — thấp hơn giải pháp IQAir khoảng 11 lần — với toàn bộ mã nguồn được công khai.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

Nguyễn Đức Thắm

ABSTRACT

Air pollution is one of the most critical public health threats worldwide, and is particularly severe in Vietnam, where the annual average PM_{2.5} concentration is nearly six times higher than the WHO guideline. The national air quality monitoring network operates only approximately 35 automated stations, which is insufficient to capture micro-level air quality variations across urban areas. Existing commercial solutions such as PAM Air, IQAir, and PurpleAir all require WiFi or cellular connectivity at each monitoring point, incur high hardware costs, and use closed-source software. None of these systems simultaneously satisfy four criteria: multi-parameter sensing, long-range data transmission without WiFi, open-source availability, and low hardware cost. To address these limitations, this thesis adopts a four-tier IoT architecture combining low-cost sensors with LoRa long-range communication, eliminating the need for network infrastructure at each monitoring point.

The system consists of Sensor Nodes (ESP32 microcontroller integrated with PMS7003, CCS811, and AHT10 sensors) that measure six environmental parameters, encode the data into 18-byte binary packets, and transmit them via LoRa to a Gateway. The Gateway forwards the data over WiFi to a Backend Server (Node.js, TimescaleDB), which computes the Air Quality Index (AQI) according to US EPA standards and broadcasts real-time updates via Socket.IO to a React web frontend. The thesis presents four main contributions: (i) a fully operational end-to-end multi-tier IoT system comprising 14,036 lines of code deployed in a production environment, (ii) an 18-byte binary LoRa protocol that reduces air-time by approximately 11 times compared to JSON, coupled with a data deduplication mechanism, (iii) a telemetry processing pipeline on the Backend that ensures zero data loss, validated by 44 out of 44 test scenarios passing, and (iv) a hardware cost of approximately 875,000 VND (36 USD) per Sensor Node — roughly 11 times lower than the IQAir solution — with the entire source code released as open-source.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.1.1 PAM Air	5
2.1.2 IQAir AirVisual.....	5
2.1.3 PurpleAir	6
2.1.4 Hệ thống quan trắc quốc gia VN-AQI	6
2.1.5 So sánh và đánh giá tổng hợp	6
2.2 Tổng quan chức năng	7
2.2.1 Biểu đồ use case tổng quát	7
2.2.2 Biểu đồ use case phân rã nhóm chức năng Người dùng.....	8
2.2.3 Biểu đồ use case phân rã nhóm chức năng Quản trị	10
2.2.4 Quy trình nghiệp vụ	11
2.3 Đặc tả chức năng	13
2.3.1 Đặc tả use case Xem dashboard chất lượng không khí	14
2.3.2 Đặc tả use case Xem lịch sử dữ liệu	14
2.3.3 Đặc tả use case Quản lý thiết bị.....	15
2.3.4 Đặc tả use case Giám sát và xử lý cảnh báo	15
2.3.5 Đặc tả use case Xuất dữ liệu CSV	16
2.4 Yêu cầu phi chức năng	16

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	18
3.1 Kiến trúc tổng thể hệ thống	18
3.2 Nền tảng phần cứng IoT	19
3.2.1 Vi điều khiển ESP32	19
3.2.2 Module cảm biến	20
3.3 Giao thức truyền thông LoRa.....	21
3.3.1 Tổng quan về LoRa	21
3.3.2 Module AS32-TTL-100	21
3.4 Công nghệ phần mềm phía máy chủ	22
3.4.1 Node.js và Express	22
3.4.2 PostgreSQL, TimescaleDB và PostGIS	22
3.4.3 Prisma ORM	23
3.4.4 Socket.IO	23
3.5 Công nghệ phần mềm phía giao diện	23
3.5.1 React và Vite	23
3.5.2 Thư viện trực quan hóa dữ liệu	24
3.6 Lý thuyết tính toán chỉ số chất lượng không khí	24
3.6.1 Khái niệm và phân loại mức AQI.....	24
3.6.2 Công thức và bảng điểm ngắt	25
3.6.3 Ví dụ tính toán.....	26
3.7 Hạ tầng triển khai	26
3.7.1 Docker và Docker Compose.....	26
3.7.2 Nginx.....	27

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	28
4.1 Thiết kế kiến trúc.....	28
4.1.1 Lựa chọn kiến trúc phần mềm	28
4.1.2 Thiết kế tổng quan.....	29
4.1.3 Mô tả luồng dữ liệu giữa các gói.....	30
4.2 Thiết kế chi tiết.....	31
4.2.1 Thiết kế phần cứng.....	31
4.2.2 Thiết kế firmware.....	32
4.2.3 Thiết kế giao diện	33
4.2.4 Thiết kế module Backend.....	34
4.2.5 Thiết kế cơ sở dữ liệu	37
4.3 Xây dựng ứng dụng.....	38
4.3.1 Thư viện và công cụ sử dụng.....	38
4.3.2 Kết quả đạt được	39
4.3.3 Minh họa các chức năng chính	40
4.4 Kiểm thử.....	47
4.4.1 Kiểm thử chức năng Xem dashboard.....	47
4.4.2 Kiểm thử chức năng Quản lý thiết bị.....	48
4.4.3 Kiểm thử chức năng Giám sát cảnh báo.....	48
4.4.4 Kiểm thử chức năng Xem lịch sử dữ liệu	49
4.4.5 Kiểm thử chức năng Xem bản đồ AQI.....	49
4.4.6 Kiểm thử chức năng Xuất dữ liệu CSV.....	49
4.4.7 Kiểm thử đơn vị module tính AQI.....	49
4.4.8 Tổng kết kiểm thử	50

4.5 Triển khai	51
4.5.1 Kết quả vận hành thực tế.....	52
4.5.2 So sánh với các hệ thống hiện có	69
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	72
5.1 Xây dựng hệ thống IoT đa tầng end-to-end	72
5.1.1 Bài toán và thách thức	72
5.1.2 Giải pháp	72
5.1.3 Kết quả	73
5.2 Thiết kế giao thức LoRa binary và cơ chế chống trùng lặp dữ liệu	73
5.2.1 Bài toán và thách thức	73
5.2.2 Giải pháp	73
5.2.3 Kết quả	74
5.3 Kiến trúc Backend phân lớp và pipeline xử lý telemetry	74
5.3.1 Bài toán và thách thức	74
5.3.2 Giải pháp	75
5.3.3 Kết quả	75
5.4 Hệ thống chi phí thấp với mã nguồn mở hoàn toàn.....	76
5.4.1 Bài toán và thách thức	76
5.4.2 Giải pháp	76
5.4.3 Kết quả	76
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	78
6.1 Kết luận	78
6.2 Hướng phát triển.....	79
TÀI LIỆU THAM KHẢO.....	82

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống	7
Hình 2.2	Biểu đồ use case phân rã nhóm chức năng Người dùng	8
Hình 2.3	Biểu đồ use case phân rã nhóm chức năng Quản trị	10
Hình 2.4	Biểu đồ hoạt động quy trình thu thập và hiển thị dữ liệu	12
Hình 2.5	Biểu đồ hoạt động quy trình phát hiện và xử lý cảnh báo	13
Hình 3.1	Kiến trúc tổng thể hệ thống giám sát chất lượng không khí	19
Hình 4.1	Biểu đồ gói UML tổng quan của hệ thống	29
Hình 4.2	Biểu đồ quan hệ module cho backend services	30
Hình 4.3	Sơ đồ mạch Sensor Node	31
Hình 4.4	Sơ đồ mạch Gateway	32
Hình 4.5	Cấu trúc gói tin LoRa	33
Hình 4.6	Biểu đồ trình tự xử lý dữ liệu Telemetry	35
Hình 4.7	Biểu đồ trình tự xem Dashboard thời gian thực	36
Hình 4.8	Biểu đồ trình tự xử lý cảnh báo vượt ngưỡng	37
Hình 4.9	Biểu đồ thực thể-quan hệ (ERD)	37
Hình 4.10	Dashboard chất lượng không khí	41
Hình 4.11	Dashboard tổng quan quản trị hệ thống	42
Hình 4.12	Giao diện trang Lịch sử dữ liệu	43
Hình 4.13	Giao diện trang Bản đồ AQI	43
Hình 4.14	Giao diện trang Xếp hạng ô nhiễm	44
Hình 4.15	Giao diện chức năng Xuất dữ liệu CSV	44
Hình 4.16	Giao diện trang Quản lý tài khoản	45
Hình 4.17	Nhật ký hệ thống (audit log)	45
Hình 4.18	Giao diện giám sát cảnh báo	46
Hình 4.19	Giao diện cấu hình ngưỡng cảnh báo	46
Hình 4.20	Giao diện quản lý thiết bị	47
Hình 4.21	Biểu đồ triển khai hệ thống	51
Hình 4.22	Kết quả lệnh <code>docker ps</code> trên máy chủ production	52
Hình 4.23	Gateway GW_013 gửi heartbeat	53
Hình 4.24	Server ghi nhận Gateway GW_013 online	53
Hình 4.25	Bật Sensor Node 4C (NODE_076)	54
Hình 4.26	Gateway nhận gói tin từ Sensor Node 4C (NODE_076)	54
Hình 4.27	Server nhận gói tin và đánh dấu Sensor Node 4C online	55
Hình 4.28	Hệ thống ghi nhận Sensor Node 4C (NODE_076) offline	55

Hình 4.29	Hệ thống cảnh báo Sensor Node 4C (NODE_076) offline . . .	56
Hình 4.30	Gateway vẫn duy trì trạng thái online	56
Hình 4.31	Hệ thống ghi nhận Gateway offline	57
Hình 4.32	Hệ thống cảnh báo Gateway offline	57
Hình 4.33	Bố trí thiết bị thử nghiệm đốt hương	58
Hình 4.34	Dashboard pha Baseline (trước khi đốt hương)	59
Hình 4.35	Nhật ký giám sát trong pha Baseline	60
Hình 4.36	Dashboard pha Đốt hương (AQI mức Nguy hại)	61
Hình 4.37	Nhật ký giám sát trong pha Ô nhiễm	61
Hình 4.38	Nhật ký giám sát trong pha Phục hồi	62

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các hệ thống giám sát chất lượng không khí hiện có . . .	6
Bảng 2.2	Đặc tả use case Xem dashboard chất lượng không khí	14
Bảng 2.3	Đặc tả use case Xem lịch sử dữ liệu	14
Bảng 2.4	Đặc tả use case Quản lý thiết bị	15
Bảng 2.5	Đặc tả use case Giám sát và xử lý cảnh báo	15
Bảng 2.6	Đặc tả use case Xuất dữ liệu CSV	16
Bảng 2.7	Các yêu cầu phi chức năng của hệ thống	16
Bảng 3.1	Thông số kỹ thuật các cảm biến sử dụng trong hệ thống	20
Bảng 3.2	Phân loại các mức chỉ số AQI theo tiêu chuẩn US EPA [9] . .	25
Bảng 3.3	Bảng điểm ngắt AQI cho PM2.5 [9]	25
Bảng 3.4	Bảng điểm ngắt AQI cho PM10 [9]	26
Bảng 4.1	Danh sách thư viện và công cụ phía Backend	39
Bảng 4.2	Danh sách thư viện và công cụ phía Frontend	39
Bảng 4.3	Thống kê quy mô mã nguồn hệ thống	40
Bảng 4.4	Đối sánh yêu cầu đặc tả với kết quả đạt được	40
Bảng 4.5	Trường hợp kiểm thử chức năng Xem dashboard	47
Bảng 4.6	Trường hợp kiểm thử chức năng Quản lý thiết bị	48
Bảng 4.7	Trường hợp kiểm thử chức năng Giám sát cảnh báo	48
Bảng 4.8	Trường hợp kiểm thử chức năng Xem lịch sử dữ liệu	49
Bảng 4.9	Trường hợp kiểm thử chức năng Xem bản đồ AQI	49
Bảng 4.10	Trường hợp kiểm thử chức năng Xuất dữ liệu CSV	49
Bảng 4.11	Kết quả kiểm thử đơn vị module <code>aqiService</code>	50
Bảng 4.12	Dữ liệu đo lường kích bản đốt hương	58
Bảng 4.13	Kết quả thử nghiệm thời gian sử dụng pin tối đa	63
Bảng 4.14	Kết quả thử nghiệm khoảng cách truyền LoRa tối đa	64
Bảng 4.15	Dữ liệu đo lường ven đường giao thông	65
Bảng 4.16	Dữ liệu đo lường trong phòng khách	66
Bảng 4.17	Dữ liệu đo lường trong bếp khi nấu ăn	67
Bảng 4.18	So sánh giá trị trung bình giữa ba môi trường	68
Bảng 4.19	So sánh hệ thống đề xuất với các hệ thống hiện có	69
Bảng 4.20	Chi phí linh kiện (BOM) cho Sensor Node và Gateway	70

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
ACID	Tính nguyên tử, nhất quán, cô lập, bền vững (Atomicity, Consistency, Isolation, Durability)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
AQI	Chỉ số chất lượng không khí (Air Quality Index)
BOM	Danh mục vật tư (Bill of Materials)
CRUD	Tạo, Đọc, Cập nhật, Xóa (Create, Read, Update, Delete)
CSV	Giá trị phân cách bằng dấu phẩy (Comma-Separated Values)
FEM	Phương pháp Tương đương Liên bang (Federal Equivalent Method)
GPS	Hệ thống định vị toàn cầu (Global Positioning System)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
HTTP	Giao thức truyền siêu văn bản (HyperText Transfer Protocol)
IoT	Internet vạn vật (Internet of Things)
JSON	Ký pháp đối tượng JavaScript (JavaScript Object Notation)
JWT	Mã thông báo web JSON (JSON Web Token)
LoRa	Truyền thông tầm xa công suất thấp (Long Range)
LoRaWAN	Giao thức mạng diện rộng công suất thấp dựa trên LoRa (Long Range Wide Area Network)
MCU	Vi điều khiển (Microcontroller Unit)
NDIR	Nhiệt kế hồng ngoại không tán sắc (Non-Dispersive Infrared)

Thuật ngữ	Ý nghĩa
ORM	Ánh xạ đối tượng – quan hệ (Object-Relational Mapping)
REST	Chuyển giao trạng thái đại diện (Representational State Transfer)
RTOS	Hệ điều hành thời gian thực (Real-Time Operating System)
SPA	Ứng dụng trang đơn (Single-Page Application)
TVOC	Tổng các hợp chất hữu cơ bay hơi (Total Volatile Organic Compounds)
UART	Bộ thu phát không đồng bộ đa năng (Universal Asynchronous Receiver-Transmitter)
US EPA	Cơ quan Bảo vệ Môi trường Hoa Kỳ (United States Environmental Protection Agency)
WHO	Tổ chức Y tế Thế giới (World Health Organization)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Ô nhiễm không khí là một trong những mối đe dọa sức khỏe cộng đồng nghiêm trọng nhất trên toàn cầu. Theo báo cáo năm 2021 của Tổ chức Y tế Thế giới (WHO), ô nhiễm không khí gây ra khoảng bảy triệu ca tử vong sớm mỗi năm, trong đó bụi mịn PM2.5 là tác nhân chính do khả năng xâm nhập sâu vào hệ hô hấp và tuần hoàn [1]. Tại khu vực Đông Nam Á, Việt Nam là quốc gia chịu ảnh hưởng nặng nề, xếp thứ 23 trên thế giới và thứ hai trong khu vực về mức độ ô nhiễm không khí theo báo cáo chất lượng không khí toàn cầu năm 2024 của IQAir, với nồng độ PM2.5 trung bình năm đạt $28,7 \mu\text{g}/\text{m}^3$ — cao gấp gần sáu lần so với ngưỡng khuyến nghị $5 \mu\text{g}/\text{m}^3$ của WHO [2]. Riêng Hà Nội ghi nhận nồng độ PM2.5 trung bình năm là $45,4 \mu\text{g}/\text{m}^3$, gấp chín lần ngưỡng WHO, khiến thành phố này xếp thứ bảy trong số 121 thủ đô được khảo sát [2].

Trong bối cảnh đó, nhu cầu giám sát chất lượng không khí ở mức vi mô — từng khu phố, trường học, bệnh viện — ngày càng trở nên cấp thiết. Hệ thống quan trắc quốc gia VN-AQI hiện chỉ có khoảng 35 trạm đo tự động trên cả nước [3], không đủ phản ánh sự biến thiên chất lượng không khí giữa các khu vực đô thị liền kề. Morawska và cộng sự đã chỉ ra rằng mạng lưới cảm biến chi phí thấp, mật độ cao là hướng tiếp cận khả thi để bổ sung cho các trạm đo chuẩn, qua đó cung cấp bức tranh toàn diện hơn về chất lượng không khí mà người dân tiếp xúc hàng ngày [4].

Tuy nhiên, các hệ thống giám sát chất lượng không khí hiện có — cả thương mại lẫn cộng đồng — đều có những hạn chế nhất định. Phần lớn các giải pháp như PAM Air, IQAir, PurpleAir yêu cầu kết nối WiFi hoặc 4G tại mỗi điểm đo, hạn chế khả năng triển khai tại các khu vực không có hạ tầng mạng sẵn có. Chi phí thiết bị thương mại cũng là rào cản lớn khi triển khai diện rộng, với mức giá từ vài triệu đến hàng trăm triệu đồng mỗi trạm. Ngoài ra, hầu hết các hệ thống đều là mã nguồn đóng, không cho phép tùy biến logic nghiệp vụ hoặc tích hợp với hệ thống bên thứ ba. Những hạn chế này đặt ra bài toán cần giải quyết: xây dựng một hệ thống giám sát chất lượng không khí đa thông số, chi phí thấp, có khả năng truyền dữ liệu tầm xa mà không cần hạ tầng WiFi tại mỗi điểm đo, và công khai toàn bộ mã nguồn.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay có bốn nhóm giải pháp giám sát chất lượng không khí tiêu biểu. PAM Air triển khai mạng lưới hơn 3.000 trạm đo tại Việt Nam nhưng sử dụng mã nguồn đóng và chỉ đo PM2.5 [5]. IQAir cung cấp nền tảng quốc tế với khả năng dự báo AI nhưng chi phí thiết bị cao (khoảng 10 triệu VNĐ mỗi thiết bị) và phần mềm không

phải mã nguồn mở [6]. PurpleAir dựa trên mô hình cộng đồng và cung cấp API mở nhưng chỉ đo bụi mịn, yêu cầu WiFi cho mỗi trạm [7]. Hệ thống quan trắc quốc gia VN-AQI đạt tiêu chuẩn FEM nhưng chi phí mỗi trạm từ hàng trăm triệu đến hàng tỷ đồng, số lượng trạm rất hạn chế [3]. Bên cạnh đó, xu hướng sử dụng cảm biến chi phí thấp kết hợp giao thức truyền thông công suất thấp tầm xa đã được nhiều nghiên cứu quốc tế khẳng định là hướng đi triển vọng [4], [8].

Phân tích tổng quan cho thấy không giải pháp nào đồng thời đáp ứng bốn tiêu chí: (i) đo đa thông số môi trường, (ii) truyền dữ liệu tầm xa mà không cần WiFi tại từng điểm đo, (iii) mã nguồn mở hoàn toàn, và (iv) chi phí phần cứng thấp. Đặc biệt, không hệ thống nào sử dụng giao thức Long Range (LoRa) — giao thức đã được chứng minh có khả năng truyền thông đến 10–15 km trong điều kiện tầm nhìn thẳng với mức tiêu thụ năng lượng rất thấp [8].

Trên cơ sở đó, đề án đặt mục tiêu xây dựng một hệ thống giám sát chất lượng không khí end-to-end với năm chức năng chính: (i) thu thập dữ liệu sáu thông số môi trường (PM_{2.5}, PM₁₀, CO₂, Tổng các hợp chất hữu cơ bay hơi (Total Volatile Organic Compounds – TVOC), nhiệt độ, độ ẩm) từ các cảm biến chi phí thấp, (ii) truyền dữ liệu tầm xa qua LoRa mà không cần WiFi tại từng trạm đo, (iii) xử lý, lưu trữ và tính chỉ số Chỉ số chất lượng không khí (Air Quality Index – AQI) theo chuẩn Cơ quan Bảo vệ Môi trường Hoa Kỳ (United States Environmental Protection Agency – US EPA) [9], (iv) hiển thị dashboard web thời gian thực cho người dùng gồm bản đồ AQI, biểu đồ lịch sử và xếp hạng ô nhiễm, và (v) cung cấp bảng quản trị cho quản trị viên gồm quản lý thiết bị, giám sát cảnh báo, cấu hình hệ thống, quản lý tài khoản và xuất dữ liệu. Hệ thống được phát triển dưới dạng mã nguồn mở hoàn toàn, cho phép bên thứ ba tái sử dụng, tùy biến và mở rộng.

1.3 Định hướng giải pháp

Để đáp ứng các mục tiêu đặt ra ở phần 1.2, em lựa chọn kiến trúc bốn tầng cho hệ thống. Tầng cảm biến sử dụng vi điều khiển ESP32 kết hợp ba module cảm biến chi phí thấp (PMS7003 đo bụi mịn, CCS811 đo CO₂ và TVOC, AHT10 đo nhiệt độ và độ ẩm). Firmware Sensor Node sử dụng cơ chế Deep-sleep để tối ưu năng lượng; trong phạm vi đề án, prototype dùng board phát triển ESP32 DevKit V1 nên tuổi thọ pin đạt khoảng 20,5 giờ — hạn chế chủ yếu đến từ các linh kiện phụ trên board (chip USB-UART, LDO) chứ không phải từ kiến trúc firmware; với thiết kế phần cứng tùy chỉnh (bare-chip ESP32), thời gian hoạt động có thể đạt nhiều tháng. Tầng trung chuyển sử dụng Gateway ESP32 nhận dữ liệu qua giao thức LoRa point-to-point ở tần số 433 MHz và chuyển tiếp lên máy chủ qua WiFi. Tầng máy chủ sử dụng Node.js/Express kết hợp PostgreSQL mở rộng bằng TimescaleDB (cơ sở dữ

liệu chuỗi thời gian) và PostGIS (truy vấn không gian), giao tiếp thời gian thực qua Socket.IO. Tầng hiển thị sử dụng React/Vite kết hợp Leaflet (bản đồ) và Apache ECharts (biểu đồ). Toàn bộ hệ thống được container hóa bằng Docker để đơn giản hóa triển khai.

Sensor Node đo sáu thông số môi trường, đóng gói thành gói tin nhị phân 18 byte và gửi qua LoRa đến Gateway. Gateway lưu gói tin vào bộ đệm vòng, sau đó gửi lên Backend Server dưới dạng HTTP POST. Backend Server xử lý dữ liệu theo pipeline ba bước: lọc trùng lặp, lưu vào cơ sở dữ liệu và tính chỉ số AQI, kiểm tra ngưỡng cảnh báo và phát sóng dữ liệu mới đến giao diện web qua Socket.IO. Frontend cập nhật dashboard theo thời gian thực mà không cần người dùng tải lại trang.

Đề án có bốn đóng góp chính. Thứ nhất, hệ thống Internet vạn vật (Internet of Things – IoT) đa tầng end-to-end hoạt động ổn định trong môi trường production, tích hợp bốn miền kỹ thuật khác biệt (phần cứng, firmware, backend, frontend) với tổng quy mô khoảng 14.036 dòng mã. Thứ hai, giao thức LoRa binary 18 byte giảm thời gian chiếm dụng kênh truyền khoảng 11 lần so với JSON, kèm cơ chế chống trùng lặp dữ liệu khi nhiều Gateway thu nhận cùng một gói tin. Thứ ba, pipeline xử lý telemetry trên Backend đảm bảo không mất dữ liệu đo ngay cả khi module cảnh báo xảy ra lỗi, với toàn bộ 44 kịch bản kiểm thử đạt kết quả pass. Thứ tư, chi phí phần cứng mỗi Sensor Node ở mức khoảng 875.000 VNĐ — thấp hơn giải pháp IQAir khoảng 11 lần — và toàn bộ mã nguồn được công khai.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 tiến hành khảo sát bốn hệ thống giám sát chất lượng không khí tiêu biểu (PAM Air, IQAir, PurpleAir, VN-AQI), phân tích ưu nhược điểm và xác định khoảng trống mà các giải pháp hiện tại chưa giải quyết được. Trên cơ sở đó, chương trình bày phân tích yêu cầu chức năng thông qua biểu đồ use case cho hai tác nhân Người dùng và Quản trị viên, đặc tả chi tiết năm use case quan trọng nhất, và xác định năm yêu cầu phi chức năng của hệ thống.

Chương 3 trình bày nền tảng lý thuyết và các công nghệ được lựa chọn cho từng tầng trong kiến trúc hệ thống. Nội dung bao gồm kiến trúc tổng thể bốn tầng, nền tảng phần cứng IoT (vi điều khiển ESP32 và các module cảm biến), giao thức truyền thông LoRa, công nghệ phần mềm phía máy chủ (Node.js, TimescaleDB, Socket.IO) và phía giao diện (React, Leaflet, ECharts), lý thuyết tính toán chỉ số AQI theo công thức nội suy tuyến tính của US EPA, và hạ tầng triển khai Docker.

Chương 4 trình bày chi tiết quá trình thiết kế kiến trúc (biểu đồ gói UML, biểu đồ trình tự), thiết kế phần cứng (sơ đồ mạch Sensor Node và Gateway), thiết kế firmware (giao thức LoRa binary, luồng hoạt động deep sleep), thiết kế module Backend (pipeline xử lý telemetry, cơ chế cảnh báo), thiết kế cơ sở dữ liệu (TimescaleDB hypertable, Continuous Aggregate), và thiết kế giao diện người dùng. Chương cũng trình bày kết quả triển khai, kiểm thử (44/44 kịch bản pass, bao gồm 19 test case đơn vị module tính AQI), và kết quả thực nghiệm trên môi trường production.

Chương 5 phân tích bốn đóng góp kỹ thuật nổi bật của đề án, bao gồm việc xây dựng hệ thống IoT đa tầng end-to-end, thiết kế giao thức LoRa binary và cơ chế chống trùng lặp dữ liệu, kiến trúc Backend phân lớp với pipeline xử lý telemetry đảm bảo tính bền vững của dữ liệu, và giải pháp chi phí thấp với mã nguồn mở hoàn toàn. Mỗi đóng góp được phân tích theo cấu trúc bài toán, giải pháp và kết quả đạt được.

Chương 6 tổng kết toàn bộ đề án, đánh giá mức độ hoàn thành so với mục tiêu đề ra, ghi nhận các hạn chế còn tồn tại (thời gian pin dài hạn, hiệu chuẩn cảm biến, thiếu ứng dụng di động), và đề xuất bốn hướng phát triển trong tương lai: tối ưu phần cứng, tích hợp chuẩn LoRaWAN, bổ sung mô hình dự báo AQI và thông báo đẩy, cùng triển khai thử nghiệm quy mô lớn hơn.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã giới thiệu bối cảnh thực tiễn của bài toán giám sát chất lượng không khí đô thị, xác định mục tiêu và phạm vi đề tài, đồng thời đưa ra định hướng giải pháp dựa trên mạng cảm biến không dây LoRa. Trên cơ sở đó, Chương 2 tiến hành khảo sát các hệ thống giám sát chất lượng không khí hiện có, phân tích ưu nhược điểm và xác định khoảng trống cần giải quyết. Tiếp theo, chương trình bày phân tích yêu cầu chức năng thông qua biểu đồ use case, đặc tả chi tiết các chức năng chính, và các yêu cầu phi chức năng của hệ thống.

2.1 Khảo sát hiện trạng

Để xác định rõ hạn chế của các giải pháp hiện tại và định hướng phát triển hệ thống, em tiến hành khảo sát bốn hệ thống giám sát chất lượng không khí tiêu biểu, bao gồm cả hệ thống trong nước và quốc tế.

2.1.1 PAM Air

PAM Air là hệ thống giám sát chất lượng không khí do công ty Việt Nam PAM Air JSC phát triển, triển khai mạng lưới hơn 3.000 trạm đo trên toàn quốc [5]. Hệ thống sử dụng các cảm biến chi phí thấp, kết nối WiFi hoặc 4G, đo chủ yếu bụi mịn PM_{2.5} — một trong những chỉ số quan trọng nhất theo khuyến nghị của WHO [1]. Dữ liệu được hiển thị trên ứng dụng di động và trang web với bản đồ theo thời gian thực.

Tuy nhiên, PAM Air có một số hạn chế. Thứ nhất, phần cứng và phần mềm đều là mã nguồn đóng, không cho phép tùy biến hoặc mở rộng. Thứ hai, mỗi trạm đo yêu cầu kết nối WiFi hoặc 4G riêng, hạn chế khả năng triển khai tại các khu vực không có hạ tầng mạng. Thứ ba, hệ thống chủ yếu đo PM_{2.5}, không tích hợp đo CO₂ hoặc TVOC — những thông số mà Morawska và cộng sự [4] đã chỉ ra là cần thiết cho đánh giá toàn diện chất lượng không khí trong nhà và ngoài trời.

2.1.2 IQAir AirVisual

IQAir AirVisual là nền tảng quốc tế cung cấp dữ liệu chất lượng không khí từ hơn 100 quốc gia, kết hợp dữ liệu từ các trạm quan trắc chính thức và các thiết bị đo cộng đồng [6]. Hệ thống có ứng dụng di động và trang web với giao diện trực quan, hỗ trợ dự báo chất lượng không khí dựa trên mô hình AI.

Hạn chế chính của IQAir là chi phí thiết bị đo cá nhân cao (khoảng 10 triệu VNĐ cho AirVisual Pro). Dữ liệu phụ thuộc vào hạ tầng đám mây quốc tế, gây rủi ro về độ trễ và quyền kiểm soát dữ liệu. Ngoài ra, người dùng không thể tự triển khai hệ thống riêng vì phần mềm không phải mã nguồn mở.

2.1.3 PurpleAir

PurpleAir là hệ thống giám sát chất lượng không khí dựa trên mô hình cộng đồng, bán các trạm đo cá nhân sử dụng cảm biến laser (Plantower PMS5003) [7]. Dữ liệu từ tất cả các trạm được tổng hợp trên bản đồ trực tuyến công khai. PurpleAir cung cấp API mở cho phép truy xuất dữ liệu.

Tuy nhiên, PurpleAir chỉ đo bụi mịn PM2.5 và PM10, không hỗ trợ đo khí CO₂, TVOC, nhiệt độ hoặc độ ẩm. Mỗi trạm đo yêu cầu kết nối WiFi trực tiếp. Hệ thống không hỗ trợ giao thức truyền tầm xa như LoRa — một giao thức đã được chứng minh phù hợp cho các ứng dụng IoT tầm xa, tiêu thụ năng lượng thấp [8] — do đó không phù hợp để triển khai mạng lưới phân tán trên diện rộng mà không có hạ tầng WiFi.

2.1.4 Hệ thống quan trắc quốc gia VN-AQI

Hệ thống quan trắc chất lượng không khí quốc gia do Bộ Tài nguyên và Môi trường vận hành, sử dụng các trạm đo chuyên dụng tiêu chuẩn FEM (Federal Equivalent Method) [3]. Dữ liệu được công bố trên trang enviinfo.cem.gov.vn theo chỉ số VN-AQI. Ưu điểm nổi bật là độ chính xác cao, đáp ứng tiêu chuẩn quốc tế.

Hạn chế lớn nhất là chi phí triển khai cực cao, mỗi trạm có giá từ hàng trăm triệu đến hàng tỷ đồng. Số lượng trạm rất ít (khoảng 35 trạm tự động trên cả nước), không đủ phủ sóng mức vi mô cho từng khu vực đô thị.

2.1.5 So sánh và đánh giá tổng hợp

Bảng 2.1 tổng hợp so sánh bốn hệ thống đã khảo sát theo các tiêu chí quan trọng.

Bảng 2.1: So sánh các hệ thống giám sát chất lượng không khí hiện có

Tiêu chí	PAM Air	IQAir	PurpleAir	VN-AQI
Cảm biến	PM2.5	PM2.5, CO ₂	PM2.5, PM10	Đầy đủ (FEM)
Truyền thông	WiFi/4G	WiFi	WiFi	Chuyên dụng
Chi phí/trạm	Trung bình	Cao	Trung bình	Rất cao
Mã nguồn mở	Không	Không	Một phần	Không
Tùy biến	Không	Không	Hạn chế	Không
LoRa	Không	Không	Không	Không
Realtime web	Có	Có	Có	Hạn chế
Admin panel	Không rõ	Không	Không	Có (nội bộ)

Từ kết quả khảo sát, có thể nhận thấy các hệ thống hiện tại đều có những hạn chế nhất định. Các hệ thống thương mại như PAM Air và IQAir không cho phép tùy biến và mở rộng. Các hệ thống cộng đồng như PurpleAir chỉ đo được bụi mịn và yêu cầu WiFi cho mỗi trạm. Hệ thống quan trắc quốc gia đáp ứng tiêu chuẩn FEM

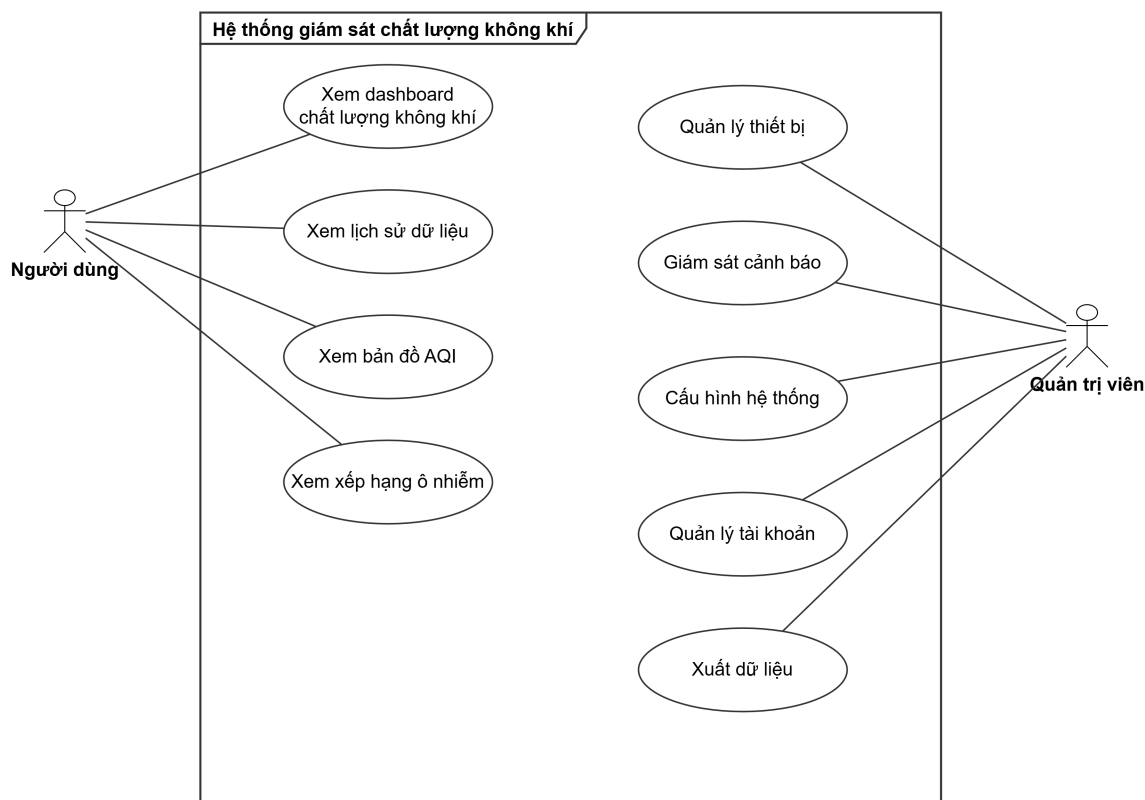
nhưng chi phí quá cao cho quy mô triển khai vi mô. Đặc biệt, không hệ thống nào hỗ trợ giao thức LoRa để truyền dữ liệu tầm xa mà không cần hạ tầng mạng WiFi tại mỗi điểm đo — trong khi LoRa đã được chứng minh có khả năng truyền thông đến 10–15 km trong điều kiện tầm nhìn thẳng với mức tiêu thụ năng lượng rất thấp [8].

Hệ thống đề xuất trong đề án này hướng tới giải quyết các hạn chế trên bằng cách kết hợp: (i) cảm biến chi phí thấp đo đa thông số (PM2.5, PM10, CO₂, TVOC, nhiệt độ, độ ẩm) theo khuyến nghị của Morawska và cộng sự [4], (ii) truyền thông LoRa không cần WiFi cho mỗi node [8], (iii) mã nguồn mở hoàn toàn, (iv) dashboard web thời gian thực với chỉ số AQI theo chuẩn US EPA [9], và (v) bảng quản trị cho quản trị viên.

2.2 Tổng quan chức năng

Phần này trình bày tổng quan các chức năng của hệ thống thông qua biểu đồ use case. Hệ thống có hai tác nhân chính: **Người dùng** (User) và **Quản trị viên** (Admin). Người dùng là đối tượng sử dụng dashboard web để tra cứu thông tin chất lượng không khí. Quản trị viên là người quản lý hệ thống, bao gồm quản lý thiết bị, cấu hình, theo dõi cảnh báo và xuất dữ liệu.

2.2.1 Biểu đồ use case tổng quát



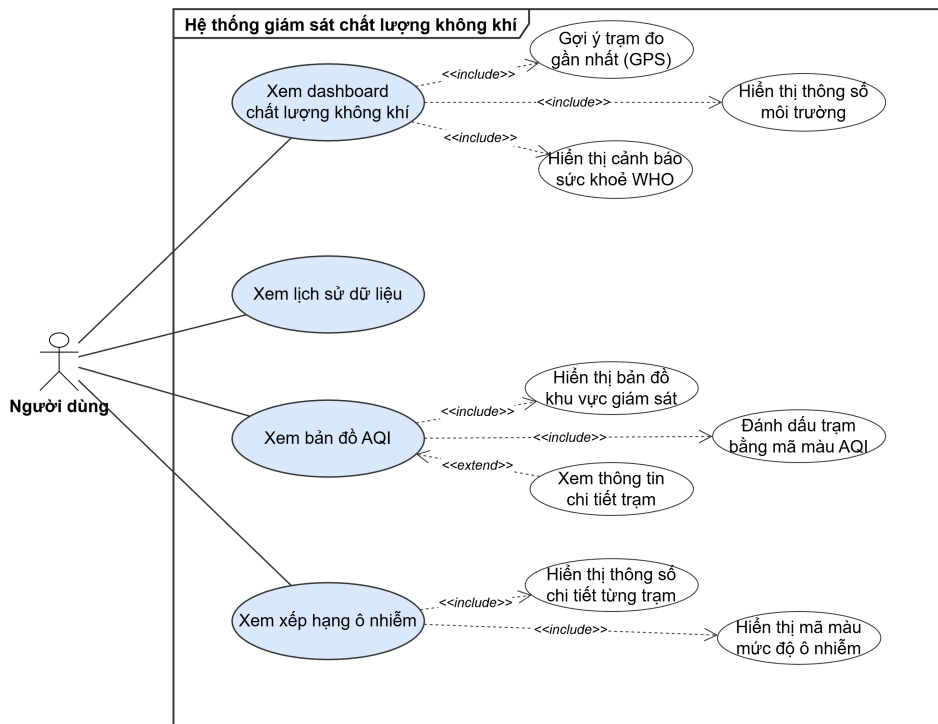
Hình 2.1: Biểu đồ use case tổng quát của hệ thống

Hình 2.1 thể hiện biểu đồ use case tổng quát của hệ thống với hai tác nhân chính: Người dùng và Quản trị viên. Mỗi tác nhân tương ứng với một nhóm chức năng riêng biệt.

Nhóm chức năng Người dùng gồm bốn use case mức cao: (i) *Xem dashboard chất lượng không khí*, hiển thị các thông số AQI, PM2.5, PM10, CO₂, TVOC, nhiệt độ, độ ẩm của trạm đo gần nhất theo thời gian thực; (ii) *Xem lịch sử dữ liệu*, tra cứu biểu đồ lịch sử theo trạm, theo thông số và theo khoảng thời gian; (iii) *Xem bản đồ AQI*, hiển thị bản đồ toàn khu vực với các trạm đo được đánh dấu bằng mã màu AQI; và (iv) *Xem xếp hạng ô nhiễm*, sắp xếp các trạm theo mức AQI từ cao đến thấp.

Nhóm chức năng Quản trị viên gồm năm use case mức cao: (i) *Quản lý thiết bị*, thêm, sửa, xóa và giám sát trạng thái Gateway và Sensor Node; (ii) *Giám sát cảnh báo*, xem danh sách cảnh báo, lọc theo mức độ nghiêm trọng, xác nhận đã xử lý; (iii) *Cấu hình hệ thống*, thiết lập ngưỡng cảnh báo cho từng thông số và chu kỳ gửi dữ liệu; (iv) *Quản lý tài khoản*, thêm, sửa, xóa tài khoản người dùng và phân quyền admin/user; và (v) *Xuất dữ liệu*, xuất dữ liệu đo lường ra tệp CSV theo trạm và khoảng thời gian.

2.2.2 Biểu đồ use case phân rã nhóm chức năng Người dùng



Hình 2.2: Biểu đồ use case phân rã nhóm chức năng Người dùng

Hình 2.2 thể hiện biểu đồ phân rã bốn use case của nhóm chức năng Người dùng. Mỗi use case được phân rã thành các chức năng con cụ thể như sau.

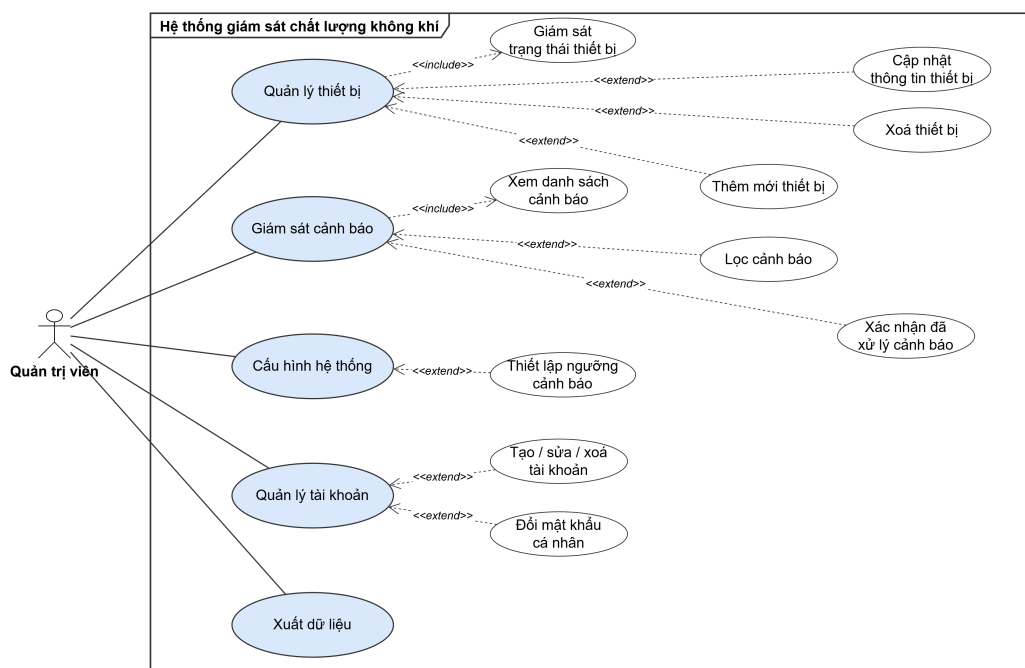
Xem dashboard chất lượng không khí. Khi người dùng truy cập dashboard, hệ thống tự động gợi ý trạm đo gần nhất dựa trên vị trí GPS thông qua Geolocation API và PostGIS. Dashboard hiển thị sáu thông số môi trường gồm PM2.5, PM10, CO₂, TVOC, nhiệt độ và độ ẩm, cùng chỉ số AQI tổng hợp được tính theo công thức của US EPA [9]. Ngoài ra, hệ thống hiển thị cảnh báo sức khỏe theo hướng dẫn của WHO [1], kèm khuyến nghị cho nhóm nhạy cảm. Dữ liệu được cập nhật thời gian thực thông qua Socket.IO.

Xem lịch sử dữ liệu. Chức năng này cho phép người dùng: (i) chọn trạm đo cần xem, (ii) chọn thông số cần tra cứu (AQI, PM2.5, PM10, CO₂, TVOC, nhiệt độ hoặc độ ẩm), và (iii) chọn chế độ xem theo giờ hoặc theo ngày. Kết quả được hiển thị dưới dạng biểu đồ sử dụng thư viện ECharts, giúp người dùng quan sát xu hướng thay đổi chất lượng không khí theo thời gian.

Xem bản đồ AQI. Hệ thống hiển thị bản đồ toàn khu vực giám sát sử dụng thư viện Leaflet. Các trạm đo được đánh dấu bằng chấm tròn với mã màu AQI theo chuẩn US EPA [9]: xanh (tốt), vàng (trung bình), cam (không tốt cho nhóm nhạy cảm), đỏ (có hại), tím (rất có hại) và nâu đỏ (nguy hiểm). Người dùng có thể nhấn vào trạm để xem thông tin chi tiết.

Xem xếp hạng ô nhiễm. Chức năng hiển thị danh sách các trạm được sắp xếp theo mức AQI từ cao đến thấp. Mỗi dòng hiển thị tên trạm, mức AQI, nồng độ PM2.5 và màu sắc tương ứng, giúp người dùng nhanh chóng xác định khu vực ô nhiễm nhất.

2.2.3 Biểu đồ use case phân rã nhóm chức năng Quản trị



Hình 2.3: Biểu đồ use case phân rã nhóm chức năng Quản trị

Hình 2.3 thể hiện biểu đồ phân rã năm use case của nhóm chức năng Quản trị. Mỗi use case được phân rã thành các thao tác cụ thể như sau.

Quản lý thiết bị. Quản trị viên có thể thêm mới Gateway bằng cách nhập tên và mô tả vị trí, trong đó hệ thống tự sinh ID theo định dạng GW_XXX. Tương tự, quản trị viên thêm mới Sensor Node bằng cách nhập tên, chọn Gateway cha và nhập tọa độ GPS, hệ thống tự sinh ID theo định dạng NODE_XXX. Quản trị viên cũng có thể cập nhật thông tin thiết bị (tên, vị trí, trạng thái) và xóa thiết bị. Ngoài ra, hệ thống hỗ trợ giám sát trạng thái thiết bị, hiển thị trạng thái online/offline, thời gian gửi dữ liệu cuối (last seen) và mức pin.

Giám sát cảnh báo. Quản trị viên xem danh sách cảnh báo với hỗ trợ phân trang, có thể lọc theo loại (tên thiết bị), mức độ (cảnh báo hoặc nguy hiểm) và trạng thái xác nhận. Quản trị viên có thể xác nhận đã xử lý cảnh báo (acknowledge). Hệ thống phát thông báo cảnh báo thời gian thực qua Socket.IO khi có sự kiện mới.

Cấu hình hệ thống. Quản trị viên thiết lập ngưỡng cảnh báo cho sáu thông số: PM2.5, PM10, CO₂, TVOC, nhiệt độ (tối thiểu/tối đa). Mỗi thông số có hai mức: cảnh báo (warn) và nguy hiểm (danger).

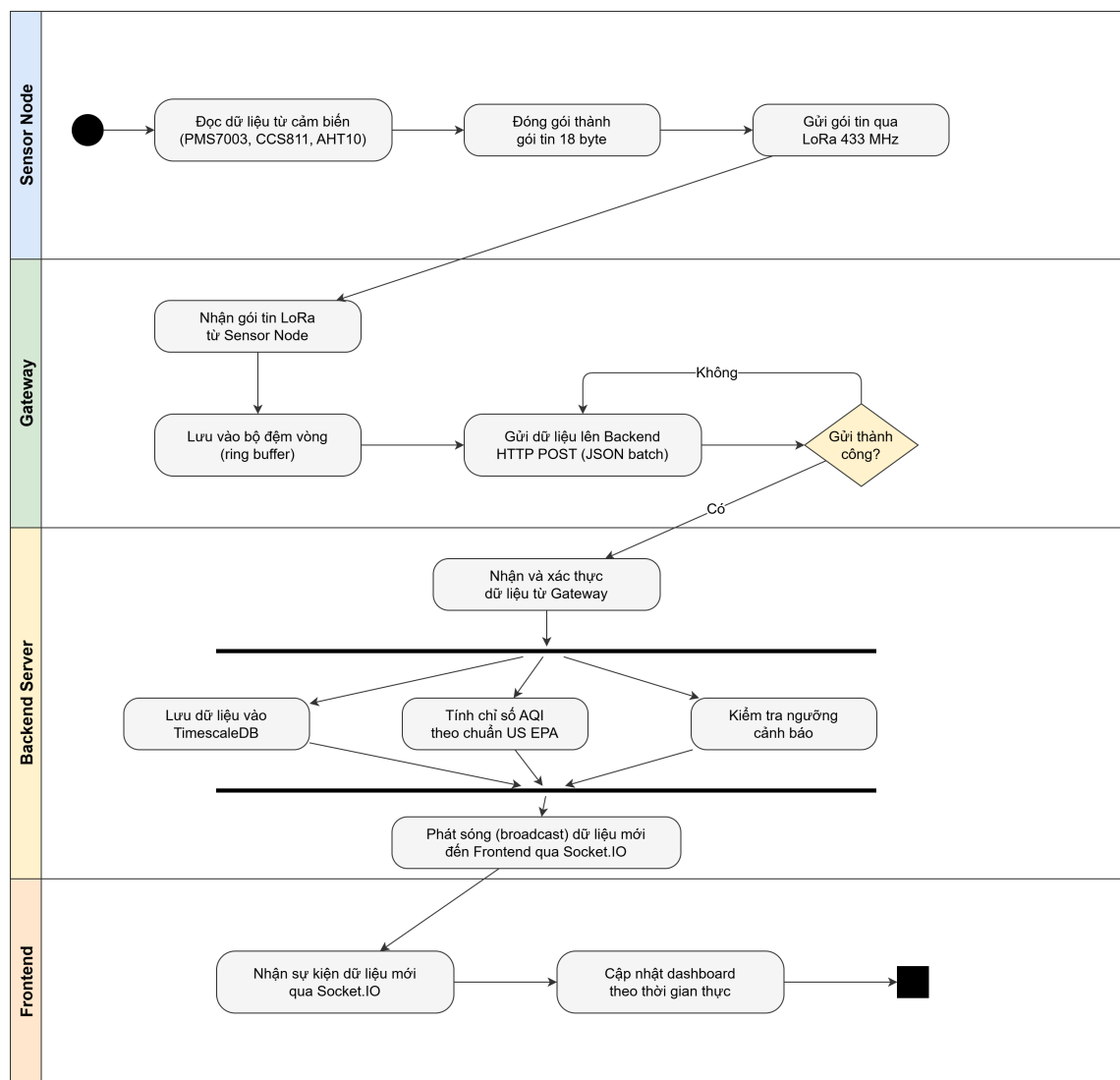
Quản lý tài khoản. Quản trị viên thực hiện tạo, sửa, xóa tài khoản người dùng và phân quyền admin hoặc user. Mỗi người dùng có thể đổi mật khẩu cá nhân. Hệ

thống bảo vệ không cho xóa tài khoản admin cuối cùng nhằm đảm bảo luôn có ít nhất một quản trị viên trong hệ thống.

2.2.4 Quy trình nghiệp vụ

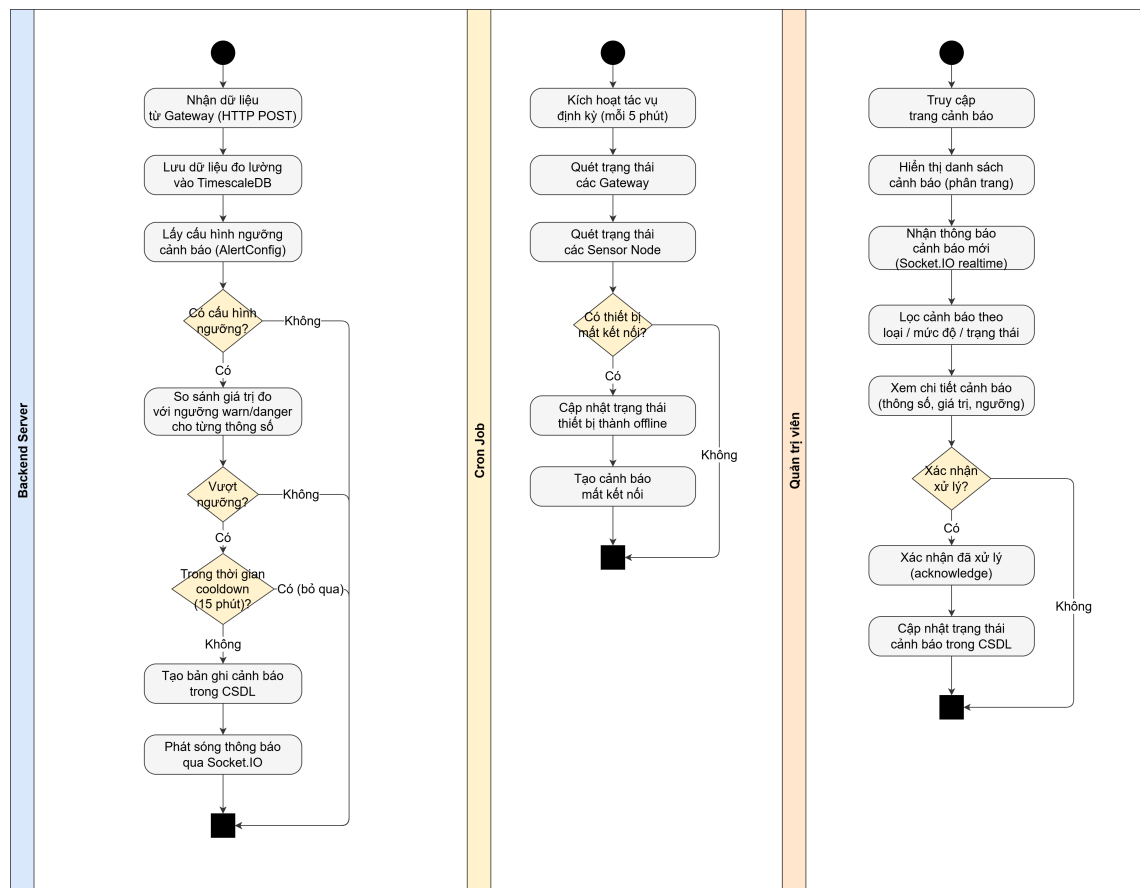
Hệ thống có hai quy trình nghiệp vụ chính.

Quy trình 1: Thu thập và hiển thị dữ liệu thời gian thực. Sensor Node đọc dữ liệu từ các cảm biến (PMS7003, CCS811, AHT10), đóng gói thành gói tin 18 byte và gửi qua LoRa đến Gateway. Gateway nhận gói tin, lưu vào bộ đệm vòng (ring buffer), sau đó gửi lên Backend Server dưới dạng HTTP POST (JSON batch). Backend Server xử lý dữ liệu: lưu vào cơ sở dữ liệu TimescaleDB, tính chỉ số AQI theo công thức US EPA [9], kiểm tra ngưỡng cảnh báo, và phát sóng (broadcast) dữ liệu mới đến tất cả Frontend thông qua Socket.IO. Frontend cập nhật dashboard theo thời gian thực mà không cần người dùng tải lại trang. Hình 2.4 minh họa toàn bộ luồng xử lý của quy trình này, từ khi Sensor Node đọc dữ liệu đến khi Frontend hiển thị kết quả.



Hình 2.4: Biểu đồ hoạt động quy trình thu thập và hiển thị dữ liệu

Quy trình 2: Phát hiện và xử lý cảnh báo. Khi Backend Server nhận dữ liệu mới từ Gateway, hệ thống tự động so sánh giá trị đo được với ngưỡng cảnh báo đã cấu hình. Nếu giá trị vượt ngưỡng, hệ thống tạo bản ghi cảnh báo trong cơ sở dữ liệu và phát sóng thông báo đến giao diện quản trị qua Socket.IO. Ngoài ra, một tác vụ định kỳ (cron job) chạy mỗi 5 phút kiểm tra các thiết bị không gửi dữ liệu trong thời gian quy định, tự động tạo cảnh báo mất kết nối. Quản trị viên xem danh sách cảnh báo, lọc theo tiêu chí, và xác nhận đã xử lý. Hình 2.5 minh họa luồng xử lý cảnh báo từ khi phát hiện bất thường đến khi quản trị viên xác nhận.



Hình 2.5: Biểu đồ hoạt động quy trình phát hiện và xử lý cảnh báo

2.3 Đặc tả chức năng

Phần này đặc tả chi tiết năm use case quan trọng nhất của hệ thống. Mỗi đặc tả bao gồm tên use case, tác nhân, mô tả, luồng sự kiện chính, luồng phát sinh, tiền điều kiện và hậu điều kiện.

2.3.1 Đặc tả use case Xem dashboard chất lượng không khí

Bảng 2.2: Đặc tả use case Xem dashboard chất lượng không khí

Tên use case	Xem dashboard chất lượng không khí
Tác nhân	Người dùng
Mô tả	Người dùng truy cập dashboard để xem các thông số chất lượng không khí theo thời gian thực tại trạm đo gần nhất.
Tiền điều kiện	Có ít nhất một trạm đo đang hoạt động trong hệ thống.
Luồng sự kiện chính	1. Người dùng truy cập trang dashboard. 2. Hệ thống yêu cầu quyền truy cập vị trí (Geolocation API). 3. Người dùng cho phép truy cập vị trí. 4. Hệ thống truy vấn trạm đo gần nhất dựa trên toạ độ GPS (PostGIS). 5. Hệ thống hiển thị sáu thông số môi trường (PM2.5, PM10, CO₂, TVOC, nhiệt độ, độ ẩm), chỉ số AQI tổng hợp, cảnh báo sức khoẻ WHO và thông tin thời tiết. 6. Hệ thống tự động cập nhật dữ liệu khi có dữ liệu mới từ Socket.IO.
Luồng phát sinh	3a. Người dùng từ chối truy cập vị trí: hệ thống hiển thị dữ liệu của trạm đo mặc định. 4a. Không có trạm nào đang hoạt động: hệ thống hiển thị thông báo “Không có dữ liệu”.
Hậu điều kiện	Người dùng xem được dữ liệu chất lượng không khí thời gian thực.

2.3.2 Đặc tả use case Xem lịch sử dữ liệu

Bảng 2.3: Đặc tả use case Xem lịch sử dữ liệu

Tên use case	Xem lịch sử dữ liệu
Tác nhân	Người dùng
Mô tả	Người dùng tra cứu dữ liệu lịch sử của một trạm đo theo thông số và khoảng thời gian mong muốn.
Tiền điều kiện	Trạm đo đã có dữ liệu đo lường trong cơ sở dữ liệu.
Luồng sự kiện chính	1. Người dùng chọn trạm đo từ danh sách. 2. Người dùng chọn thông số cần xem (AQI, PM2.5, PM10, CO₂, TVOC, nhiệt độ hoặc độ ẩm). 3. Người dùng chọn chế độ xem: theo giờ hoặc theo ngày. 4. Hệ thống truy vấn dữ liệu lịch sử từ TimescaleDB (Continuous Aggregate cho chế độ theo giờ). 5. Hệ thống hiển thị biểu đồ thể hiện xu hướng thay đổi theo thời gian.
Luồng phát sinh	4a. Không có dữ liệu trong khoảng thời gian được chọn: hệ thống hiển thị thông báo “Không có dữ liệu” và biểu đồ trống.
Hậu điều kiện	Người dùng xem được biểu đồ lịch sử dữ liệu.

2.3.3 Đặc tả use case Quản lý thiết bị

Bảng 2.4: Đặc tả use case Quản lý thiết bị

Tên use case	Quản lý thiết bị (Gateway và Sensor Node)
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện thêm, sửa, xóa và giám sát trạng thái các thiết bị trong hệ thống.
Tiền điều kiện	Quản trị viên đã đăng nhập với quyền admin (JWT hợp lệ).
Luồng sự kiện chính	1. Quản trị viên truy cập trang quản lý thiết bị. 2. Hệ thống hiển thị bảng danh sách Gateway và Sensor Node với trạng thái, thời gian gửi cuối, mức pin. 3. Quản trị viên chọn thao tác: thêm mới, sửa hoặc xóa thiết bị. 4. Hệ thống hiển thị biểu mẫu tương ứng (modal). 5. Quản trị viên nhập thông tin và xác nhận. 6. Hệ thống xác thực dữ liệu đầu vào, lưu vào cơ sở dữ liệu, và cập nhật bảng danh sách.
Luồng phát sinh	6a. Dữ liệu đầu vào không hợp lệ: hệ thống hiển thị thông báo lỗi cụ thể.
Hậu điều kiện	Danh sách thiết bị được cập nhật trong cơ sở dữ liệu. Hệ thống ghi lại thao tác vào nhật ký kiểm toán (audit log).

2.3.4 Đặc tả use case Giám sát và xử lý cảnh báo

Bảng 2.5: Đặc tả use case Giám sát và xử lý cảnh báo

Tên use case	Giám sát và xử lý cảnh báo
Tác nhân	Quản trị viên
Mô tả	Quản trị viên xem, lọc và xác nhận các cảnh báo do hệ thống tự động tạo khi phát hiện bất thường.
Tiền điều kiện	Quản trị viên đã đăng nhập. Hệ thống đã cấu hình ngưỡng cảnh báo.
Luồng sự kiện chính	1. Quản trị viên truy cập trang cảnh báo. 2. Hệ thống hiển thị danh sách cảnh báo (phân trang, mới nhất trước). 3. Quản trị viên lọc cảnh báo theo loại (tên thiết bị), mức độ (cảnh báo/nguy hiểm), trạng thái xác nhận. 4. Quản trị viên chọn một cảnh báo để xác nhận đã xử lý. 5. Hệ thống cập nhật trạng thái cảnh báo thành “đã xác nhận”.
Luồng phát sinh	2a. Khi có cảnh báo mới (realtime): hệ thống tự động hiển thị thông báo trên giao diện qua Socket.IO mà không cần tải lại trang.
Hậu điều kiện	Cảnh báo được cập nhật trạng thái trong cơ sở dữ liệu.

2.3.5 Đặc tả use case Xuất dữ liệu CSV

Bảng 2.6: Đặc tả use case Xuất dữ liệu CSV

Tên use case	Xuất dữ liệu CSV
Tác nhân	Quản trị viên
Mô tả	Quản trị viên xuất dữ liệu đo lường ra tệp CSV để phân tích ngoại tuyến hoặc lưu trữ.
Tiền điều kiện	Quản trị viên đã đăng nhập. Có dữ liệu đo lường trong cơ sở dữ liệu.
Luồng sự kiện chính	1. Quản trị viên truy cập trang xuất dữ liệu. 2. Quản trị viên chọn Sensor Node cần xuất dữ liệu. 3. Quản trị viên chọn khoảng thời gian (ngày bắt đầu, ngày kết thúc). 4. Quản trị viên nhấn nút “Xuất CSV”. 5. Hệ thống truy vấn dữ liệu từ cơ sở dữ liệu, chuyển đổi sang định dạng CSV. 6. Trình duyệt tải xuống tệp CSV.
Luồng phát sinh	5a. Không có dữ liệu trong khoảng thời gian được chọn: hệ thống hiển thị thông báo “Không có dữ liệu để xuất”.
Hậu điều kiện	Tệp CSV được tải xuống máy tính quản trị viên.

2.4 Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng đã đặc tả ở trên, hệ thống cần đáp ứng các yêu cầu phi chức năng được trình bày trong Bảng 2.7.

Bảng 2.7: Các yêu cầu phi chức năng của hệ thống

STT	Yêu cầu	Mô tả
1	Hiệu năng	Dữ liệu thời gian thực được cập nhật trên dashboard trong vòng 2 giây kể từ khi Gateway gửi dữ liệu lên server. Truy vấn lịch sử 30 ngày dữ liệu phải phản hồi trong vòng 3 giây nhờ TimescaleDB Continuous Aggregate.
2	Độ tin cậy	Hệ thống hoạt động liên tục 24/7. Cơ chế phát hiện thiết bị mất kết nối tự động (cron job định kỳ 5 phút). Gateway có bộ đệm vòng và cơ chế gửi lại (retry) khi mất kết nối server.
3	Khả năng mở rộng	Kiến trúc hub-spoke cho phép mở rộng số lượng Sensor Node mà không thay đổi phần mềm. Mỗi Gateway hỗ trợ nhiều Sensor Node. Hệ thống hỗ trợ thêm Gateway mới thông qua cơ chế tự đăng ký (provisioning).
4	Tính dễ sử dụng	Dashboard web có giao diện trực quan, responsive trên nhiều kích thước màn hình. Hệ thống hỗ trợ đa ngôn ngữ (Tiếng Việt và Tiếng Anh).
5	Bảo mật	Xác thực quản trị viên bằng JWT. Mật khẩu được mã hoá bằng bcrypt. Mọi thao tác quản trị được ghi nhận trong nhật ký kiểm toán (audit log) gồm người thực hiện, thời gian, nội dung thay đổi và địa chỉ IP.

Chương này đã khảo sát bốn hệ thống giám sát chất lượng không khí tiêu biểu, phân tích ưu nhược điểm và xác định rõ khoảng trống mà các hệ thống hiện tại chưa giải quyết được. Trên cơ sở đó, hệ thống đề xuất kết hợp cảm biến chi phí thấp, truyền thông LoRa tầm xa, mã nguồn mở và dashboard web thời gian thực. Phân tích yêu cầu đã xác định đầy đủ các chức năng cho hai tác nhân (Người dùng và Quản trị viên), đặc tả chi tiết năm use case quan trọng nhất, và định nghĩa năm yêu cầu phi chức năng. Các yêu cầu này là cơ sở để lựa chọn nền tảng công nghệ phù hợp, được trình bày trong Chương 3.

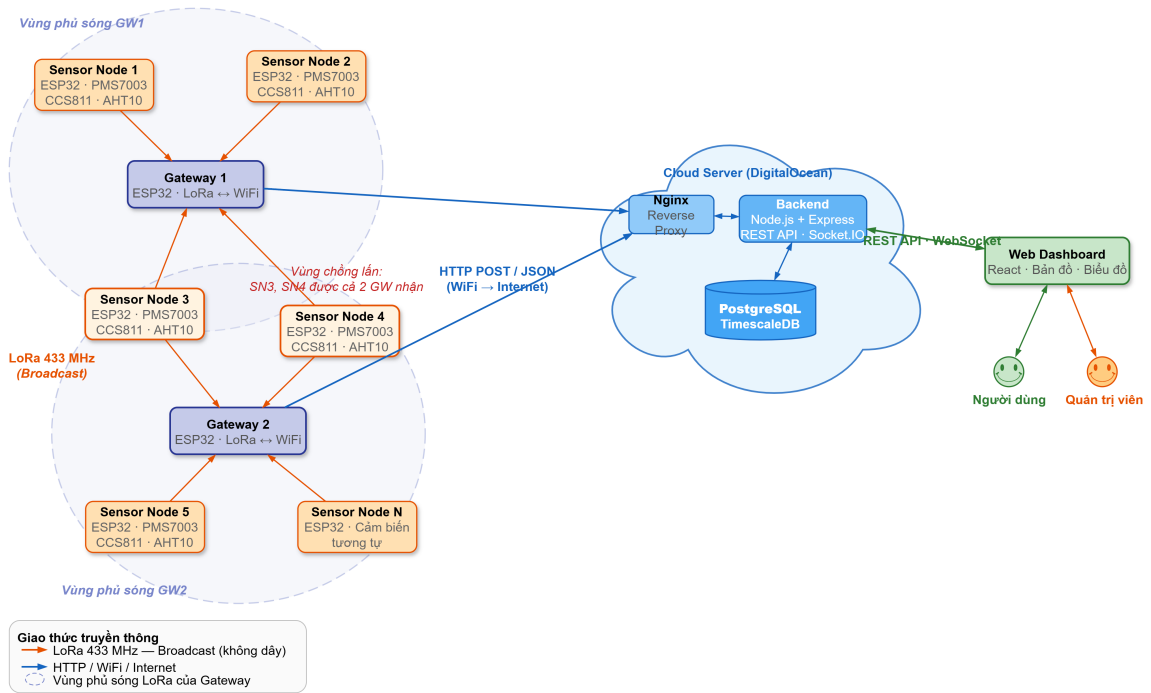
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định đầy đủ các yêu cầu chức năng và phi chức năng của hệ thống giám sát chất lượng không khí, bao gồm thu thập dữ liệu đa thông số từ cảm biến chi phí thấp, truyền dữ liệu tầm xa qua LoRa, xử lý dữ liệu chuỗi thời gian, hiển thị dashboard thời gian thực và quản trị hệ thống. Trên cơ sở đó, Chương 3 trình bày nền tảng lý thuyết và các công nghệ được lựa chọn để hiện thực hóa hệ thống. Nội dung chương bao gồm: (i) kiến trúc tổng thể trong phần 3.1, (ii) nền tảng phần cứng IoT trong phần 3.2, (iii) giao thức truyền thông LoRa trong phần 3.3, (iv) công nghệ phần mềm phía máy chủ trong phần 3.4, (v) công nghệ phần mềm phía giao diện trong phần 3.5, (vi) lý thuyết tính toán chỉ số chất lượng không khí trong phần 3.6, và (vii) hạ tầng triển khai trong phần 3.7.

3.1 Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo kiến trúc bốn tầng, bao gồm tầng cảm biến, tầng trung chuyển, tầng máy chủ và tầng hiển thị. Tầng cảm biến gồm các Sensor Node, mỗi node tích hợp vi điều khiển ESP32, các module cảm biến môi trường và module LoRa. Tầng trung chuyển là Gateway, đóng vai trò cầu nối giữa mạng LoRa và mạng Internet thông qua kết nối WiFi. Tầng máy chủ bao gồm Backend Server xử lý nghiệp vụ và cơ sở dữ liệu. Tầng hiển thị là ứng dụng web cung cấp giao diện cho người dùng và quản trị viên.

Luồng dữ liệu của hệ thống diễn ra như sau. Sensor Node đọc dữ liệu từ các cảm biến, đóng gói thành gói tin nhị phân 18 byte và phát qua LoRa. Gateway nhận gói tin, lưu vào bộ đệm vòng và gửi lên Backend Server dưới dạng HTTP POST (JSON batch). Backend Server lưu dữ liệu vào cơ sở dữ liệu TimescaleDB, tính chỉ số AQI, kiểm tra ngưỡng cảnh báo và phát sóng đến giao diện web thông qua Socket.IO. Kiến trúc này đảm bảo tính module hóa: mỗi tầng có thể phát triển và mở rộng độc lập mà không ảnh hưởng đến các tầng còn lại.



Hình 3.1: Kiến trúc tổng thể hệ thống giám sát chất lượng không khí

3.2 Nền tảng phần cứng IoT

3.2.1 Vi điều khiển ESP32

ESP32 là dòng vi điều khiển do Espressif Systems phát triển, tích hợp bộ xử lý hai nhân Xtensa LX6 32-bit hoạt động ở tần số tối đa 240 MHz, bộ nhớ SRAM 520 KB, WiFi 802.11 b/g/n và Bluetooth 4.2 [10]. Vi điều khiển này hỗ trợ đầy đủ các giao tiếp ngoại vi cần thiết cho hệ thống: UART để giao tiếp với module LoRa và cảm biến PMS7003, và I2C để giao tiếp với cảm biến CCS811, AHT10 và màn hình OLED.

Đồ án sử dụng board ESP32 DevKit V1 với framework Arduino trên nền tảng PlatformIO. Firmware của Sensor Node sử dụng kiến trúc Deep-sleep cycle thay vì chạy vòng lặp liên tục: vi điều khiển thức dậy theo chu kỳ hẹn giờ, thực hiện tuần tự các tác vụ đọc cảm biến, gửi gói LoRa và cập nhật màn hình OLED (nếu được người dùng kích hoạt), rồi quay về trạng thái Deep-sleep để tiết kiệm điện năng. Firmware của Gateway sử dụng kiến trúc superloop do thực hiện các tác vụ tuần tự: nhận gói tin LoRa, cập nhật màn hình OLED và chuyển tiếp lên Backend Server qua WiFi.

So với các lựa chọn thay thế, Arduino Mega không tích hợp WiFi nên không thể thực hiện provisioning qua web. STM32 có hiệu năng tương đương nhưng thiếu WiFi tích hợp và hệ sinh thái thư viện cảm biến ít phong phú hơn. Raspberry Pi có năng lực xử lý vượt trội nhưng chi phí cao gấp ba đến năm lần và tiêu thụ năng

lượng lớn, không phù hợp cho node cảm biến hoạt động bằng pin.

3.2.2 Module cảm biến

Hệ thống sử dụng ba loại cảm biến để đo sáu thông số môi trường theo yêu cầu đặt ra ở Chương 2. Tất cả các cảm biến đều giao tiếp qua giao tiếp nối tiếp (UART hoặc I2C), giúp đơn giản hóa thiết kế phần cứng.

PMS7003 (Plantower) là cảm biến laser đo nồng độ bụi mịn PM2.5 và PM10 trong không khí [11]. Cảm biến hoạt động theo nguyên lý tán xạ laser: luồng không khí được hút qua buồng đo, các hạt bụi đi qua chùm laser sẽ tạo ra tín hiệu tán xạ, từ đó vi xử lý tích hợp tính toán nồng độ theo đơn vị $\mu g/m^3$. PMS7003 giao tiếp qua UART với tốc độ 9600 baud, kích thước nhỏ gọn ($48 \times 37 \times 12$ mm) và chi phí khoảng 350.000 VNĐ. Lựa chọn thay thế bao gồm PMS5003 cùng hãng Plantower với thông số tương tự nhưng tỉ lệ hỏng cao hơn, và SDS011 (Nova Fitness) có kích thước lớn hơn đáng kể ($71 \times 70 \times 23$ mm).

CCS811 là cảm biến chất lượng không khí sử dụng công nghệ oxit kim loại (MOX), đo đồng thời nồng độ CO₂ tương đương (eCO₂, phạm vi 400–8192 ppm) và tổng hợp chất hữu cơ bay hơi (TVOC, phạm vi 0–1187 ppb) [12]. Cảm biến giao tiếp qua I2C, chi phí khoảng 150.000 VNĐ. Lựa chọn thay thế bao gồm SGP30 (Sensirion) có thông số tương đương và SCD30 (Sensirion) sử dụng công nghệ NDIR cho độ chính xác cao hơn nhưng chi phí đắt gấp năm lần, vượt mục tiêu chi phí thấp của đề tài.

AHT10 (Aosong Electronics) là cảm biến đo nhiệt độ (phạm vi -40 đến 85°C , sai số $\pm 0.3^\circ\text{C}$) và độ ẩm tương đối (phạm vi 0–100% RH, sai số $\pm 2\%$ RH) [13]. Cảm biến sử dụng giao tiếp I2C (địa chỉ 0x38), chia sẻ chung bus I2C với CCS811, giúp giảm số chân GPIO cần sử dụng trên ESP32. Chi phí rất thấp (khoảng 35.000 VNĐ). Lựa chọn thay thế gồm DHT22 (cùng hãng Aosong, giao tiếp 1-wire, sai số $\pm 0.5^\circ\text{C}$) và SHT31 (Sensirion) có độ chính xác cao hơn ($\pm 0.2^\circ\text{C}$) nhưng chi phí gấp năm lần.

Bảng 3.1 tổng hợp thông số kỹ thuật của các cảm biến được sử dụng trong hệ thống.

Bảng 3.1: Thông số kỹ thuật các cảm biến sử dụng trong hệ thống

Cảm biến	Thông số đo	Phạm vi	Giao tiếp	Chi phí
PMS7003	PM2.5, PM10	0–2000 $\mu g/m^3$	UART	~350.000 VNĐ
CCS811	eCO ₂ , eTVOC	400–8192 ppm, 0–1187 ppb	I2C	~150.000 VNĐ
AHT10	Nhiệt độ, độ ẩm	-40 – 85°C , 0–100% RH	I2C	~35.000 VNĐ

3.3 Giao thức truyền thông LoRa

3.3.1 Tổng quan về LoRa

LoRa (Long Range) là kỹ thuật điều chế lớp vật lý do Semtech phát triển, sử dụng phương pháp trải phổ Chirp Spread Spectrum (CSS) trên băng tần sub-GHz không cần cấp phép (433 MHz, 868 MHz hoặc 915 MHz tùy khu vực) [8]. Đặc điểm nổi bật của LoRa là khả năng truyền tầm xa (5–15 km trong điều kiện tầm nhìn thẳng) với mức tiêu thụ năng lượng rất thấp, tuy nhiên tốc độ truyền dữ liệu bị giới hạn (0,3–37,5 kbps). Đặc tính này phù hợp với bài toán giám sát môi trường, nơi khối lượng dữ liệu nhỏ (18 byte mỗi gói tin) nhưng yêu cầu phạm vi phủ sóng rộng.

Cần phân biệt LoRa (lớp vật lý) với LoRaWAN (giao thức mạng). LoRaWAN định nghĩa kiến trúc mạng và giao thức truyền thông trên nền LoRa, bao gồm quản lý thiết bị, mã hóa và kiến trúc hình sao. Đồ án sử dụng LoRa ở chế độ point-to-point (không qua LoRaWAN) để đơn giản hóa kiến trúc và giảm chi phí, vì hệ thống chỉ cần truyền dữ liệu từ Sensor Node đến Gateway trong phạm vi một khu vực (campus hoặc khu đô thị).

Kết quả khảo sát ở Chương 2 cho thấy tất cả bốn hệ thống hiện tại (PAM Air, IQAir, PurpleAir, VN-AQI) đều sử dụng WiFi hoặc 4G, yêu cầu hạ tầng mạng tại mỗi điểm đo. LoRa giải quyết hạn chế này bằng cách cho phép truyền dữ liệu tầm xa mà không cần WiFi tại mỗi node. Các lựa chọn thay thế bao gồm: (i) WiFi có tầm phủ ngắn (dưới 100 m) và cần điểm truy cập tại mỗi vị trí, (ii) Zigbee có tầm phủ ngắn (dưới 100 m) dù tiêu thụ năng lượng thấp, (iii) NB-IoT và 4G yêu cầu SIM và phí dữ liệu hàng tháng, và (iv) Sigfox phụ thuộc vào nhà mạng Sigfox chưa phủ sóng tại Việt Nam.

3.3.2 Module AS32-TTL-100

Đồ án sử dụng module AS32-TTL-100 dựa trên chip SX1278 (Semtech), hoạt động ở tần số 433 MHz ISM band với công suất phát 100 mW (20 dBm). Module giao tiếp với vi điều khiển qua UART ở chế độ transparent — nghĩa là dữ liệu ghi vào cổng UART sẽ được module tự động điều chế và phát qua sóng LoRa, và ngược lại. Chế độ này đơn giản hóa đáng kể firmware so với việc sử dụng chip SX1278 trực tiếp qua SPI, vốn yêu cầu cấu hình thành ghi phức tạp.

Module hỗ trợ bốn chế độ hoạt động được điều khiển qua hai chân MD0 và MD1: chế độ truyền bình thường (MD0=0, MD1=0), chế độ tiết kiệm năng lượng, chế độ cấu hình và chế độ ngủ (MD0=1, MD1=1). Khoảng cách truyền lý thuyết đạt 3–5 km trong môi trường đô thị và trên 10 km trong điều kiện tầm nhìn thẳng.

3.4 Công nghệ phần mềm phía máy chủ

3.4.1 Node.js và Express

Node.js là môi trường thực thi JavaScript phía máy chủ, xây dựng trên engine V8 của Google Chrome [14]. Đặc điểm quan trọng nhất của Node.js là mô hình event-driven, non-blocking I/O: thay vì tạo luồng mới cho mỗi kết nối (như mô hình truyền thống), Node.js sử dụng vòng lặp sự kiện đơn luồng để xử lý đồng thời hàng nghìn kết nối. Mô hình này phù hợp với hệ thống giám sát thời gian thực, nơi Backend cần duy trì nhiều kết nối Socket.IO đồng thời và xử lý dữ liệu telemetry liên tục từ các Gateway.

Express là framework web phổ biến nhất cho Node.js, cung cấp hệ thống middleware, routing và xử lý HTTP request/response. Đồ án sử dụng Express để xây dựng REST API cho các chức năng CRUD thiết bị, truy vấn dữ liệu, xuất CSV và quản lý tài khoản. Các middleware bảo mật bao gồm Helmet (thiết lập HTTP security headers), CORS (kiểm soát truy cập cross-origin) và xác thực JWT.

So với các lựa chọn thay thế: Python kết hợp Flask hoặc FastAPI có hiệu năng hạn chế hơn cho kịch bản real-time do mô hình đồng thời khác biệt. Java kết hợp Spring Boot có kiến trúc vững chắc nhưng chi phí tài nguyên và độ phức tạp cao cho dự án quy mô nhỏ. Go kết hợp Gin có hiệu năng vượt trội nhưng hệ sinh thái thư viện nhỏ hơn đáng kể.

3.4.2 PostgreSQL, TimescaleDB và PostGIS

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, tuân thủ ACID và hỗ trợ hệ thống extension phong phú [15]. Đồ án mở rộng PostgreSQL bằng hai extension chuyên biệt: TimescaleDB và PostGIS.

TimescaleDB là extension chuyển đổi PostgreSQL thành cơ sở dữ liệu chuỗi thời gian (time-series), được thiết kế cho dữ liệu có nhãn thời gian liên tục [16]. Đồ án sử dụng ba tính năng chính của TimescaleDB. Thứ nhất, *Hypertable*: TimescaleDB tự động phân mảnh (partition) bảng dữ liệu theo thời gian, giúp tối ưu hiệu năng ghi và truy vấn khi lượng dữ liệu tăng liên tục. Thứ hai, *Continuous Aggregate*: materialized view được tự động cập nhật khi có dữ liệu mới, cho phép truy vấn lịch sử tổng hợp với hiệu năng cao mà không cần tính toán lại từ dữ liệu thô — đáp ứng yêu cầu phi chức năng truy vấn 30 ngày trong vòng 3 giây (Bảng 2.7). Thứ ba, *Retention Policy*: tự động xóa dữ liệu cũ theo chu kỳ cấu hình để kiểm soát dung lượng lưu trữ.

PostGIS là extension bổ sung kiểu dữ liệu không gian và các hàm xử lý địa lý cho PostgreSQL [17]. PostGIS cho phép lưu trữ tọa độ GPS dưới dạng đối tượng

hình học và thực hiện các phép tính khoảng cách địa lý, từ đó hỗ trợ truy vấn trạm đo gần nhất dựa trên vị trí người dùng — đáp ứng yêu cầu chức năng tự động gợi ý trạm gần nhất trên dashboard (Bảng 2.2). Chi tiết cách áp dụng TimescaleDB và PostGIS vào thiết kế cơ sở dữ liệu sẽ được trình bày trong Chương 4.

So với các lựa chọn thay thế: InfluxDB chuyên biệt cho time-series nhưng không hỗ trợ dữ liệu không gian, buộc phải sử dụng hai cơ sở dữ liệu riêng biệt. MongoDB là cơ sở dữ liệu NoSQL linh hoạt nhưng không tối ưu cho truy vấn time-series aggregate. MySQL thiếu extension tương đương TimescaleDB và PostGIS.

3.4.3 Prisma ORM

Prisma là ORM thế hệ mới cho Node.js, áp dụng phương pháp schema-first: lược đồ cơ sở dữ liệu được định nghĩa trong một file khai báo, từ đó Prisma Client tự động sinh ra các hàm truy vấn type-safe [18].

Ưu điểm chính của Prisma so với viết SQL thủ công là giảm lỗi runtime nhờ kiểm tra kiểu tại thời điểm biên dịch, hỗ trợ migration tự động khi thay đổi lược đồ, và cú pháp truy vấn trực quan. Ngoài ra, Prisma cho phép kết hợp với raw SQL khi cần sử dụng các tính năng đặc thù của extension (TimescaleDB, PostGIS) mà ORM chưa hỗ trợ trực tiếp.

3.4.4 Socket.IO

Socket.IO là thư viện giao tiếp thời gian thực hai chiều giữa máy chủ và trình duyệt, xây dựng trên nền WebSocket với cơ chế fallback về HTTP long-polling khi WebSocket không khả dụng [19]. Đề án sử dụng Socket.IO cho hai chức năng: (i) phát sóng (broadcast) dữ liệu đo mới đến tất cả giao diện dashboard khi Backend nhận dữ liệu từ Gateway, và (ii) phát thông báo cảnh báo thời gian thực đến giao diện quản trị khi phát hiện giá trị vượt ngưỡng.

So với WebSocket thuần, Socket.IO cung cấp thêm cơ chế tự động kết nối lại (auto-reconnect), hệ thống phòng (room) để phát sóng có chọn lọc và xử lý graceful degradation. So với Server-Sent Events (SSE), Socket.IO hỗ trợ giao tiếp hai chiều, cần thiết cho các tính năng tương tác trong tương lai.

3.5 Công nghệ phần mềm phía giao diện

3.5.1 React và Vite

React là thư viện xây dựng giao diện người dùng do Meta phát triển, dựa trên kiến trúc component — mỗi thành phần giao diện là một đơn vị độc lập, có thể tái sử dụng và kết hợp để tạo nên giao diện phức tạp [20]. React sử dụng Virtual DOM để tối ưu hiệu năng cập nhật giao diện: khi trạng thái thay đổi, React so sánh Virtual DOM mới với phiên bản trước, chỉ cập nhật các phần tử thực sự thay đổi

trên DOM thật. Cơ chế này phù hợp với dashboard thời gian thực, nơi dữ liệu liên tục được cập nhật qua Socket.IO nhưng chỉ một số thành phần giao diện cần vẽ lại.

Vite là công cụ build thể hệ mới, sử dụng ES Module trên trình duyệt trong quá trình phát triển và Rollup để đóng gói sản phẩm [21]. Vite cung cấp Hot Module Replacement (HMR) gần như tức thì, tăng tốc đáng kể quy trình phát triển so với Webpack truyền thống.

So với các lựa chọn thay thế: Vue.js nhẹ hơn nhưng hệ sinh thái thư viện bên thứ ba nhỏ hơn. Angular là framework toàn diện nhưng có đường cong học tập dốc và chi phí tài nguyên cao cho dự án quy mô nhỏ. Svelte có hiệu năng runtime tốt nhưng hệ sinh thái chưa trưởng thành.

3.5.2 Thư viện trực quan hóa dữ liệu

Đồ án sử dụng hai thư viện trực quan hóa chính. **Leaflet** là thư viện bản đồ tương tác mã nguồn mở, nhẹ (khoảng 42 KB gzip) và hỗ trợ đầy đủ các tính năng bản đồ: marker, popup, zoom và lớp phủ [22]. Đồ án sử dụng Leaflet kết hợp React-Leaflet để hiển thị bản đồ AQI với marker mã màu theo chuẩn US EPA tại mỗi trạm đo. So với Google Maps API (phải trả phí sau số lượng request nhất định) và Mapbox (freemium, yêu cầu API key), Leaflet hoàn toàn miễn phí và sử dụng tile từ OpenStreetMap.

Apache ECharts là thư viện biểu đồ hiệu năng cao, hỗ trợ hơn 20 loại biểu đồ và khả năng xử lý tập dữ liệu lớn [23]. Đồ án sử dụng ECharts thông qua wrapper `echarts-for-react` để hiển thị biểu đồ lịch sử dữ liệu theo giờ và theo ngày. So với Chart.js (nhẹ hơn nhưng ít tính năng nâng cao), ECharts hỗ trợ tốt hơn cho responsive tương tác phức tạp.

3.6 Lý thuyết tính toán chỉ số chất lượng không khí

3.6.1 Khái niệm và phân loại mức AQI

Chỉ số chất lượng không khí (AQI) là thước đo tổng hợp mức độ ô nhiễm không khí, được US EPA định nghĩa trên thang điểm từ 0 đến 500 [9]. Giá trị AQI càng cao thì mức ô nhiễm càng nghiêm trọng và ảnh hưởng sức khỏe càng lớn. US EPA phân chia thang AQI thành sáu mức như trình bày trong Bảng 3.2.

Bảng 3.2: Phân loại các mức chỉ số AQI theo tiêu chuẩn US EPA [9]

AQI	Mức độ	Mã màu	Ảnh hưởng sức khỏe
0–50	Tốt (Good)	Xanh lá	Không ảnh hưởng
51–100	Trung bình (Moderate)	Vàng	Nhóm nhạy cảm nên hạn chế
101–150	Không tốt cho nhóm nhạy cảm	Cam	Nhóm nhạy cảm bị ảnh hưởng
151–200	Có hại (Unhealthy)	Đỏ	Mọi người bắt đầu bị ảnh hưởng
201–300	Rất có hại (Very Unhealthy)	Tím	Cảnh báo sức khỏe nghiêm trọng
301–500	Nguy hiểm (Hazardous)	Nâu đỏ	Toàn bộ dân số bị ảnh hưởng

3.6.2 Công thức và bảng điểm ngắt

Đồ án tính AQI cho từng chất ô nhiễm dạng hạt (PM_{2.5}, PM₁₀) theo công thức nội suy tuyến tính của US EPA:

$$I_p = \frac{I_{Hi} - I_{Lo}}{BP_{Hi} - BP_{Lo}} \times (C_p - BP_{Lo}) + I_{Lo} \quad (3.1)$$

Trong đó I_p là giá trị AQI của chất ô nhiễm p , C_p là nồng độ đo được (đơn vị $\mu g/m^3$), BP_{Hi} và BP_{Lo} là các điểm ngắt nồng độ (breakpoint) trên và dưới chứa C_p , I_{Hi} và I_{Lo} là giá trị AQI tương ứng với các điểm ngắt đó. Chỉ số AQI tổng hợp được xác định bằng giá trị lớn nhất trong các AQI thành phần: $AQI = \max(I_{PM2.5}, I_{PM10})$.

Bảng 3.3 và Bảng 3.4 trình bày các điểm ngắt nồng độ cho PM_{2.5} và PM₁₀ theo tiêu chuẩn US EPA, được hệ thống sử dụng trực tiếp trong module tính toán trên Backend Server.

Bảng 3.3: Bảng điểm ngắt AQI cho PM_{2.5} [9]

$BP_{Lo} (\mu g/m^3)$	$BP_{Hi} (\mu g/m^3)$	I_{Lo}	I_{Hi}
0,0	12,0	0	50
12,1	35,4	51	100
35,5	55,4	101	150
55,5	150,4	151	200
150,5	250,4	201	300
250,5	500,4	301	500

Bảng 3.4: Bảng điểm ngắt AQI cho PM10 [9]

$BP_{Lo} (\mu g/m^3)$	$BP_{Hi} (\mu g/m^3)$	I_{Lo}	I_{Hi}
0	54	0	50
55	154	51	100
155	254	101	150
255	354	151	200
355	424	201	300
425	604	301	500

3.6.3 Ví dụ tính toán

Giả sử cảm biến đo được $C_{PM2.5} = 45,3 \mu g/m^3$ và $C_{PM10} = 82 \mu g/m^3$. Quy trình tính AQI diễn ra như sau.

Đối với PM2.5, tra Bảng 3.3: giá trị 45,3 nằm trong khoảng $[35,5; 55,4]$, tương ứng $I_{Lo} = 101$, $I_{Hi} = 150$, $BP_{Lo} = 35,5$, $BP_{Hi} = 55,4$. Thay vào công thức (3.1):

$$I_{PM2.5} = \frac{150 - 101}{55,4 - 35,5} \times (45,3 - 35,5) + 101 = \frac{49}{19,9} \times 9,8 + 101 \approx 125$$

Đối với PM10, tra Bảng 3.4: giá trị 82 nằm trong khoảng $[55; 154]$, tương ứng $I_{Lo} = 51$, $I_{Hi} = 100$, $BP_{Lo} = 55$, $BP_{Hi} = 154$. Thay vào công thức (3.1):

$$I_{PM10} = \frac{100 - 51}{154 - 55} \times (82 - 55) + 51 = \frac{49}{99} \times 27 + 51 \approx 64$$

Chỉ số AQI tổng hợp là $AQI = \max(125, 64) = 125$, thuộc mức “Không tốt cho nhóm nhạy cảm” (Bảng 3.2). Kết quả này cho thấy PM2.5 là chất ô nhiễm chính quyết định mức AQI, phù hợp với thực tế tại các đô thị Việt Nam, nơi bụi mịn PM2.5 thường là tác nhân ô nhiễm chủ đạo.

Hệ thống áp dụng quy trình tính toán này trên Backend Server mỗi khi nhận dữ liệu mới từ Gateway, đồng thời ánh xạ kết quả sang mã màu tương ứng (Bảng 3.2) để hiển thị trên bản đồ và dashboard.

3.7 Hạ tầng triển khai

3.7.1 Docker và Docker Compose

Docker là nền tảng container hóa cho phép đóng gói ứng dụng cùng toàn bộ phụ thuộc (thư viện, runtime, cấu hình) vào một đơn vị triển khai nhất quán gọi là container [24]. Docker Compose mở rộng Docker cho ứng dụng đa container, cho phép định nghĩa và quản lý toàn bộ hệ thống trong một file `docker-compose.yml`.

Đồ án container hóa bốn thành phần: (i) cơ sở dữ liệu PostgreSQL/TimescaleDB, (ii) Backend Server (Node.js), (iii) Nginx phục vụ static files và reverse proxy, và (iv) Certbot quản lý chứng chỉ SSL. Việc container hóa đảm bảo môi trường nhất quán giữa phát triển và sản xuất, đơn giản hóa quy trình triển khai trên VPS chỉ với một lệnh duy nhất. So với triển khai trực tiếp trên VPS (cài đặt phụ thuộc thủ công, khó tái tạo), Docker loại bỏ hoàn toàn vấn đề “hoạt động trên máy em nhưng không hoạt động trên server”. So với Kubernetes, Docker Compose đơn giản và phù hợp hơn cho hệ thống quy mô nhỏ chạy trên một máy chủ duy nhất.

3.7.2 Nginx

Nginx đóng vai trò reverse proxy, chuyển tiếp các yêu cầu HTTP/WebSocket từ client đến Backend Server, đồng thời phục vụ các tệp tĩnh (static files) của Frontend đã được build bởi Vite. Kiến trúc này giúp Backend Server không bị lộ trực tiếp ra Internet, tăng cường bảo mật và cho phép cấu hình HTTPS tập trung tại Nginx.

Chương này đã trình bày nền tảng lý thuyết và các công nghệ được lựa chọn cho từng tầng trong kiến trúc hệ thống. Ở tầng cảm biến, vi điều khiển ESP32 kết hợp ba module cảm biến (PMS7003, CCS811, AHT10) cho phép đo sáu thông số môi trường với chi phí thấp. Tầng trung chuyển sử dụng giao thức LoRa qua module AS32-TTL-100 giải quyết bài toán truyền dữ liệu tầm xa mà không cần hạ tầng WiFi tại mỗi điểm đo. Ở tầng máy chủ, Node.js kết hợp TimescaleDB và Socket.IO đáp ứng yêu cầu xử lý dữ liệu chuỗi thời gian và cập nhật thời gian thực. Ở tầng hiển thị, React kết hợp Leaflet và ECharts cung cấp dashboard trực quan. Toàn bộ hệ thống được container hóa bằng Docker để đơn giản hóa triển khai. Với các công nghệ đã lựa chọn, Chương 4 sẽ trình bày chi tiết thiết kế kiến trúc, triển khai các chức năng và đánh giá kết quả thực nghiệm.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 3 đã trình bày nền tảng lý thuyết và các công nghệ được lựa chọn cho từng tầng trong kiến trúc hệ thống. Trên cơ sở đó, Chương 4 trình bày chi tiết quá trình thiết kế, triển khai và đánh giá hệ thống. Nội dung chương bao gồm thiết kế kiến trúc (phần 4.1), thiết kế chi tiết (phần 4.2), xây dựng ứng dụng (phần 4.3), kiểm thử (phần 4.4), và triển khai (phần 4.5).

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phân mềm

Hệ thống được thiết kế theo kiến trúc bốn tầng, bao gồm tầng cảm biến (Sensor Layer), tầng trung chuyển (Gateway Layer), tầng máy chủ (Server Layer) và tầng hiển thị (Presentation Layer). Kiến trúc bốn tầng được lựa chọn vì ba lý do chính. Thứ nhất, mỗi tầng có trách nhiệm rõ ràng và có thể phát triển độc lập: tầng cảm biến thu thập dữ liệu môi trường, tầng trung chuyển chuyển tiếp dữ liệu từ mạng LoRa sang Internet, tầng máy chủ xử lý nghiệp vụ và lưu trữ, tầng hiển thị cung cấp giao diện cho người dùng. Thứ hai, kiến trúc này cho phép mở rộng theo chiều ngang tại từng tầng — ví dụ thêm Sensor Node mới mà không thay đổi phần mềm Backend. Thứ ba, độ phức tạp phù hợp với quy mô hệ thống: triển khai trên một máy chủ VPS duy nhất, không yêu cầu kiến trúc Microservice phức tạp.

Tầng cảm biến gồm các Sensor Node, mỗi node tích hợp vi điều khiển ESP32, ba module cảm biến (PMS7003, CCS811, AHT10) và module LoRa AS32-TTL-100. Firmware được thiết kế tối ưu điện năng bằng cơ chế Deep-sleep: thiết bị thức dậy theo chu kỳ để đọc cảm biến và gửi LoRa, chỉ hiển thị OLED khi người dùng nhấn nút. Dữ liệu được đóng gói thành gói tin nhị phân 18 byte và phát qua LoRa 433 MHz.

Tầng trung chuyển gồm các Gateway, mỗi Gateway tích hợp ESP32 với module LoRa và kết nối WiFi. Gateway nhận gói tin LoRa, lưu vào bộ đệm vòng (ring buffer), sau đó gửi lên tầng máy chủ dưới dạng HTTP POST (JSON batch). Ở tầng này, hệ thống áp dụng mô hình Hub-Spoke: nhiều Sensor Node (spoke) gửi dữ liệu đến một Gateway (hub) trung tâm, cho phép một Gateway phục vụ nhiều Sensor Node trong phạm vi phủ sóng mà không cần định tuyến phức tạp.

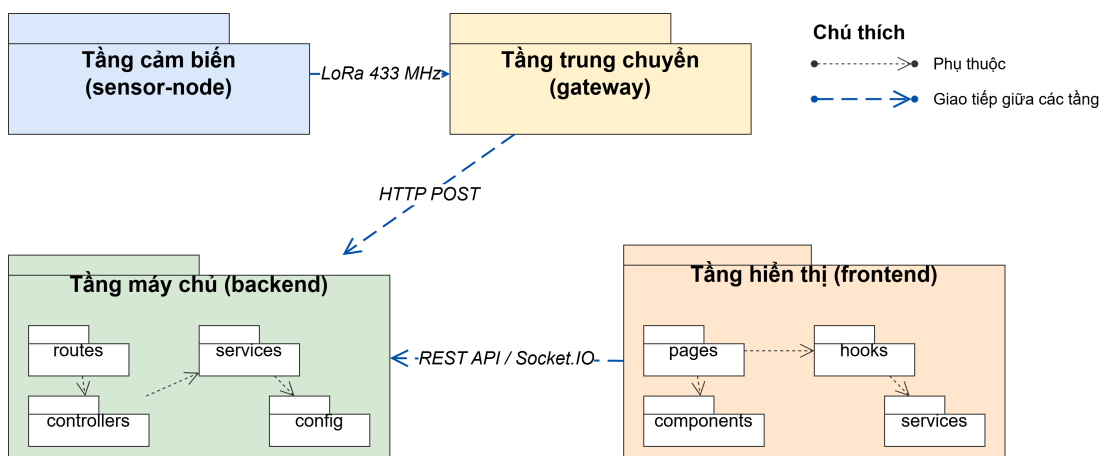
Tầng máy chủ bao gồm Backend Server (Node.js/Express) và cơ sở dữ liệu (PostgreSQL với TimescaleDB và PostGIS), được đóng gói trong Docker container. Backend Server được tổ chức theo mô hình phân lớp gồm ba lớp: lớp Controller

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

tiếp nhận HTTP request, lớp Service chứa logic nghiệp vụ, và lớp Data Access tương tác với cơ sở dữ liệu thông qua Prisma ORM. Ngoài ra, tầng máy chủ còn có Nginx đóng vai trò reverse proxy và phục vụ tệp tĩnh, cùng các tác vụ nền (cron job) kiểm tra thiết bị offline và dọn dẹp dữ liệu cũ.

Tầng hiển thị là ứng dụng web SPA xây dựng bằng React 19 và Vite 8, giao tiếp với tầng máy chủ qua REST API (HTTP/HTTPS) và Socket.IO (WebSocket). Tầng này cung cấp hai nhóm giao diện: dashboard cho người dùng (bản đồ AQI, biểu đồ lịch sử, xếp hạng) và trang quản trị cho quản trị viên (quản lý thiết bị, cảnh báo, cấu hình, xuất dữ liệu).

4.1.2 Thiết kế tổng quan



Hình 4.1: Biểu đồ gói UML tổng quan của hệ thống

Hệ thống được tổ chức thành bốn nhóm gói chính tương ứng với bốn tầng kiến trúc, mỗi nhóm chứa các gói con với trách nhiệm cụ thể.

Nhóm gói Tầng cảm biến (sensor-node) chứa các module: drivers (giao tiếp cảm biến PMS7003, CCS811, AHT10, màn hình OLED, module LoRa), core (quản lý cấu hình NVS, provisioning qua WiFi AP, quản lý chu kỳ deep-sleep), và config (định nghĩa chân GPIO, tham số LoRa).

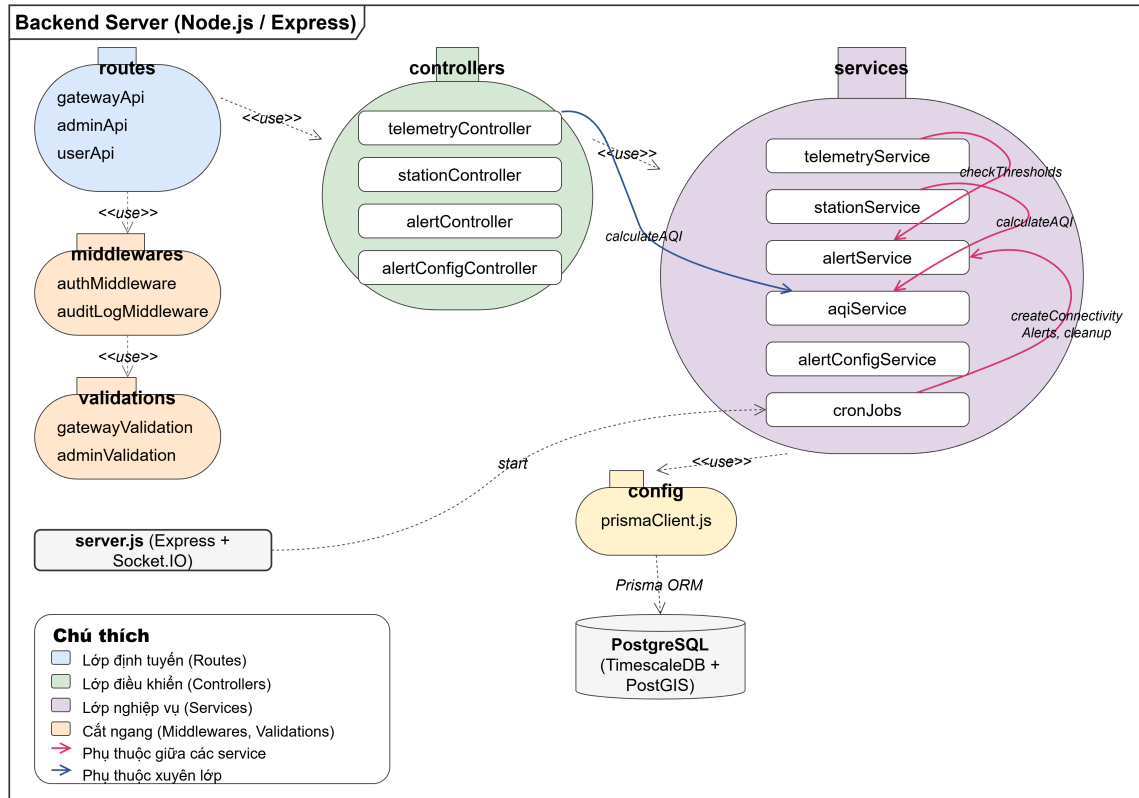
Nhóm gói Tầng trung chuyển (gateway) chứa các module: drivers (giao tiếp LoRa), core (bộ đệm vòng, gửi HTTP batch lên tầng máy chủ), và config (cấu hình WiFi, địa chỉ Backend). Nhóm gói này hoạt động độc lập, không phụ thuộc vào nhóm gói tầng cảm biến.

Nhóm gói Tầng máy chủ (backend) gồm bốn gói chính: routes (định nghĩa endpoint API), controllers (tiếp nhận request, trả response), services (logic nghiệp vụ), và config (kết nối Prisma Client và cấu hình hệ thống). Luồng phụ thuộc tuân thủ nguyên tắc một chiều: routes → controllers → services → config, đảm bảo gói tầng dưới không phụ thuộc ngược lại gói tầng trên.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Nhóm gói Tầng hiển thị (frontend) gồm bốn gói chính: `pages` (các trang giao diện), `components` (thành phần tái sử dụng), `hooks` (custom React hooks để nạp dữ liệu và xử lý logic), và `services` (giao tiếp API). Mô hình phụ thuộc được thiết kế rõ ràng: gói `pages` phụ thuộc vào `components` và `hooks`; gói `hooks` phụ thuộc vào `services`; gói `services` chịu trách nhiệm gọi REST API và giao tiếp Socket.IO đến Tầng máy chủ.

4.1.3 Mô tả luồng dữ liệu giữa các gói



Hình 4.2: Biểu đồ quan hệ module cho backend services

Hình 4.2 mô tả quan hệ phụ thuộc giữa các module trên Backend. Hai nhóm module quan trọng nhất là nhóm xử lý dữ liệu telemetry và nhóm hiển thị dashboard.

Nhóm xử lý dữ liệu telemetry bao gồm bốn module: `telemetryController` tiếp nhận request, `telemetryValidation` xác thực đầu vào, `telemetryService` xử lý nghiệp vụ (deduplication, lưu trữ, kiểm tra cảnh báo), và `aqiService` tính chỉ số AQI. Luồng xử lý chi tiết được trình bày qua ba biểu đồ trình tự ở phần 4.2.4.

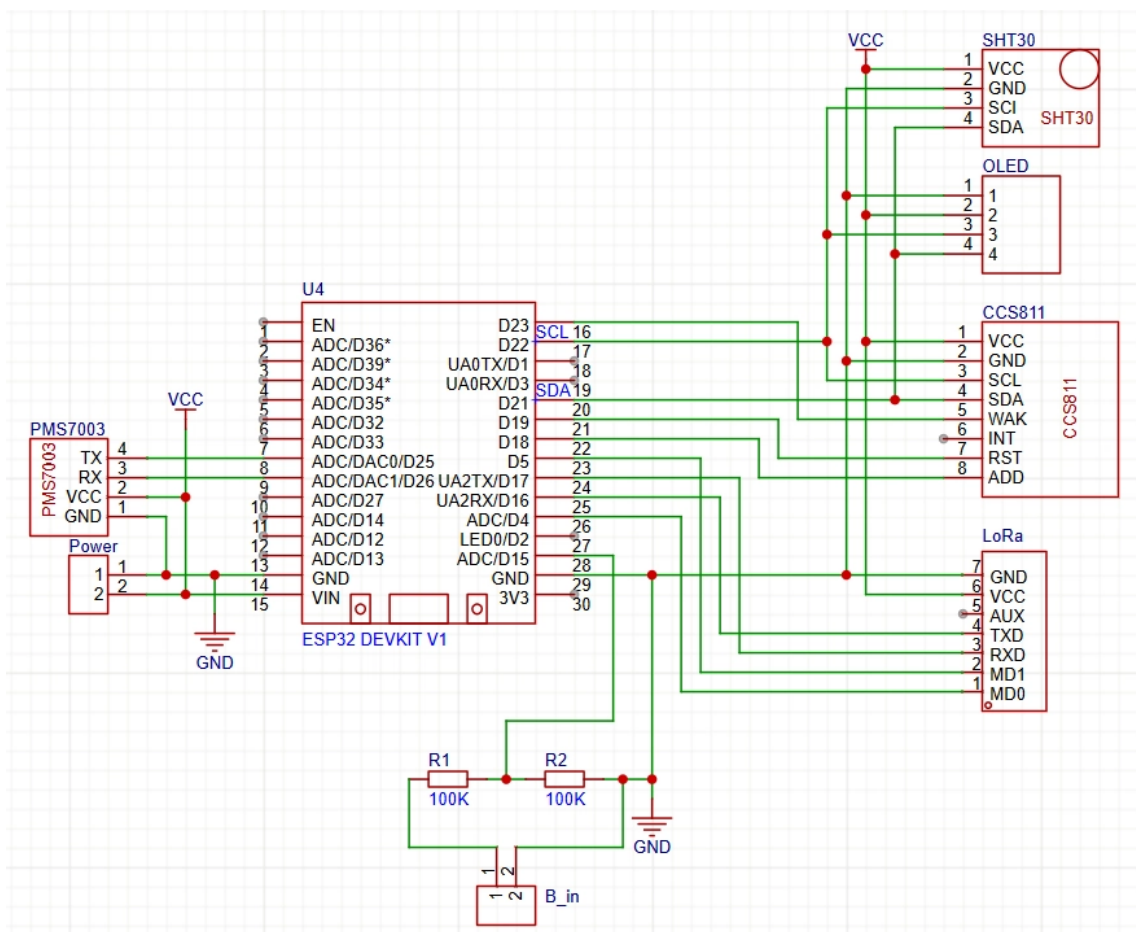
Nhóm hiển thị dashboard trên Frontend sử dụng hook `useDashboard` (gọi API lấy dữ liệu ban đầu) và hook `useSocket` (nhận cập nhật thời gian thực qua Socket.IO). Chi tiết luồng end-to-end từ truy cập Dashboard đến cập nhật realtime được mô tả ở Hình 4.7.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế phần cứng

Phần này trình bày thiết kế mạch kết nối phần cứng cho Sensor Node và Gateway.

Sensor Node sử dụng board ESP32 DevKit V1 (30 chân) làm trung tâm, kết nối với bốn module ngoại vi qua ba giao tiếp khác nhau. Module LoRa AS32-TTL-100 giao tiếp qua UART1 (RX: GPIO16, TX: GPIO17) với hai chân điều khiển chế độ (MD0: GPIO4, MD1: GPIO5). Module cảm biến bụi PMS7003 giao tiếp qua UART2 (RX: GPIO25, TX: GPIO26). Module CCS811 và AHT10 chia sẻ chung bus I2C (SDA: GPIO21, SCL: GPIO22) với địa chỉ lần lượt là 0x5A và 0x38; CCS811 có thêm chân WAK (GPIO23) để đánh thức từ chế độ ngủ. Màn hình OLED SSD1306 cũng dùng chung bus I2C trên, địa chỉ 0x3C. Pin 18650 được đo qua mạch phân áp (voltage divider R1=R2=100K Ω) nối vào GPIO15 (ADC). Nút BOOT (GPIO0) được tận dụng làm nút factory reset: giữ 5 giây để xóa cấu hình và vào chế độ provisioning.

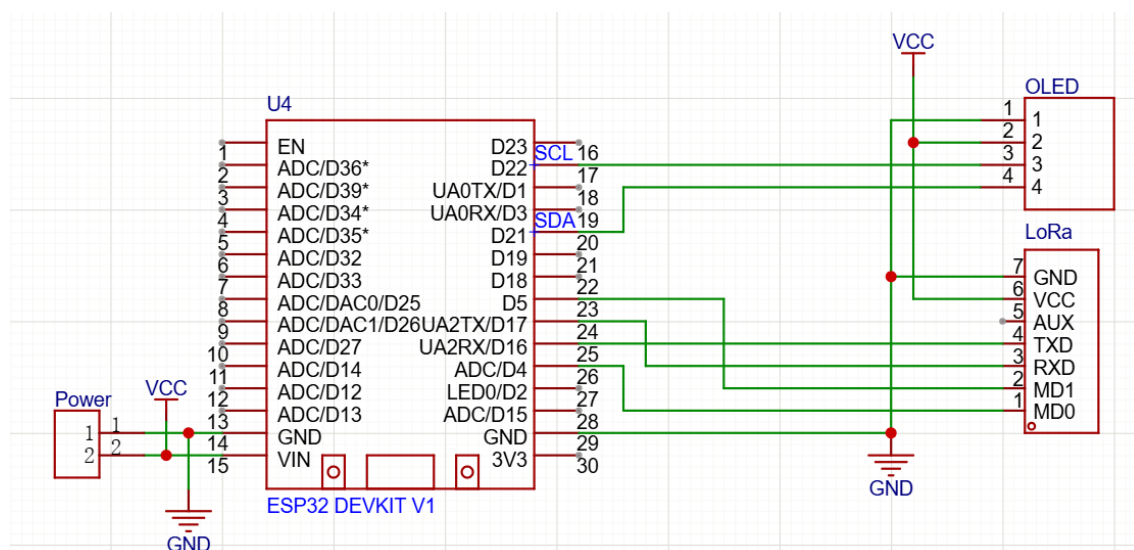


Hình 4.3: Sơ đồ mạch Sensor Node

Gateway sử dụng cùng board ESP32 DevKit V1 và kết nối với hai module

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

ngoại vi: module LoRa AS32-TTL-100 qua UART1 (cùng cấu hình chân với Sensor Node), và màn hình OLED SSD1306 qua I2C để hiển thị trạng thái hoạt động.



Hình 4.4: Sơ đồ mạch Gateway

4.2.2 Thiết kế firmware

Phần này trình bày thiết kế firmware cho Sensor Node và Gateway.

Firmware Sensor Node được thiết kế tối ưu năng lượng dựa trên cơ chế Deep-sleep của hệ điều hành quản lý ESP32. Thay vì chạy vòng lặp liên tục, vi điều khiển chỉ thức dậy theo chu kỳ hẹn giờ Timer hoặc qua ngắt bằng nút cứng (nút BOOT). Khi được đánh thức bởi Timer, firmware thực hiện tuần tự các tác vụ khởi tạo phần cứng, gọi hệ thống lấy mẫu cảm biến, chờ cấp nhiệt (warm-up) cảm biến bụi thư viện, lấy mẫu pin qua ADC, đóng gói tải trọng dữ liệu gửi qua LoRa, và cuối cùng đối chiếu lưu trữ lại kết quả vừa gửi vào vùng nhớ RTC trước khi quay về lại trạng thái Deep-sleep. Ngược lại, nếu được đánh thức bởi người dùng, thiết bị bỏ qua quá trình lấy mẫu, mà truy cập vùng nhớ RTC để hiển thị dữ liệu lịch sử ngay lên màn hình OLED trong vòng 10 giây trước khi tiếp tục tắt hiển thị. Kiến trúc này giúp ngắt triệt để điện năng của các module ngoại vi, triệt tiêu hao phí thừa so với cơ chế FreeRTOS thông thường.

Cấu trúc gói tin LoRa được định nghĩa dưới dạng struct C packed (không có padding), tổng kích thước 18 byte. Gói tin gồm: `nodeId` (1 byte, ID node 1–255), `pktType` (1 byte, phân biệt gói dữ liệu/heartbeat/lỗi), `msgId` (1 byte, bộ đếm tuần hoàn 0–255 phục vụ deduplication), bảy trường dữ liệu cảm biến (PM1, PM2.5, PM10 mỗi trường 2 byte nhân 10, CO₂ và TVOC mỗi trường 2 byte, nhiệt độ 2 byte có dấu nhân 10, độ ẩm 2 byte nhân 10), và `battery` (1 byte, 0–100%). Các giá trị dạng số thực được nhân 10 và lưu dạng số nguyên để tránh sử dụng floating-point trong truyền thông, giảm kích thước gói tin. Giá trị sentinel (0xFFFF cho

unsigned, 0x7FFF cho signed) đánh dấu cảm biến bị lỗi.

Cấu trúc gói tin LoRa (18 bytes)

nodeId	pktType	msgId	pm1	pm25	pm10	co2	tvoc	temperature	humidity	battery
1B	1B	1B	2B	2B	2B	2B	2B	2B	2B	1B

Tổng kích thước: 18 Bytes

Hình 4.5: Cấu trúc gói tin LoRa

Cơ chế provisioning cho phép cấu hình Node ID qua giao diện web. Khi chưa có cấu hình (hoặc sau factory reset), Sensor Node khởi động WiFi AP với SSID “AirQuality-SN-Setup”, phục vụ trang web captive portal tại địa chỉ 192.168.4.1. Người dùng nhập Node ID, firmware lưu vào bộ nhớ NVS (Non-Volatile Storage) và khởi động lại ở chế độ hoạt động bình thường.

Firmware Gateway sử dụng kiến trúc superloop (không dùng FreeRTOS) do chỉ thực hiện các tác vụ tuần tự: nhận gói tin LoRa từ UART, cập nhật trạng thái lên màn hình OLED và tích lũy vào bộ đệm vòng (ring buffer). Khi đủ số lượng gói hoặc hết chu kỳ thời gian, Gateway serialize bộ đệm thành JSON array và gửi lên tầng máy chủ qua HTTP POST. Gateway cũng hỗ trợ cơ chế self-provisioning: tự đăng ký với Backend Server khi kết nối lần đầu.

4.2.3 Thiết kế giao diện

Giao diện hệ thống được thiết kế cho trình duyệt web, hỗ trợ responsive trên nhiều kích thước màn hình từ máy tính (độ phân giải tối thiểu 1280×720) đến thiết bị di động (tối thiểu 375×667). Hệ thống hỗ trợ hai ngôn ngữ (Tiếng Việt và Tiếng Anh) thông qua thư viện `react-i18next`.

Giao diện được tổ chức thành hai nhóm layout riêng biệt. **UserLayout** dành cho người dùng, gồm thanh điều hướng ngang (navbar) ở trên cùng với các liên kết đến bốn trang: Dashboard, Lịch sử, Bản đồ và Xếp hạng. **AdminLayout** dành cho quản trị viên, gồm thanh điều hướng dọc (sidebar) bên trái có thể thu gọn, chứa các mục: Dashboard, Gateways, Sensor Nodes, Cảnh báo, Cấu hình, Người dùng, Xuất dữ liệu và Nhật ký.

Hệ thống sử dụng bộ thành phần giao diện tái sử dụng (component library) tự xây dựng, bao gồm 28 component: Button, Modal, Badge, Input, Select, Table, Pagination, Card, Tooltip và các component chuyên biệt như AQICard, MetricCard, HealthAdvisory. Tất cả component tuân thủ thiết kế nhất quán: dark theme với bảng màu chính là xanh dương đậm, các trạng thái tương tác (hover, focus, disabled) được

4.2.4 Thiết kế module Backend

Phần này trình bày thiết kế chi tiết cho bốn module quan trọng nhất trên Backend Server.

Module telemetryService chịu trách nhiệm xử lý dữ liệu đo lường từ Gateway. Module này cung cấp hai hàm chính: `processTelemetry` (với tham số `gateway_id`, `data`) xử lý gói dữ liệu batch và `processHeartbeat` (với tham số `gateway_id`) xử lý tín hiệu heartbeat. Hàm `processTelemetry` thực hiện tuần tự ba bước: lọc trùng lặp qua module `dedupCache`, thực thi transaction cập nhật trạng thái thiết bị và lưu dữ liệu đo lường, sau đó kiểm tra ngưỡng cảnh báo ngoài transaction.

Module aqiService triển khai thuật toán tính AQI theo chuẩn US EPA đã trình bày ở Chương 3. Module cung cấp năm hàm: `calculateSubAQI` tính AQI thành phần cho một chất ô nhiễm, `calculateAQI` tính AQI tổng hợp (giá trị lớn nhất giữa PM2.5 và PM10), `getAQIInfo` ánh xạ giá trị AQI sang mức và mã màu, `getCO2Info` và `getTVOCInfo` đánh giá mức CO₂ và TVOC. Bảng breakpoint được lưu trữ dưới dạng mảng hằng số trong mã nguồn, tương ứng chính xác với Bảng 3.3 và Bảng 3.4 ở Chương 3.

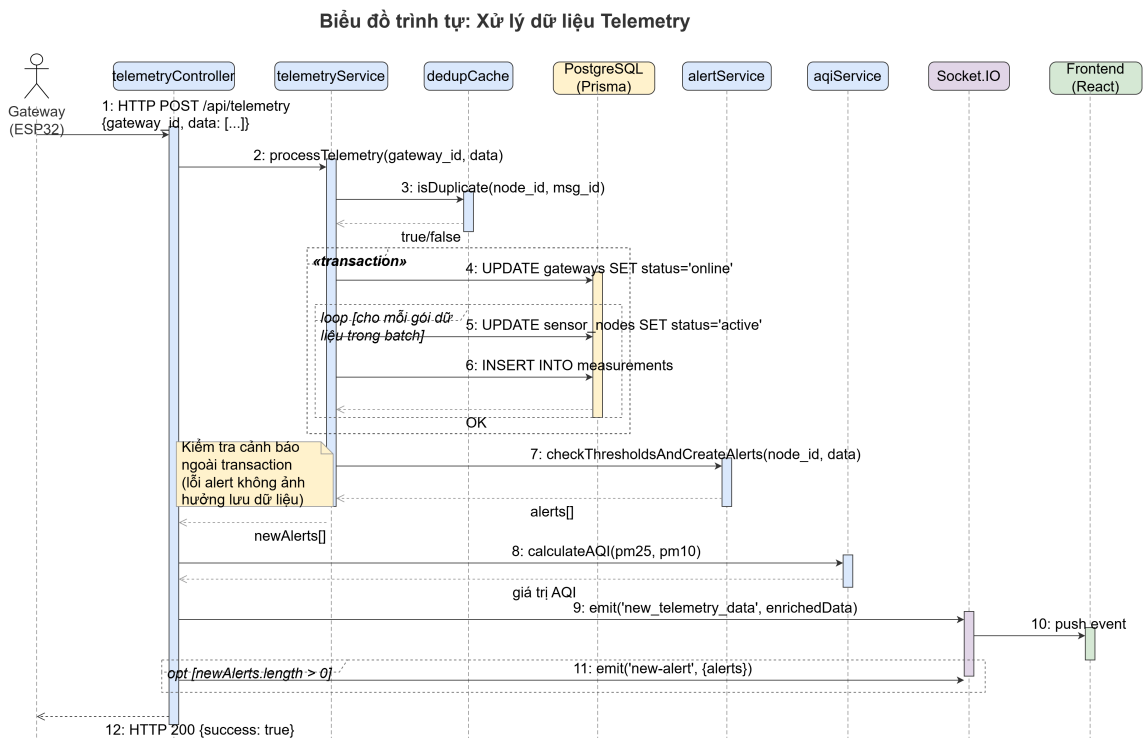
Module alertService quản lý toàn bộ vòng đời cảnh báo. Module cung cấp sáu hàm: `getAlerts` truy vấn danh sách cảnh báo với hỗ trợ lọc và phân trang, `acknowledgeAlert` xác nhận xử lý, `deleteAlert` xóa cảnh báo, `checkThresholdsAndCreateAlerts` kiểm tra năm thông số (PM2.5, PM10, CO₂, TVOC, nhiệt độ) so với ngưỡng cấu hình và tạo cảnh báo khi vượt ngưỡng, `createConnectivityAlerts` tạo cảnh báo mất kết nối cho thiết bị offline, và `cleanupOldAlerts` xóa cảnh báo quá 30 ngày. Cơ chế cooldown 15 phút ngăn chặn tạo cảnh báo trùng lặp cho cùng thiết bị và cùng thông số trong khoảng thời gian ngắn.

Module stationService cung cấp dữ liệu cho các trang người dùng. Hàm `getNearestStation` sử dụng PostGIS để truy vấn trạm đo gần nhất dựa trên tọa độ GPS của người dùng thông qua hàm `ST_Distance`. Hàm `getStationHistory` truy vấn dữ liệu lịch sử từ Continuous Aggregate `hourly_measurements` cho chế độ xem theo giờ, hoặc từ bảng `measurements` gốc cho chế độ xem theo ngày.

Để minh họa luồng tương tác giữa các thành phần, phần tiếp theo trình bày ba biểu đồ trình tự (sequence diagram) cho ba use case quan trọng nhất của hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Biểu đồ trình tự 1: Xử lý dữ liệu Telemetry. Hình 4.6 mô tả luồng xử lý khi Gateway gửi dữ liệu đo lường lên Backend — đây là luồng cốt lõi của hệ thống IoT. Gateway gửi HTTP POST chứa batch dữ liệu đến `telemetryController`. Controller chuyển tiếp cho `telemetryService`, module này thực hiện ba bước tuần tự: (i) lọc gói tin trùng lặp qua `dedupCache` (xử lý trường hợp nhiều Gateway cùng nhận một gói LoRa), (ii) thực thi database transaction để cập nhật trạng thái Gateway, cập nhật trạng thái Sensor Node và lưu dữ liệu đo lường vào bảng `measurements`, (iii) kiểm tra ngưỡng cảnh báo qua `alertService` — bước này chạy ngoài transaction để lỗi cảnh báo không ảnh hưởng đến luồng lưu dữ liệu chính. Sau đó, Controller tính chỉ số AQI qua `aqiService` và phát sự kiện `new_telemetry_data` qua Socket.IO đến Frontend. Nếu có cảnh báo mới, sự kiện `new-alert` cũng được phát để thông báo cho quản trị viên.

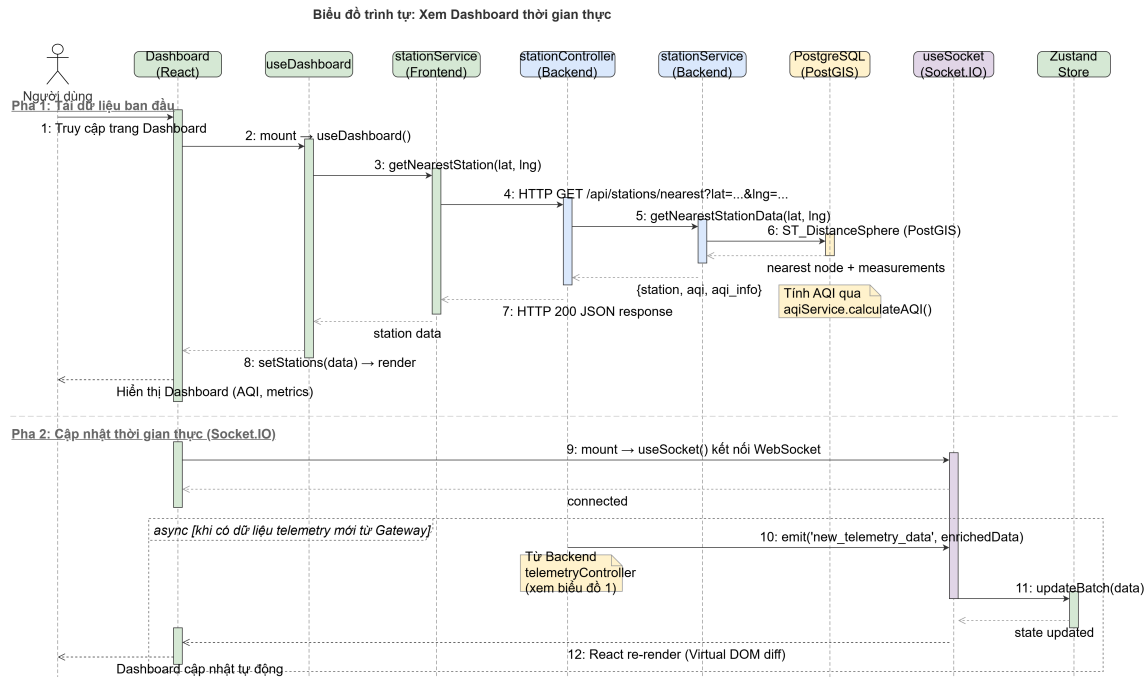


Hình 4.6: Biểu đồ trình tự xử lý dữ liệu Telemetry

Biểu đồ trình tự 2: Xem Dashboard thời gian thực. Hình 4.7 mô tả luồng end-to-end từ khi người dùng truy cập trang Dashboard đến khi nhận cập nhật thời gian thực. Luồng gồm hai pha. Pha 1 (tải dữ liệu ban đầu): khi React mount component Dashboard, hook `useDashboard` gọi API `/api/stations/nearest` kèm toạ độ GPS của người dùng. Backend sử dụng PostGIS (`ST_DistanceSphere`) để tìm trạm đo gần nhất, tính AQI và trả kết quả về Frontend để hiển thị. Pha 2 (cập nhật thời gian thực): hook `useSocket` kết nối WebSocket đến Backend. Mỗi khi có dữ liệu telemetry mới từ Gateway, Backend phát sự kiện `new_telemetry_data`;

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

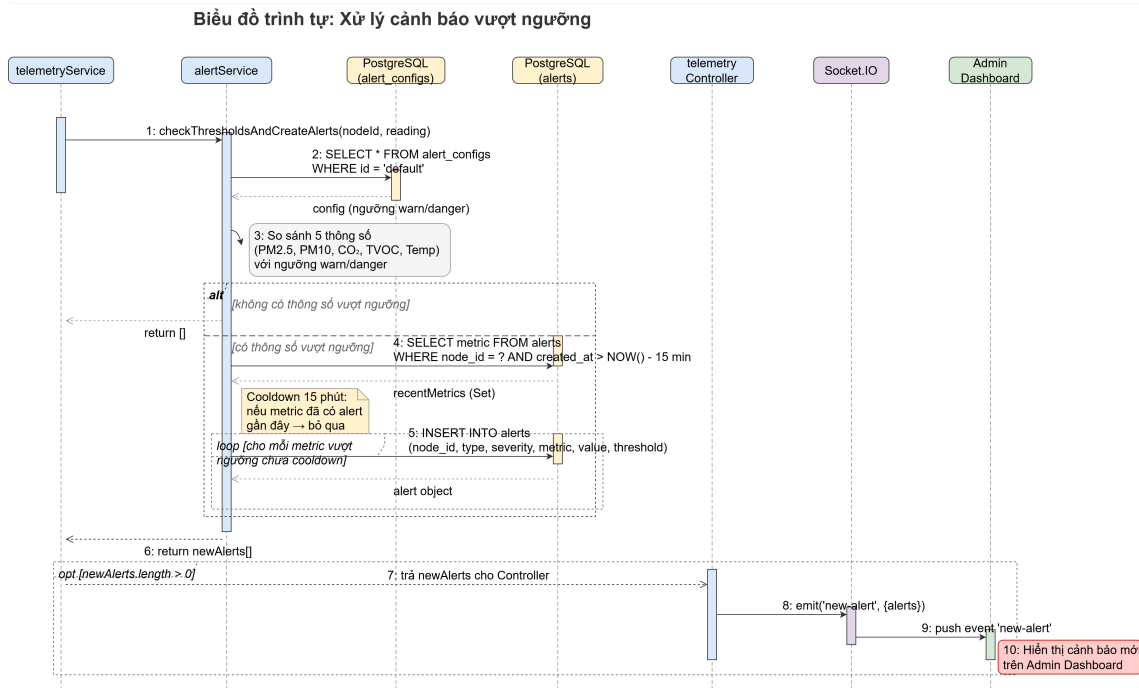
hook `useSocket` nhận sự kiện, cập nhật Zustand store, kích hoạt React Virtual DOM so sánh và chỉ vẽ lại các thành phần thay đổi trên giao diện.



Hình 4.7: Biểu đồ trình tự xem Dashboard thời gian thực

Biểu đồ trình tự 3: Xử lý cảnh báo vượt ngưỡng. Hình 4.8 mô tả chi tiết luồng kiểm tra và tạo cảnh báo khi dữ liệu đo lường vượt ngưỡng cấu hình. Sau khi `telemetryService` lưu dữ liệu thành công, module gọi hàm `checkThresholdsAndCreateAlerts` của `alertService`. Module `alertService` thực hiện bốn bước: (i) truy vấn bảng `alert_configs` để lấy ngưỡng warn/danger cho năm thông số (PM2.5, PM10, CO₂, TVOC, nhiệt độ), (ii) so sánh từng giá trị đo với ngưỡng tương ứng, (iii) kiểm tra cơ chế cooldown 15 phút — nếu cùng Sensor Node và cùng thông số đã có cảnh báo trong 15 phút gần đây thì bỏ qua để tránh tạo cảnh báo trùng lặp, (iv) tạo bản ghi cảnh báo mới trong bảng `alerts`. Kết quả trả về Controller, nếu có cảnh báo mới, Controller phát sự kiện `new-alert` qua Socket.IO để Admin Dashboard hiển thị thông báo tức thì.

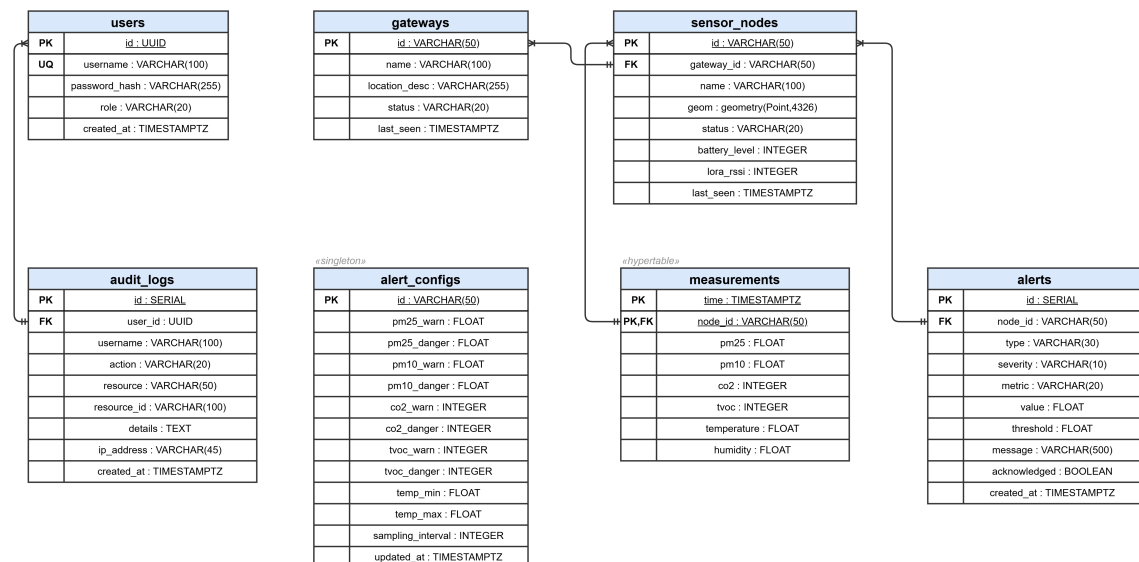
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.8: Biểu đồ trình tự xử lý cảnh báo vượt ngưỡng

4.2.5 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu được thiết kế trên PostgreSQL 15 với hai extension: TimescaleDB cho dữ liệu chuỗi thời gian và PostGIS cho dữ liệu không gian. Lược đồ gồm bảy bảng chính và một materialized view (trên biểu đồ Hình 4.9 chỉ thể hiện 7 bảng chính).



Hình 4.9: Biểu đồ thực thể-quan hệ (ERD)

Bảng users lưu thông tin tài khoản với khóa chính là UUID tự sinh, bao gồm username (duy nhất), password_hash (mã hóa bcrypt) và role (admin hoặc user).

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng gateways lưu thông tin Gateway với khóa chính là mã định danh dạng chuỗi (ví dụ “GW_001”), bao gồm tên, mô tả vị trí, trạng thái online/offline và thời điểm gửi dữ liệu cuối (last_seen).

Bảng sensor_nodes lưu thông tin Sensor Node với khóa ngoại tham chiếu đến bảng gateways (quan hệ 1-N: một Gateway quản lý nhiều Sensor Node, xóa Gateway sẽ xóa cascade các Sensor Node). Cột geom kiểu geometry(Point, 4326) lưu tọa độ GPS theo hệ tọa độ WGS84, cho phép truy vấn không gian qua PostGIS. Các cột battery_level và lora_rssi lưu thông tin trạng thái phần cứng.

Bảng measurements là hypertable TimescaleDB, được tự động phân mảnh theo cột thời gian time. Khóa chính composite gồm (time, node_id) đảm bảo mỗi Sensor Node chỉ có một bản ghi tại mỗi thời điểm. Bảng lưu sáu thông số: PM2.5, PM10, CO₂, TVOC, nhiệt độ và độ ẩm. Retention policy tự động xóa dữ liệu thô quá 3 tháng.

Materialized view hourly_measurements là Continuous Aggregate tự động tính giá trị trung bình theo giờ cho mỗi Sensor Node. View này được TimescaleDB tự động cập nhật khi có dữ liệu mới ghi vào bảng measurements.

Bảng alert_configs lưu cấu hình ngưỡng cảnh báo dưới dạng singleton row (chỉ một bản ghi với id = “default”). Cấu hình bao gồm ngưỡng warn và danger cho bốn thông số (PM2.5, PM10, CO₂, TVOC), ngưỡng nhiệt độ tối thiểu/tối đa, và chu kỳ lấy mẫu.

Bảng alerts lưu cảnh báo tự động với hai loại: “threshold” (vượt ngưỡng) và “connectivity” (mất kết nối). Mỗi cảnh báo ghi nhận thông số vi phạm (metric), giá trị đo được (value), ngưỡng so sánh (threshold) và trạng thái xác nhận (acknowledged). Ba chỉ mục trên cột node_id, created_at và acknowledged tối ưu hiệu năng truy vấn lọc và phân trang.

Bảng audit_logs ghi lại mọi hành động quản trị, bao gồm người thực hiện (user_id, username), loại thao tác (action: CREATE, UPDATE, DELETE), đối tượng thao tác (resource, resource_id), chi tiết thay đổi (details dạng JSON) và địa chỉ IP. Ba chỉ mục trên cột user_id, created_at và resource phục vụ truy vấn tra cứu nhật ký.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.1 và Bảng 4.2 liệt kê các công cụ, thư viện và phiên bản sử dụng để phát triển hệ thống, phân nhóm theo Backend và Frontend.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.1: Danh sách thư viện và công cụ phía Backend

Mục đích	Công cụ	Phiên bản
Runtime	Node.js	22.x
Web framework	Express	4.21.0
ORM	Prisma Client	7.5.0
Giao tiếp real-time	Socket.IO	4.7.5
Xác thực JWT	jsonwebtoken	9.0.3
Mã hóa mật khẩu	bcryptjs	3.0.3
Bảo mật HTTP headers	Helmet	8.1.0
Tác vụ định kỳ	node-cron	4.2.1
Xuất CSV	json2csv	6.0.0
CSDL	PostgreSQL	15
Extension time-series	TimescaleDB	2.11.0
Extension geospatial	PostGIS	3.3.2

Bảng 4.2: Danh sách thư viện và công cụ phía Frontend

Mục đích	Công cụ	Phiên bản
UI framework	React	19.2.4
Build tool	Vite	8.0.4
Routing	React Router	7.14.0
HTTP client	Axios	1.14.0
Bản đồ tương tác	Leaflet + React-Leaflet	1.9.4 / 5.0.0
Biểu đồ	ECharts	6.0.0
Đa ngôn ngữ	react-i18next	17.0.2
Quản lý trạng thái	Zustand	5.0.12
Icon	Lucide React	1.7.0
Socket client	socket.io-client	4.8.3

Ngoài ra, hệ thống sử dụng các công cụ phát triển và triển khai: VS Code làm IDE chính, PlatformIO với framework Arduino để phát triển firmware ESP32, Docker và Docker Compose để container hóa, Nginx làm reverse proxy, và Certbot để quản lý chứng chỉ SSL.

4.3.2 Kết quả đạt được

Sản phẩm cuối cùng bao gồm bốn thành phần được đóng gói riêng biệt. Thứ nhất, **Firmware Sensor Node** chạy trên ESP32, sử dụng cơ chế Deep-sleep để tối ưu năng lượng thông qua việc thức giấc định kỳ để tuần tự đọc các cảm biến và gửi qua LoRa. Thứ hai, **Firmware Gateway** chạy trên ESP32, sử dụng kiến trúc superloop để nhận gói LoRa và chuyển tiếp lên Backend Server qua WiFi. Thứ ba, **Backend Server** là ứng dụng Node.js/Express cung cấp REST API và giao tiếp real-time qua Socket.IO. Thứ tư, **Frontend** là ứng dụng React SPA được build bởi Vite thành các tệp tĩnh, phục vụ bởi Nginx.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.3 thống kê quy mô mã nguồn của từng thành phần.

Bảng 4.3: Thống kê quy mô mã nguồn hệ thống

Thành phần	Ngôn ngữ chính	LOC	Ghi chú
Firmware Sensor Node	C/C++	1262	Deep-sleep
Firmware Gateway	C/C++	1608	Superloop
Backend Server	JavaScript (Node.js)	2732	Express 4.21
Frontend	JavaScript (React)	8050	Vite 8.0
Database	SQL	107	PostgreSQL
Triển khai	YAML, Nginx conf	277	Docker Compose

Bảng 4.4 đối sánh kết quả đạt được với các yêu cầu chức năng và phi chức năng đã đặc tả ở Chương 2, cho thấy hệ thống đã hiện thực đầy đủ toàn bộ chín chức năng và đáp ứng năm yêu cầu phi chức năng.

Bảng 4.4: Đối sánh yêu cầu đặc tả với kết quả đạt được

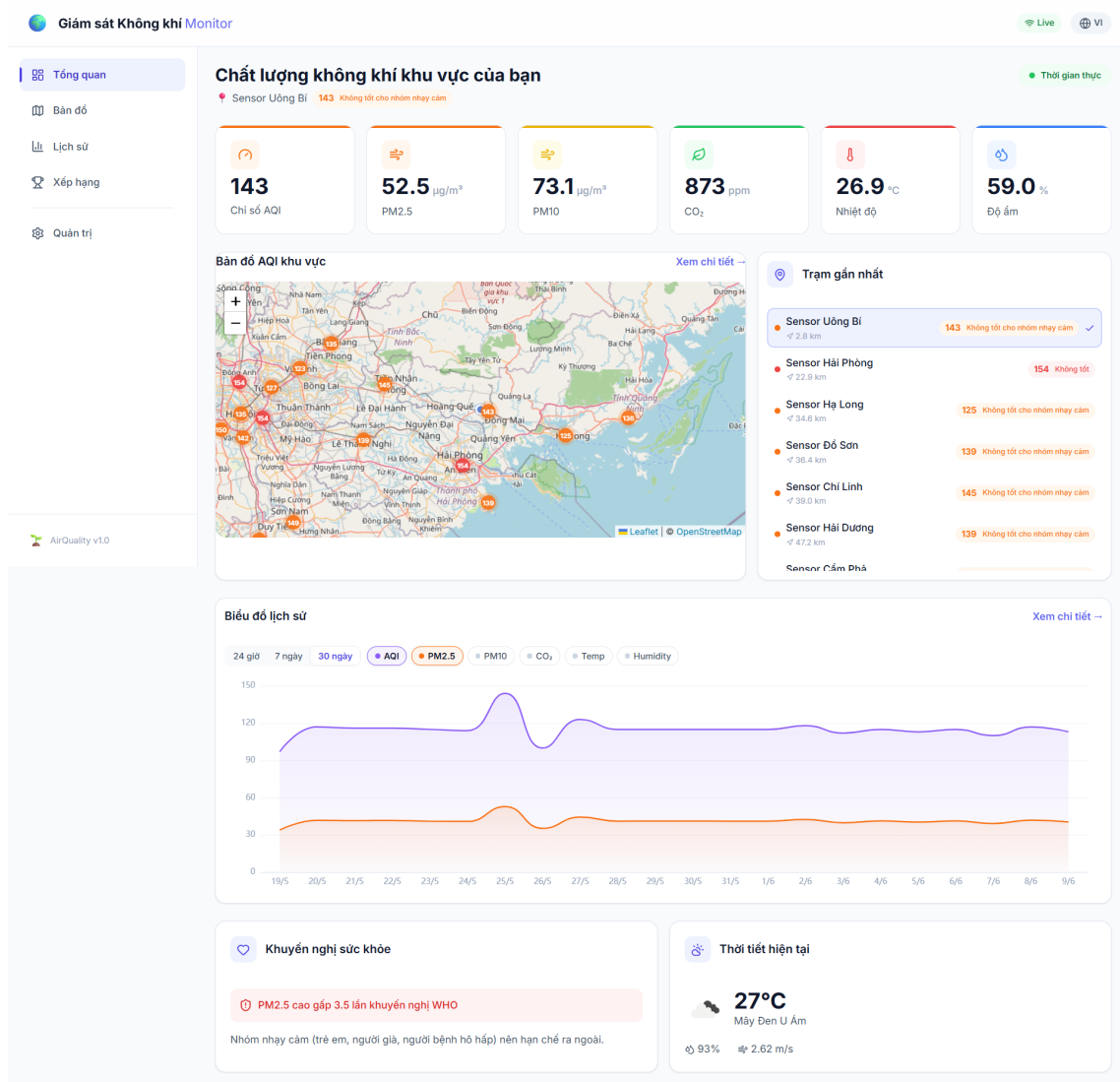
STT	Yêu cầu (Chương 2)	Trạng thái	Minh chứng
<i>Yêu cầu chức năng</i>			
1	Xem dashboard (Bảng 2.2)	Đạt	Mục 4.3.3, kiểm thử Bảng 4.5
2	Xem lịch sử dữ liệu (Bảng 2.3)	Đạt	Mục 4.3.3
3	Xem bản đồ AQI	Đạt	Mục 4.3.3
4	Xem xếp hạng ô nhiễm	Đạt	Mục 4.3.3
5	Quản lý thiết bị (Bảng 2.4)	Đạt	Mục 4.3.3, kiểm thử Bảng 4.6
6	Giám sát cảnh báo (Bảng 2.5)	Đạt	Mục 4.3.3, kiểm thử Bảng 4.7
7	Cấu hình hệ thống	Đạt	Mục 4.3.3
8	Quản lý tài khoản	Đạt	Mục 4.3.3
9	Xuất dữ liệu CSV (Bảng 2.6)	Đạt	Mục 4.3.3
<i>Yêu cầu phi chức năng (Bảng 2.7)</i>			
10	Độ tin cậy: 24/7, phát hiện offline	Đạt	Cron job 5 phút, ring buffer trên Gateway
11	Khả năng mở rộng: Hub-Spoke	Đạt	Mục 4.1.1, tự đăng ký Gateway
12	Tính dễ sử dụng: responsive, đa ngôn ngữ	Đạt	Mục 4.2.3, i18n Việt-Anh
13	Bảo mật: JWT, bcrypt, audit log	Đạt	Mục 4.2.4, kiểm thử TC6 Bảng 4.6

4.3.3 Minh họa các chức năng chính

Phần này minh họa các chức năng chính của hệ thống thông qua giao diện thực tế trên môi trường production. **Lưu ý:** trong hệ thống triển khai thực tế, chỉ có một trạm đo duy nhất (NODE_076) thu thập dữ liệu thật từ cảm biến phần cứng; các trạm đo còn lại hiển thị trên giao diện sử dụng dữ liệu mô phỏng (simulated data) được nạp vào cơ sở dữ liệu với mục đích minh họa đầy đủ các chức năng đa trạm của hệ thống như bản đồ AQI, xếp hạng ô nhiễm và chuyển đổi giữa các trạm.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Dashboard chất lượng không khí. Khi người dùng truy cập trang Dashboard, hệ thống yêu cầu quyền truy cập vị trí, sau đó ưu tiên hiển thị dữ liệu của trạm đo gần nhất. Giao diện trang chủ được bố cục từ trên xuống dưới với hàng đầu tiên là sáu thẻ thông số nổi bật hiển thị lần lượt các chỉ số: AQI tổng hợp, PM2.5, PM10, CO₂, nhiệt độ và độ ẩm, mỗi thẻ được thiết kế kèm viền màu cảnh báo mức độ tương ứng. Ngay bên dưới là khu vực Bản đồ AQI khu vực cùng danh sách các trạm đo lân cận. Ở nửa dưới bảng điều khiển, hệ thống cung cấp biểu đồ lịch sử theo dõi xu hướng thay đổi của các thông số theo thời gian, kết hợp với các thẻ tính năng bổ trợ như Khuyến nghị sức khỏe và Thời tiết hiện tại. Dữ liệu được cập nhật liên tục theo thời gian thực khi Backend phát sự kiện qua Socket.IO.

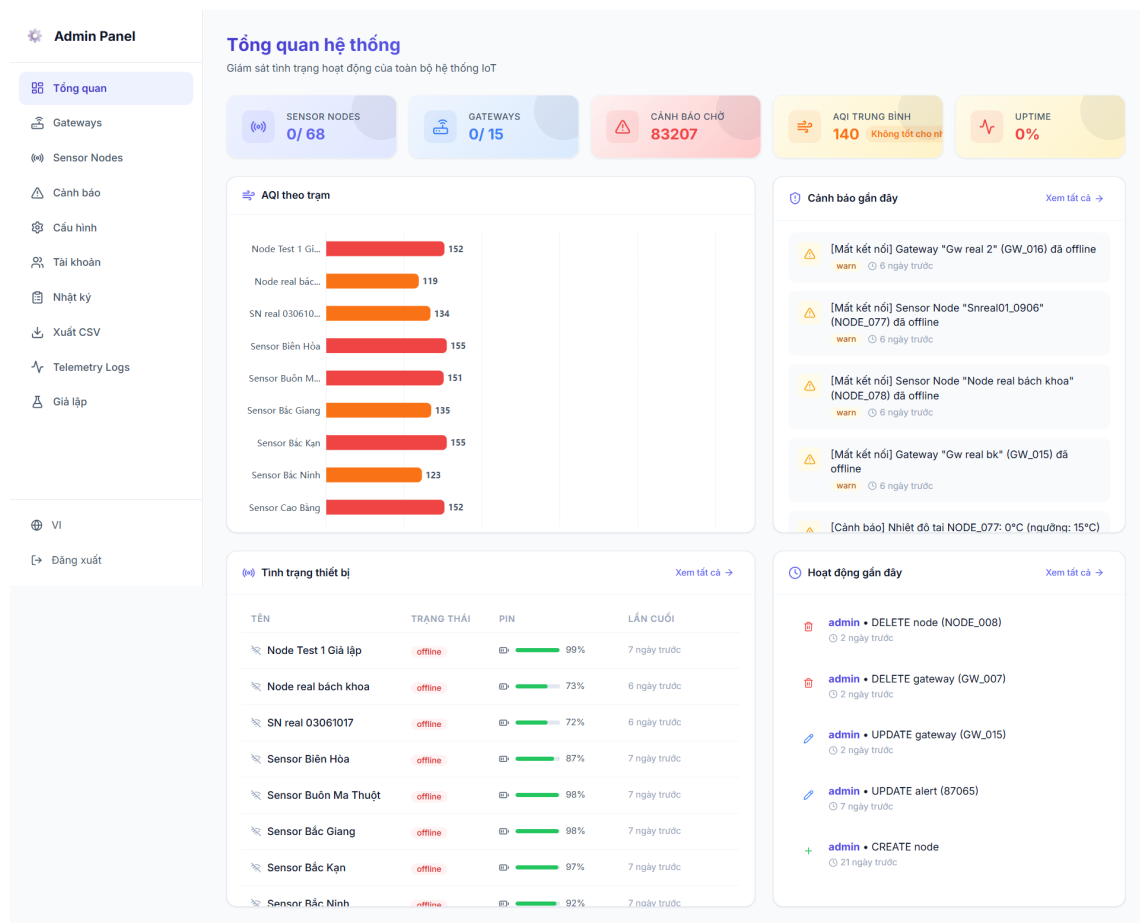


Hình 4.10: Dashboard chất lượng không khí

Dashboard quản trị hệ thống. Khi quản trị viên đăng nhập và truy cập trang Tổng quan (Admin Dashboard), hệ thống hiển thị bức tranh toàn cảnh về trạng thái hoạt động: số lượng Sensor Node và Gateway đang hoạt động, số cảnh báo chờ xử

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

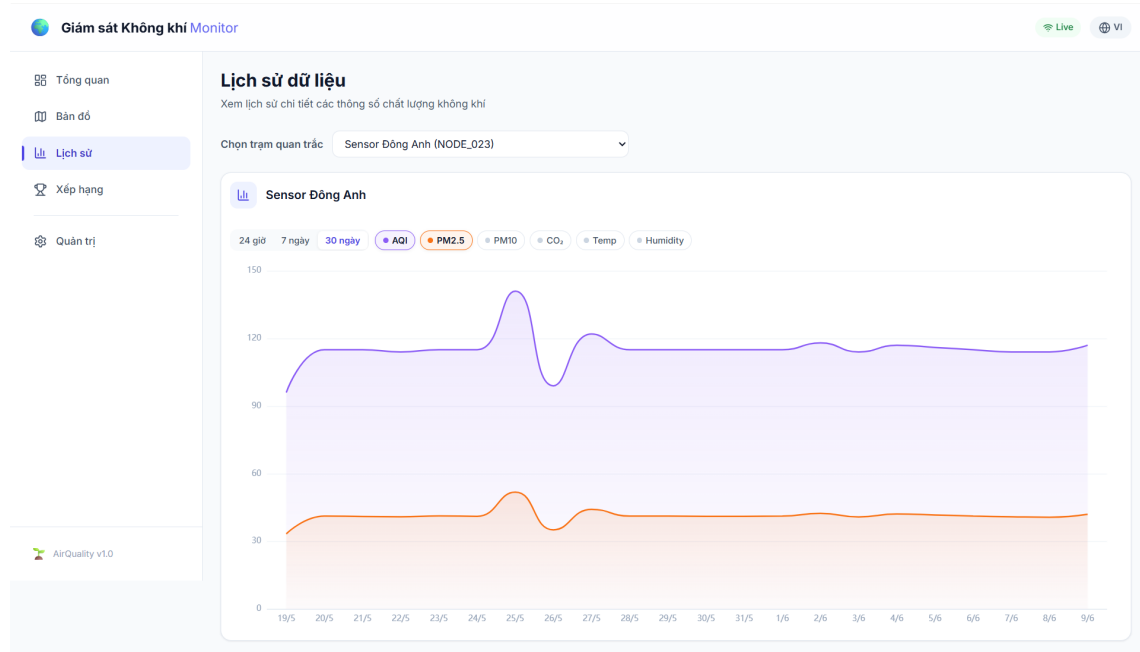
lý, chỉ số AQI trung bình toàn hệ thống và uptime. Phần bên trái liệt kê biểu đồ AQI theo trạm theo thời gian thực; phần bên phải tổng hợp các cảnh báo gần đây và nhật ký hoạt động (audit log) mới nhất, giúp quản trị viên nắm bắt nhanh tình hình mà không cần điều hướng qua nhiều trang.



Hình 4.11: Dashboard tổng quan quản trị hệ thống

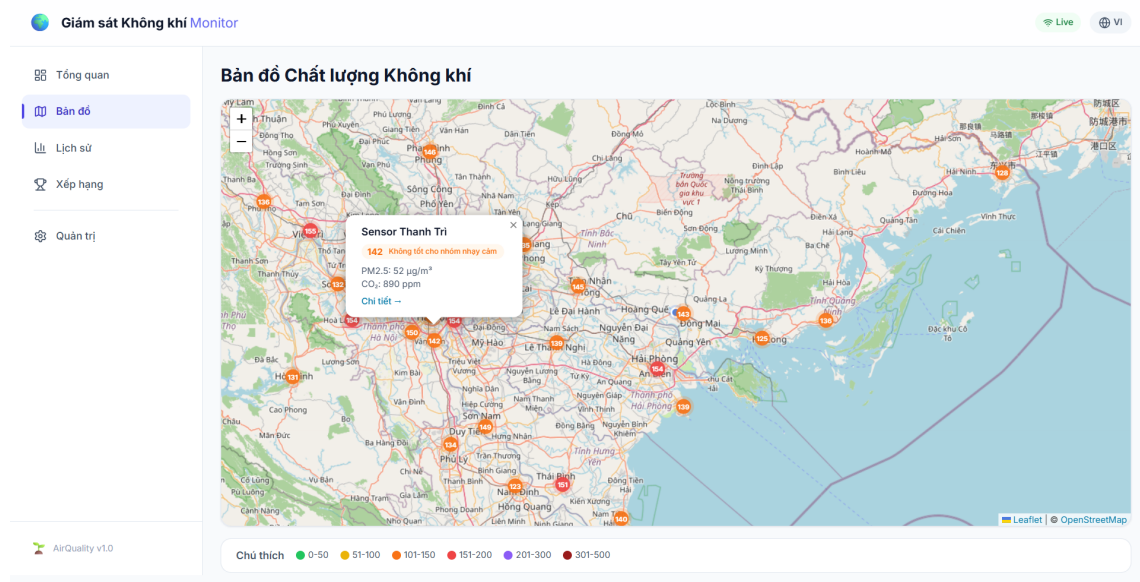
Lịch sử dữ liệu. Trang lịch sử cho phép người dùng chọn trạm đo, thông số cần xem và chế độ hiển thị (theo giờ hoặc theo ngày). Biểu đồ được vẽ bằng ECharts, hỗ trợ tooltip và responsive. Chế độ theo giờ truy vấn từ Continuous Aggregate `hourly_measurements` để đảm bảo hiệu năng.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.12: Giao diện trang Lịch sử dữ liệu

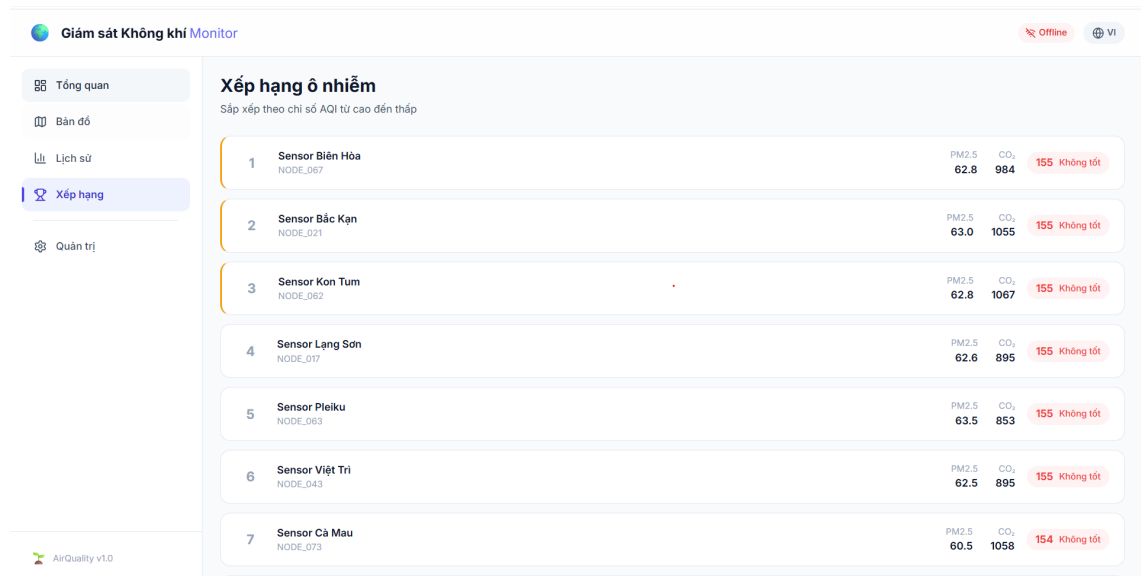
Bản đồ AQI. Trang bản đồ hiển thị toàn bộ các trạm đo trên bản đồ Leaflet với tile OpenStreetMap. Mỗi trạm được đánh dấu bằng marker hình tròn với mã màu AQI. Người dùng nhấn vào marker để xem popup thông tin chi tiết gồm tên trạm, giá trị AQI, PM2.5 và thời gian cập nhật cuối.



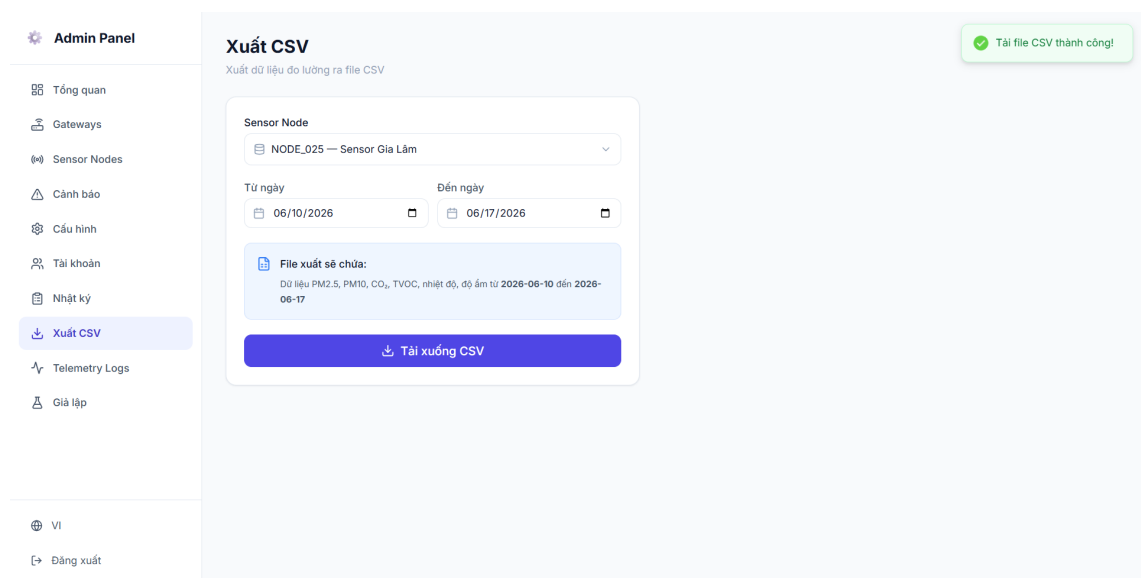
Hình 4.13: Giao diện trang Bản đồ AQI

Các chức năng quản trị khác. Hình 4.14 đến Hình 4.20 minh họa các chức năng hỗ trợ bao gồm: xếp hạng ô nhiễm (sắp xếp trạm theo AQI), quản lý tài khoản (phân quyền admin/user), xuất dữ liệu (tải CSV), nhật ký hệ thống (audit log), giám sát cảnh báo, cấu hình ngưỡng, và quản lý thiết bị.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

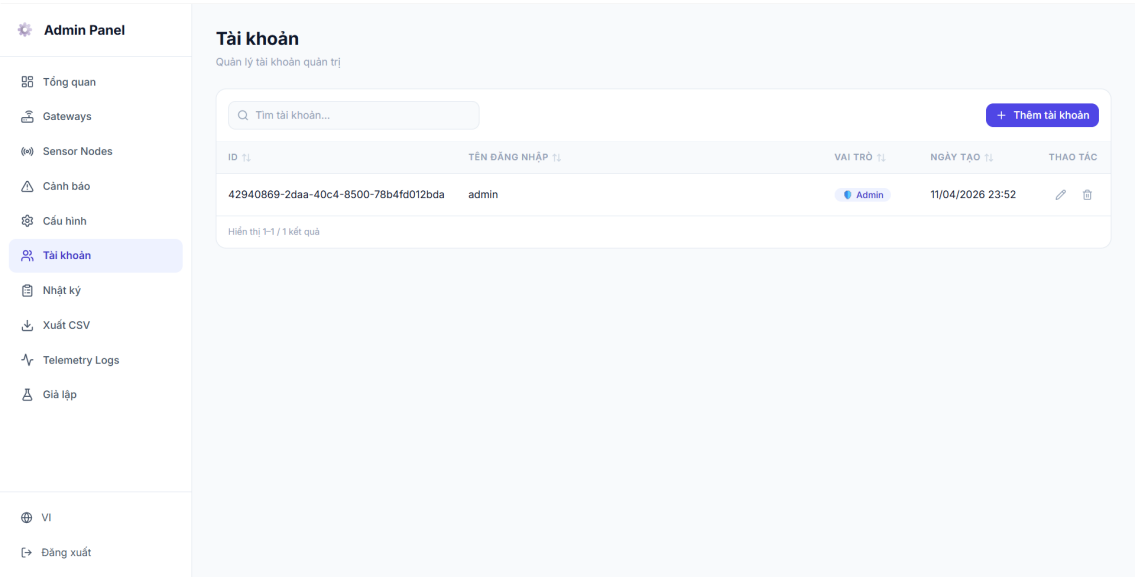


Hình 4.14: Giao diện trang Xếp hạng ô nhiễm

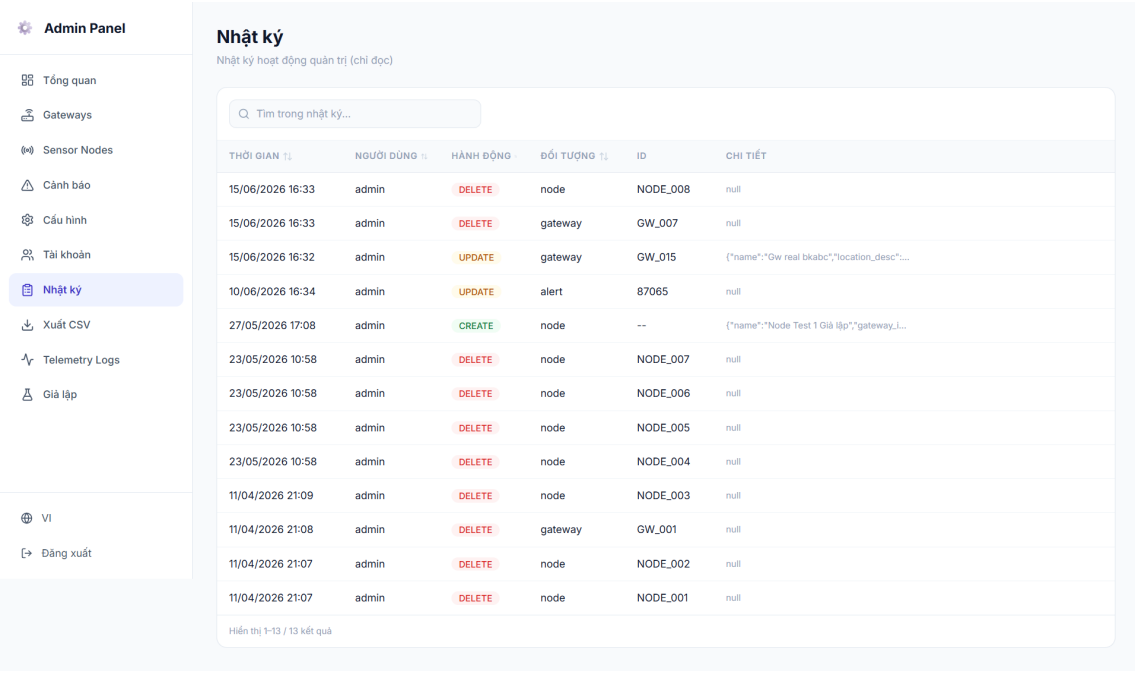


Hình 4.15: Giao diện chức năng Xuất dữ liệu CSV

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

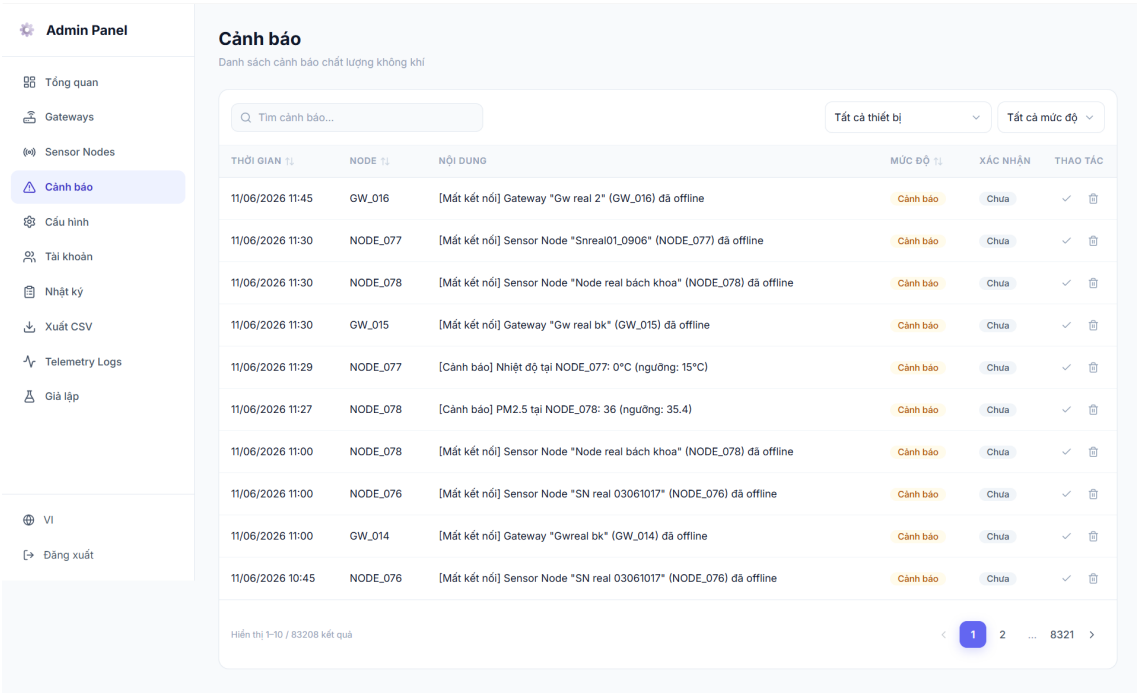


Hình 4.16: Giao diện trang Quản lý tài khoản

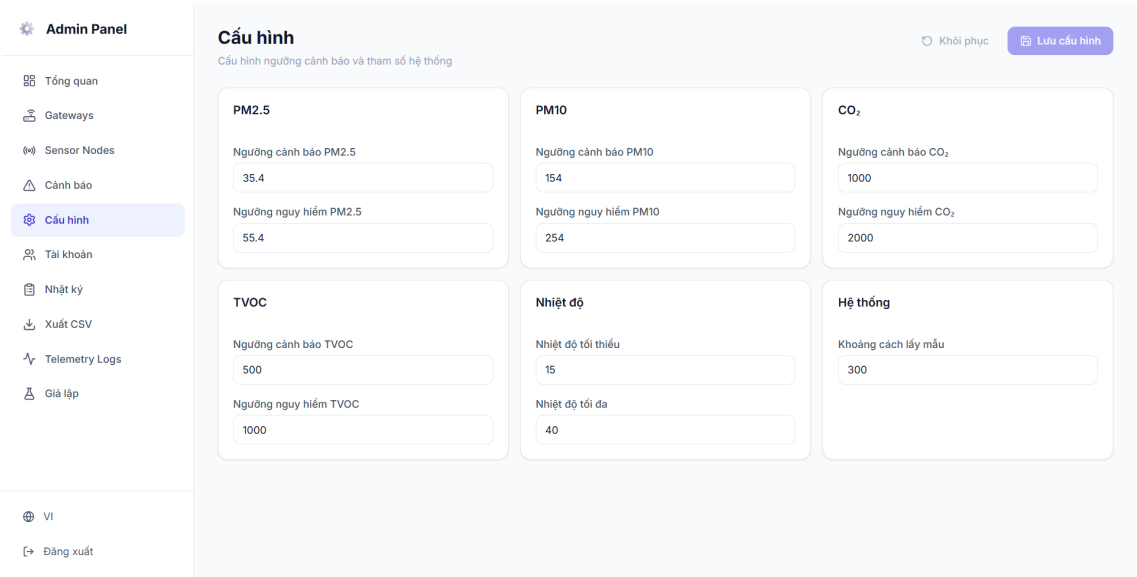


Hình 4.17: Nhật ký hệ thống (audit log)

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

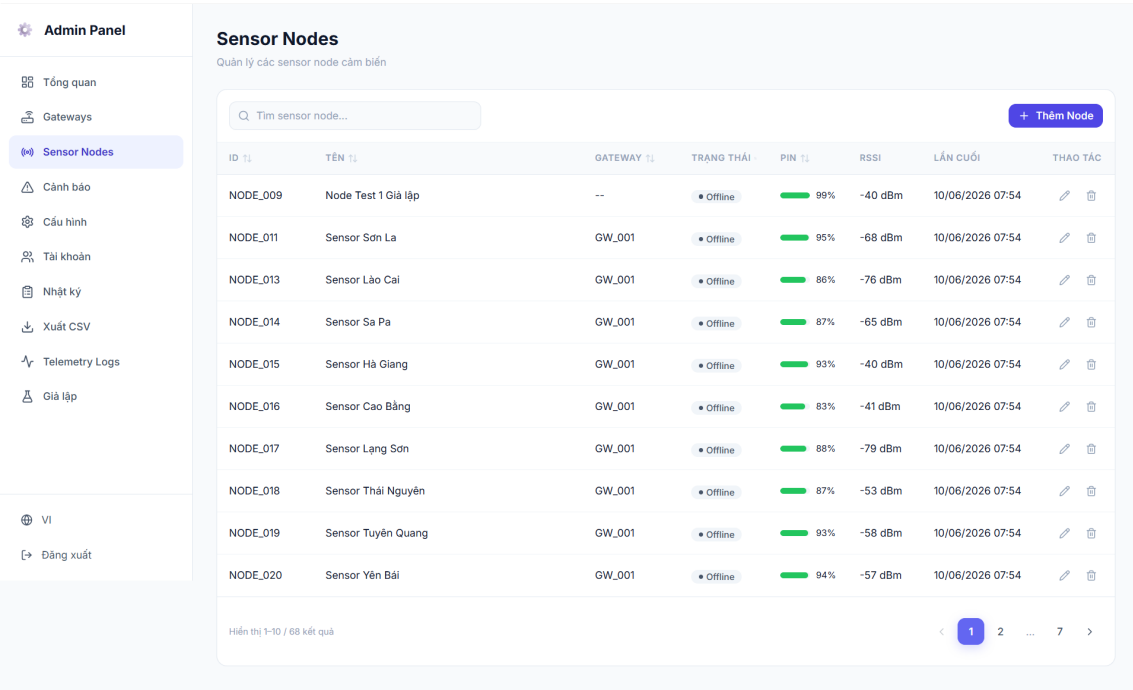


Hình 4.18: Giao diện giám sát cảnh báo



Hình 4.19: Giao diện cấu hình ngưỡng cảnh báo

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.20: Giao diện quản lý thiết bị

4.4 Kiểm thử

Hệ thống được kiểm thử bằng hai phương pháp. Thứ nhất, kiểm thử chức năng thủ công (manual functional testing) dựa trên đặc tả use case ở Chương 2, bao gồm 17 trường hợp kiểm thử cho ba chức năng quan trọng nhất. Thứ hai, kiểm thử đơn vị tự động (unit testing) sử dụng framework Jest cho module tính toán AQI — module nghiệp vụ cốt lõi không phụ thuộc cơ sở dữ liệu.

4.4.1 Kiểm thử chức năng Xem dashboard

Bảng 4.5 trình bày các trường hợp kiểm thử cho chức năng xem dashboard chất lượng không khí.

Bảng 4.5: Trường hợp kiểm thử chức năng Xem dashboard

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Truy cập dashboard, cho phép truy cập vị trí	Hiển thị dữ liệu trạm gần nhất	Đúng	Đạt
2	Truy cập dashboard, từ chối truy cập vị trí	Hiển thị dữ liệu trạm mặc định	Đúng	Đạt
3	Dashboard nhận dữ liệu mới qua Socket.IO	Các thẻ metric cập nhật tự động	Đúng	Đạt
4	Không có trạm nào hoạt động	Hiển thị thông báo “Không có dữ liệu”	Đúng	Đạt
5	Kiểm tra mã màu AQI	Thẻ AQI và badge hiển thị đúng mã màu theo mức AQI	Đúng	Đạt

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.4.2 Kiểm thử chức năng Quản lý thiết bị

Bảng 4.6 trình bày các trường hợp kiểm thử cho chức năng quản lý thiết bị.

Bảng 4.6: Trường hợp kiểm thử chức năng Quản lý thiết bị

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Thêm mới Gateway với thông tin hợp lệ	Gateway được tạo, hiển thị trong danh sách	Đúng	Đạt
2	Thêm mới Sensor Node, chọn Gateway cha	Sensor Node được tạo, liên kết đúng Gateway cha	Đúng	Đạt
3	Sửa thông tin thiết bị	Thông tin cập nhật thành công	Đúng	Đạt
4	Xóa Gateway có Sensor Node con	Xóa cascade cả Sensor Node con	Đúng	Đạt
5	Thêm thiết bị với dữ liệu không hợp lệ (để trống tên)	Hiển thị thông báo lỗi	Đúng	Đạt
6	Kiểm tra audit log sau thao tác	Nhật ký ghi nhận đúng thao tác, người thực hiện.	Đúng	Đạt

4.4.3 Kiểm thử chức năng Giám sát cảnh báo

Bảng 4.7 trình bày các trường hợp kiểm thử cho chức năng giám sát và xử lý cảnh báo.

Bảng 4.7: Trường hợp kiểm thử chức năng Giám sát cảnh báo

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Gửi dữ liệu PM2.5 vượt ngưỡng warn	Cảnh báo mức warn được tạo tự động	Đúng	Đạt
2	Gửi dữ liệu PM2.5 vượt ngưỡng danger	Cảnh báo mức danger được tạo, phát qua Socket.IO	Đúng	Đạt
3	Gửi dữ liệu vượt ngưỡng lần 2 trong 15 phút	Không tạo cảnh báo trùng (cooldown)	Đúng	Đạt
4	Thiết bị không gửi dữ liệu quá thời gian quy định	Cron job tạo cảnh báo mất kết nối	Đúng	Đạt
5	Lọc cảnh báo theo mức độ	Danh sách chỉ hiển thị cảnh báo đúng mức độ	Đúng	Đạt
6	Xác nhận cảnh báo (acknowledge)	Trạng thái cập nhật thành “đã xác nhận”	Đúng	Đạt

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.4.4 Kiểm thử chức năng Xem lịch sử dữ liệu

Bảng 4.8 trình bày các trường hợp kiểm thử cho chức năng xem lịch sử dữ liệu.

Bảng 4.8: Trường hợp kiểm thử chức năng Xem lịch sử dữ liệu

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Chọn trạm đo và xem biểu đồ theo ngày (30 ngày)	Biểu đồ hiển thị đúng dữ liệu	Phản hồi dưới 3000ms, đúng dữ liệu	Đạt
2	Chuyển chế độ hiển thị từ theo ngày sang theo giờ	Biểu đồ cập nhật đúng dạng dữ liệu	Đúng	Đạt
3	Tương tác với biểu đồ ECharts	Tooltip hiển thị chi tiết	Đúng	Đạt

4.4.5 Kiểm thử chức năng Xem bản đồ AQI

Bảng 4.9 trình bày các trường hợp kiểm thử cho chức năng xem bản đồ AQI.

Bảng 4.9: Trường hợp kiểm thử chức năng Xem bản đồ AQI

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Truy cập trang bản đồ	Hiển thị bản đồ Leaflet với tất cả marker trạm đo	Hiển thị đủ marker, mã màu đúng	Đạt
2	Nhấn vào marker trạm đo	Popup hiển thị tên trạm, AQI, PM2.5, thời gian	Hiển thị đầy đủ thông tin	Đạt
3	Kiểm tra mã màu marker theo mức AQI	Màu marker tương ứng với mức AQI (xanh, vàng, cam, đỏ)	Màu đúng theo chuẩn USEPA	Đạt

4.4.6 Kiểm thử chức năng Xuất dữ liệu CSV

Bảng 4.10 trình bày các trường hợp kiểm thử cho chức năng xuất dữ liệu CSV.

Bảng 4.10: Trường hợp kiểm thử chức năng Xuất dữ liệu CSV

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Xuất CSV với khoảng thời gian hợp lệ	Tải file CSV chứa đầy đủ cột dữ liệu	File CSV tải thành công, đủ cột	Đạt
2	Xuất CSV khi không có dữ liệu trong khoảng thời gian	Thông báo “Không có dữ liệu”	Hiển thị thông báo đúng	Đạt

4.4.7 Kiểm thử đơn vị module tính AQI

Module `aqiService` được kiểm thử đơn vị bằng framework Jest. Module này chứa logic tính toán AQI theo chuẩn US EPA, là module nghiệp vụ thuần (pure

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

function) không phụ thuộc cơ sở dữ liệu hay dịch vụ bên ngoài, do đó phù hợp để viết unit test trực tiếp mà không cần mock.

Bảng 4.11 trình bày kết quả kiểm thử đơn vị cho năm hàm trong module.

Bảng 4.11: Kết quả kiểm thử đơn vị module `aqiService`

STT	Hàm kiểm thử	Mô tả	Kết quả mong đợi	Đạt
1	<code>calculateSubAQI</code>	PM2.5 = 0 (biên dưới)	AQI = 0	Đạt
2	<code>calculateSubAQI</code>	PM2.5 = 12.0 (biên trên Good)	AQI = 50	Đạt
3	<code>calculateSubAQI</code>	PM2.5 = 35.4 (biên trên Moderate)	AQI = 100	Đạt
4	<code>calculateSubAQI</code>	PM2.5 = 55.5 (biên dưới Unhealthy)	AQI = 151	Đạt
5	<code>calculateSubAQI</code>	PM2.5 = 500.4 (biên trên max)	AQI = 500	Đạt
6	<code>calculateSubAQI</code>	PM2.5 = 600 (vượt ngưỡng)	AQI = 500	Đạt
7	<code>calculateAQI</code>	PM2.5 = 25, PM10 = 50	Max(78, 46) = 78	Đạt
8	<code>calculateAQI</code>	PM2.5 = 0, PM10 = 0	AQI = 0	Đạt
9	<code>calculateAQI</code>	PM2.5 = 5, PM10 = 200	AQI theo PM10 > 100	Đạt
10	<code>getAQIInfo</code>	AQI = 25	Mức Good	Đạt
11	<code>getAQIInfo</code>	AQI = 75	Mức Moderate	Đạt
12	<code>getAQIInfo</code>	AQI = 125	Mức Unhealthy Sensitive	Đạt
13	<code>getAQIInfo</code>	AQI = 175	Mức Unhealthy	Đạt
14	<code>getAQIInfo</code>	AQI = 250	Mức Very Unhealthy	Đạt
15	<code>getAQIInfo</code>	AQI = 400	Mức Hazardous	Đạt
16	<code>getCO2Info</code>	CO ₂ = 400	Mức Tốt	Đạt
17	<code>getCO2Info</code>	CO ₂ = 1200	Mức Kém	Đạt
18	<code>getTVOCInfo</code>	TVOC = 30	Mức Tốt	Đạt
19	<code>getTVOCInfo</code>	TVOC = 500	Mức Kém	Đạt

Kết quả: 19/19 trường hợp kiểm thử đều đạt (pass). Các test case tập trung kiểm tra giá trị biên (boundary values) của bảng breakpoint AQI, đảm bảo công thức nội suy tuyến tính trả về đúng giá trị tại các điểm chuyển mức.

4.4.8 Tổng kết kiểm thử

Tổng cộng 44 trường hợp kiểm thử đã được thực hiện, chia thành hai nhóm. Nhóm kiểm thử chức năng thủ công gồm 25 test case trên sáu chức năng: Xem dashboard (5 test case), Quản lý thiết bị (6 test case), Giám sát cảnh báo (6 test case), Xem lịch sử dữ liệu (3 test case), Xem bản đồ AQI (3 test case), và Xuất dữ liệu CSV (2 test case). Nhóm kiểm thử đơn vị tự động gồm 19 test case cho module tính AQI. Tất cả 44/44 trường hợp đều đạt, tỉ lệ 100%. Kết quả cho thấy cả logic

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Dockerfile, khởi tạo cơ sở dữ liệu từ `init.sql`, và kết nối các container qua mạng nội bộ `app-network`. Toàn bộ cấu hình nhạy cảm (mật khẩu database, JWT secret) được lưu trong file `.env.production` không đưa lên kho mã nguồn.

Hình 4.22 cho thấy bốn container đang chạy ổn định trên môi trường production.

```
*** System restart required ***
Last login: Sun Jun 14 14:23:39 2026 from 42.114.184.18
root@ubuntu-s-1vcpu-1gb-sgp1-01:~# docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"


| NAMES        | IMAGE                         | STATUS                | PORTS                                                     |
|--------------|-------------------------------|-----------------------|-----------------------------------------------------------|
| datn_nginx   | datn-nginx                    | Up 18 hours           | 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp |
| datn_backend | datn-backend                  | Up 18 hours (healthy) | 3000/tcp                                                  |
| datn_db      | timescale/timescaledb-ha:pg15 | Up 2 months           | 8080/tcp, 127.0.0.1:5432->5432/tcp, 8081/tcp              |
| datn_certbot | certbot/certbot               | Up 2 months           | 80/tcp, 443/tcp                                           |


```

Hình 4.22: Kết quả lệnh `docker ps` trên máy chủ production

Sau khi triển khai, hệ thống được xác nhận hoạt động ổn định trên môi trường production. Giao diện web truy cập được tại địa chỉ `https://datn.thamnguyen.dev` với chứng chỉ SSL hợp lệ. Kết nối WebSocket (Socket.IO) hoạt động đúng: khi Gateway gửi dữ liệu mới, dashboard cập nhật trong thời gian thực mà không cần tải lại trang. Bốn container Docker (db, backend, nginx, certbot) duy trì trạng thái healthy liên tục.

4.5.1 Kết quả vận hành thực tế

Hệ thống được đưa vào vận hành thử nghiệm để đánh giá các chỉ tiêu phi chức năng và tính ổn định của thiết bị. Kết quả đánh giá tập trung vào hai khía cạnh cốt lõi: hiệu năng và độ tin cậy.

Đánh giá hiệu năng: Hệ thống cho thấy tốc độ phản hồi nhanh chóng và ổn định trong quá trình thử nghiệm. Khi người dùng truy cập trang chủ (Dashboard), dữ liệu của trạm đo gần nhất được hệ thống tải về gần như tức thời. Đối với truy vấn khối lượng dữ liệu lớn như dải lịch sử 30 ngày (yêu cầu số 1 ở Bảng 2.7), nhờ ứng dụng cơ chế Continuous Aggregate của TimescaleDB, dữ liệu biểu đồ được hệ thống trích xuất mượt mà, luôn đáp ứng dễ dàng tiêu chuẩn phản hồi dưới 3 giây. Ở tính năng thời gian thực, luồng dữ liệu thông qua Socket.IO hoạt động trơn tru: ngay sau khi có dữ liệu mới từ Gateway, giao diện người dùng lập tức tự động cập nhật số liệu mới nhất mà không xảy ra độ trễ (lag) đáng kể nào.

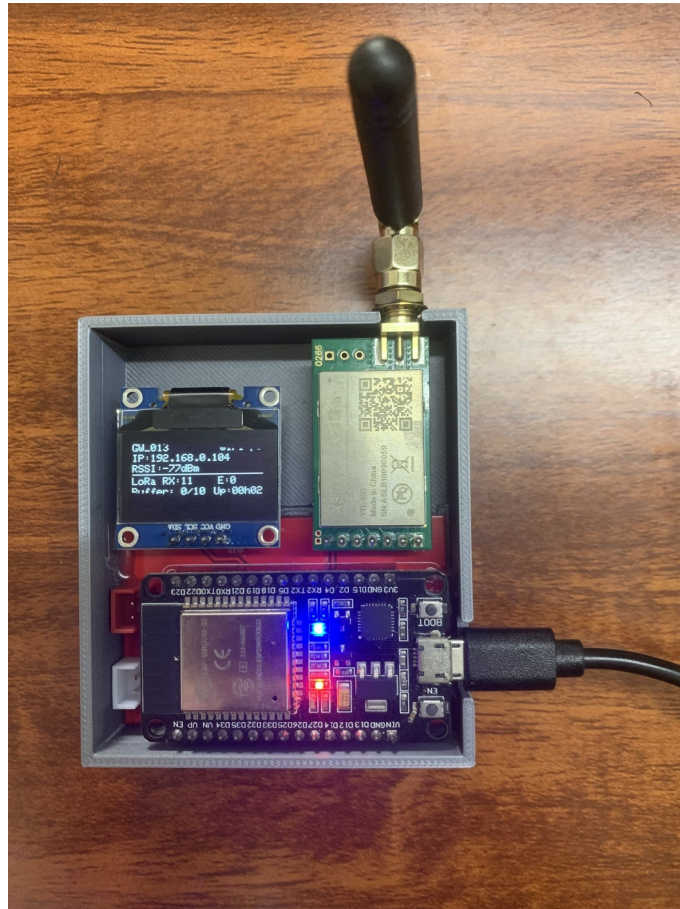
Đánh giá độ tin cậy: Các container Docker duy trì uptime 100% trong suốt thời gian theo dõi 7 ngày thử nghiệm (xác nhận qua lệnh `docker ps` và log hệ thống), không xảy ra hiện tượng tràn bộ nhớ hay khởi động lại ngoài ý muốn. Tác vụ định kỳ phát hiện thiết bị offline hoạt động chính xác: cảnh báo mất kết nối tự động được tạo và gửi đến frontend sau đúng 40 phút đối với Sensor Node (do đặc thù tiết kiệm pin), và sau 5 phút đối với Gateway khi hệ thống ngừng nhận được dữ liệu.

Kịch bản thử nghiệm mất kết nối thiết bị: Nhằm kiểm chứng quá trình giám

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

sát kết nối theo thời gian thực (yêu cầu số 2 ở Bảng 2.7), hệ thống được thực nghiệm với kịch bản mất kết nối chi tiết từng bước như sau:

Bước 1: Bật Gateway có ID: GW_013. Gateway sẽ gửi heartbeat lên server:



Hình 4.23: Gateway GW_013 gửi heartbeat

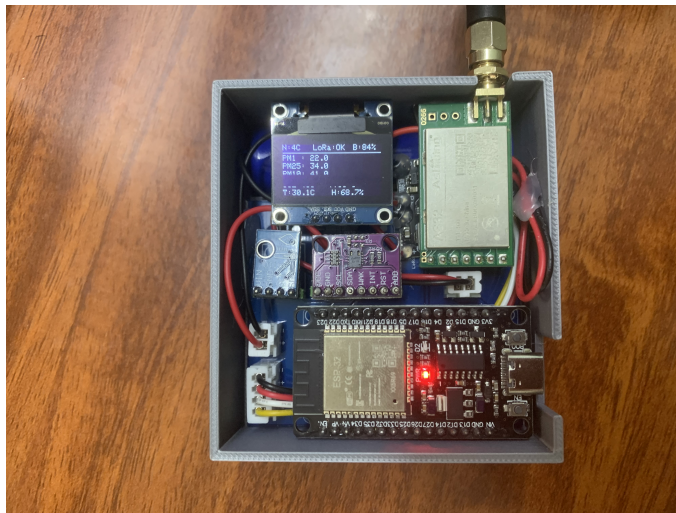
Server đã ghi nhận GW_013 online trên hệ thống:

Gateways						
Quản lý các gateway thu thập dữ liệu						
Q Tìm gateway...				+ Thêm Gateway		
ID	TÊN	VỊ TRÍ	IP	TRẠNG THÁI	HOẠT ĐỘNG LẦN CUỐI	THAO TÁC
GW_013	Gwreal	Nhà	--	Online	10/06/2026 10:43	✎ 🗑
GW_001	Gateway GW_001	--	--	Offline	10/06/2026 10:05	✎ 🗑
GW_007	Gw15050943	Bk	--	Offline	10/06/2026 07:54	✎ 🗑
GW_010	Gw bách khoa	Bách khoa	--	Offline	04/06/2026 12:47	✎ 🗑
GW_009	GW-03061011	Nhà	--	Offline	03/06/2026 11:49	✎ 🗑

Hình 4.24: Server ghi nhận Gateway GW_013 online

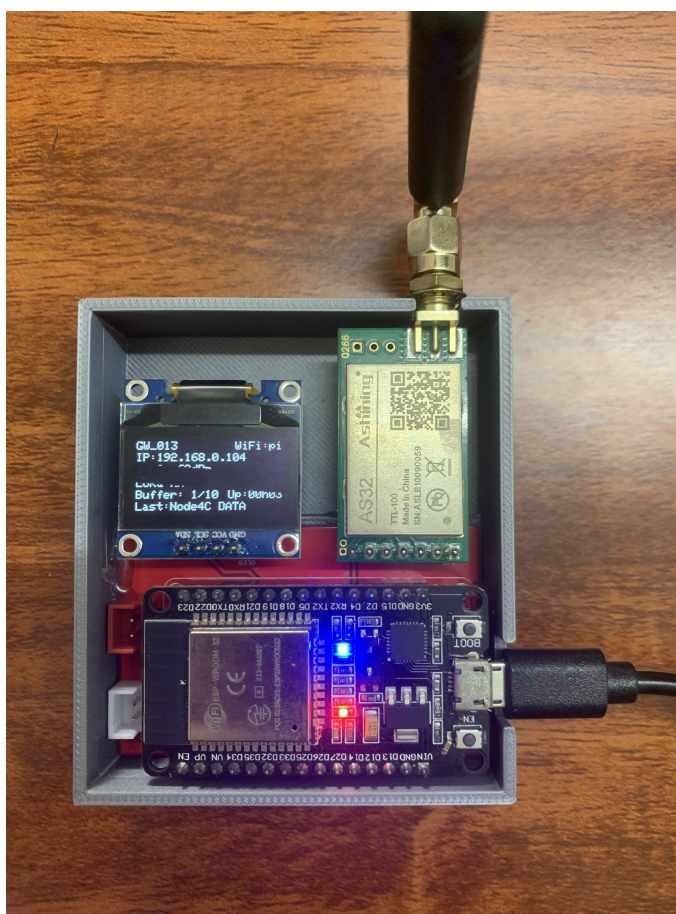
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bước 2: Tiếp tục bật Sensor Node có ID: 4C (NODE_076):



Hình 4.25: Bật Sensor Node 4C (NODE_076)

Lúc này Gateway đã nhận được gói tin từ Sensor Node 4C (NODE_076) và gửi lên server:

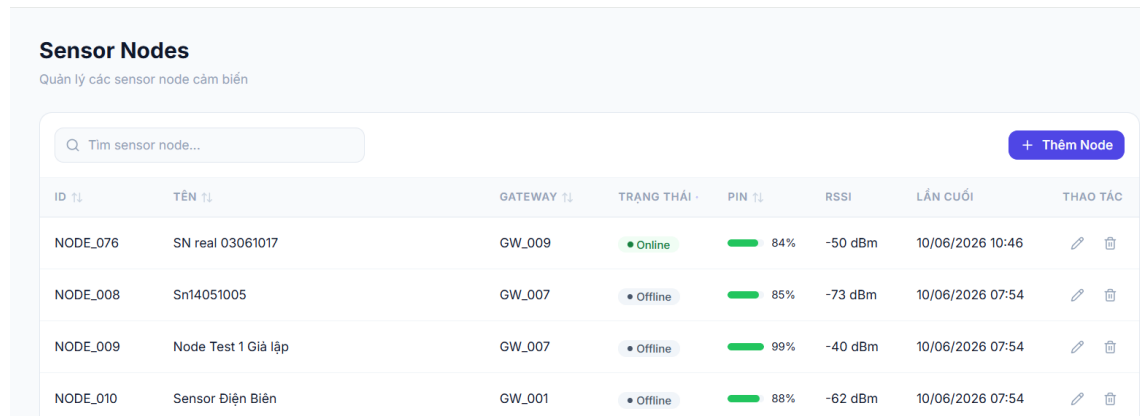


Hình 4.26: Gateway nhận gói tin từ Sensor Node 4C (NODE_076)

Server nhận được gói tin từ Gateway và đánh dấu Sensor Node 4C (NODE_076)

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

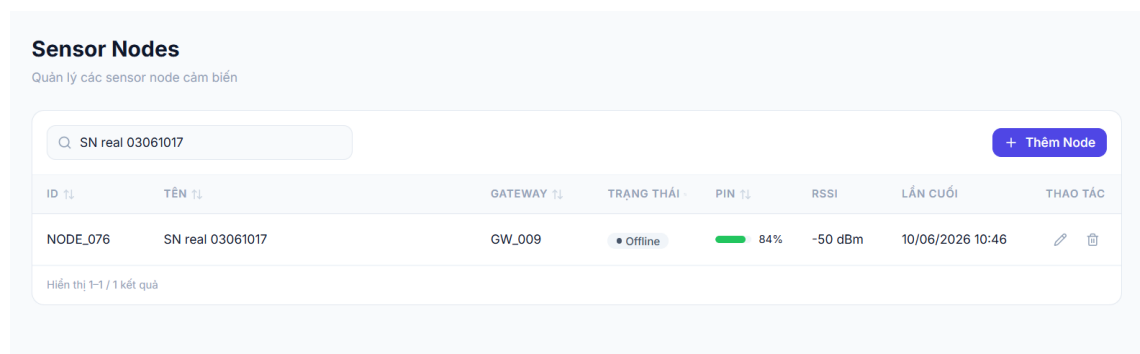
chuyển sang trạng thái online:



ID ↑↓	TÊN ↑↓	GATEWAY ↑↓	TRẠNG THÁI	PIN ↑↓	RSSI	LẦN CUỐI	THAO TÁC
NODE_076	SN real 03061017	GW_009	Online	84%	-50 dBm	10/06/2026 10:46	
NODE_008	Sn14051005	GW_007	Offline	85%	-73 dBm	10/06/2026 07:54	
NODE_009	Node Test 1 Giả lập	GW_007	Offline	99%	-40 dBm	10/06/2026 07:54	
NODE_010	Sensor Điện Biên	GW_001	Offline	88%	-62 dBm	10/06/2026 07:54	

Hình 4.27: Server nhận gói tin và đánh dấu Sensor Node 4C online

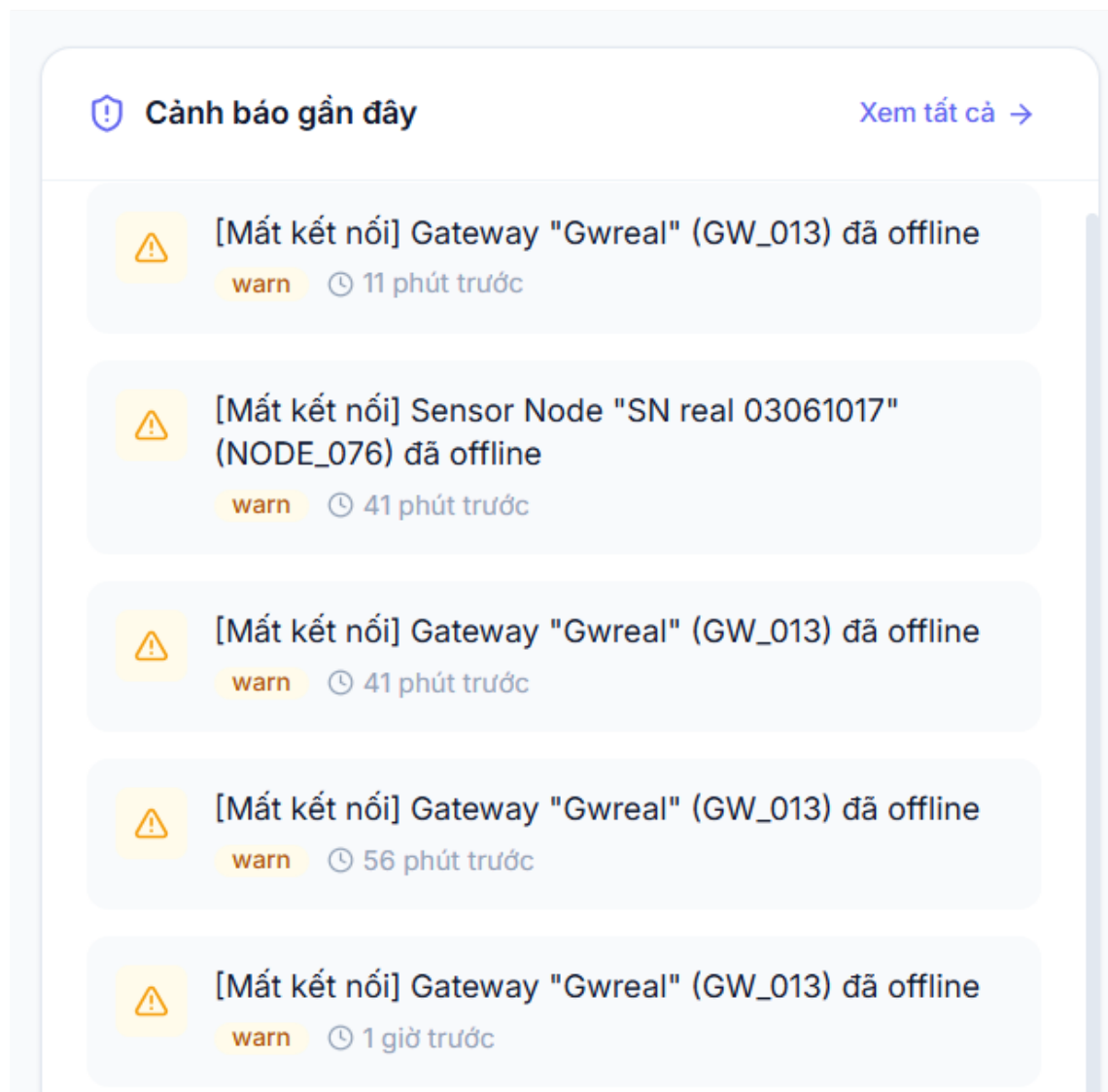
Bước 3: Tắt Sensor Node 4C (NODE_076). Do đặc thù tối ưu năng lượng bằng chế độ deep-sleep, tham số timeout được cấu hình dài hơn. Lúc này Gateway vẫn sẽ hiện online vì vẫn gửi heartbeat đều đặn lên server (Hình 4.30). Sau 40 phút server không nhận được gói tin nào từ Sensor Node 4C, hệ thống sẽ tự động chuyển trạng thái của Node này sang offline và cảnh báo tới admin:



ID ↑↓	TÊN ↑↓	GATEWAY ↑↓	TRẠNG THÁI	PIN ↑↓	RSSI	LẦN CUỐI	THAO TÁC
NODE_076	SN real 03061017	GW_009	Offline	84%	-50 dBm	10/06/2026 10:46	

Hình 4.28: Hệ thống ghi nhận Sensor Node 4C (NODE_076) offline

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.29: Hệ thống cảnh báo Sensor Node 4C (NODE_076) offline

ID	TÊN	VỊ TRÍ	IP	TRẠNG THÁI	HOẠT ĐỘNG LẦN CUỐI	THAO TÁC
GW_013	Gwreal	Nhà	--	Online	10/06/2026 11:41	✎ 🗑
GW_001	Gateway GW_001	--	--	Offline	10/06/2026 10:05	✎ 🗑
GW_007	Gw15050943	Bk	--	Offline	10/06/2026 07:54	✎ 🗑
GW_010	Gw bách khoa	Bách khoa	--	Offline	04/06/2026 12:47	✎ 🗑

Hình 4.30: Gateway vẫn duy trì trạng thái online

Bước 4: Tiến hành tắt Gateway. Khác với Sensor Node, Gateway sử dụng nguồn điện trực tiếp và có chu kỳ phát hiện bất thường ngắn hơn. Sẽ có độ trễ cảnh báo nhỏ, sau đúng 5 phút không nhận được heartbeat, server lập tức đánh dấu Gateway là offline và cảnh báo tới admin:

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Gateways						
Quản lý các gateway thu thập dữ liệu						
Tìm gateway...				+ Thêm Gateway		
ID ↑↓	TÊN ↑↓	VỊ TRÍ ↑↓	IP ↑↓	TRẠNG THÁI ↑↓	HOẠT ĐỘNG LẦN CUỐI	THAO TÁC
GW_013	Gwreal	Nhà	--	Offline	10/06/2026 11:41	✎ ✕
GW_001	Gateway GW_001	--	--	Offline	10/06/2026 10:05	✎ ✕
GW_007	Gw15050943	Bk	--	Offline	10/06/2026 07:54	✎ ✕
GW_010	Gw bách khoa	Bách khoa	--	Offline	04/06/2026 12:47	✎ ✕

Hình 4.31: Hệ thống ghi nhận Gateway offline

 **Cảnh báo gần đây**

[Xem tất cả →](#)

 **[Mất kết nối] Gateway "Gwreal" (GW_013) đã offline**
warn ⌚ 5 phút trước

 **[Mất kết nối] Gateway "Gwreal" (GW_013) đã offline**
warn ⌚ 20 phút trước

 **[Mất kết nối] Sensor Node "SN real 03061017" (NODE_076) đã offline**
warn ⌚ 50 phút trước

Hình 4.32: Hệ thống cảnh báo Gateway offline

Kịch bản thử nghiệm môi trường ô nhiễm (đốt hương): Để đánh giá tốc độ phản hồi và độ nhạy của các cảm biến, một thử nghiệm đốt hương (nhang) được thực hiện trong không gian phòng kín (diện tích $\sim 15\text{m}^2$ – xem Hình 4.33 về cách bố trí không gian). Quá trình thử nghiệm diễn ra trong 17 phút và được chia làm ba pha: pha Baseline (từ 16:20 đến 16:24, phòng đóng cửa, không có nguồn ô nhiễm), pha Đốt hương (từ 16:25 đến 16:32, thắp một nén hương cách Sensor Node khoảng 30 cm), và pha Phục hồi (từ 16:33 đến 16:37, dập tắt hương và mở cửa sổ để không khí lưu thông). Dữ liệu thu thập được từ Sensor Node trong suốt quá trình thử nghiệm được trình bày chi tiết tại Bảng 4.12.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.33: Bố trí thiết bị thử nghiệm đốt hương

Bảng 4.12: Dữ liệu đo lường kích bản đốt hương

Thời gian	Pha	PM2.5	PM10	eCO ₂	eTVOC	Nhiệt độ	Độ ẩm
16:20	Baseline	18	19	451	7	32.6	62.2
16:21	Baseline	20	22	433	5	32.6	62.2
16:22	Baseline	18	19	436	5	32.5	62.5
16:23	Baseline	19	20	442	6	32.5	62.4
16:24	Baseline	20	22	450	7	32.5	62.3
16:25	Ô nhiễm	22	24	448	7	32.6	62.3
16:26	Ô nhiễm	416	513	543	21	32.6	62.4
16:27	Ô nhiễm	321	374	646	37	32.6	62.5
16:28	Ô nhiễm	199	231	1178	117	32.6	62.5
16:29	Ô nhiễm	213	249	1801	451	32.6	62.5
16:30	Ô nhiễm	332	394	1095	105	32.6	62.5
16:31	Ô nhiễm	247	293	945	83	32.6	62.5
16:32	Ô nhiễm	262	306	902	76	32.5	62.7
16:33	Phục hồi	237	275	683	43	32.5	62.3
16:34	Phục hồi	149	170	529	19	32.3	62.0
16:35	Phục hồi	71	82	481	12	32.4	61.3
16:36	Phục hồi	48	60	418	2	32.4	61.2
16:37	Phục hồi	41	50	405	0	32.4	61.2

Phân tích dữ liệu thực nghiệm: Kết quả từ Bảng 4.12 cho thấy sự thay đổi rõ rệt của chất lượng không khí qua ba pha. Tại pha Baseline (5 phút đầu), nồng độ các chất ô nhiễm duy trì ở mức thấp và ổn định. Giá trị PM2.5 trung bình đạt ~19

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

$\mu\text{g}/\text{m}^3$, tương ứng với chỉ số AQI ở mức “Tốt” (màu xanh lá cây). Giá trị eCO_2 dao động quanh mức 442 ppm, nằm trong khoảng an toàn (như hiển thị trên giao diện Dashboard ở Hình 4.34). Cùng với đó, danh sách nhật ký tổng hợp các thông số (Hình 4.35) cho thấy dãy dữ liệu đo được duy trì ở trạng thái bình ổn với biên độ dao động rất nhỏ.



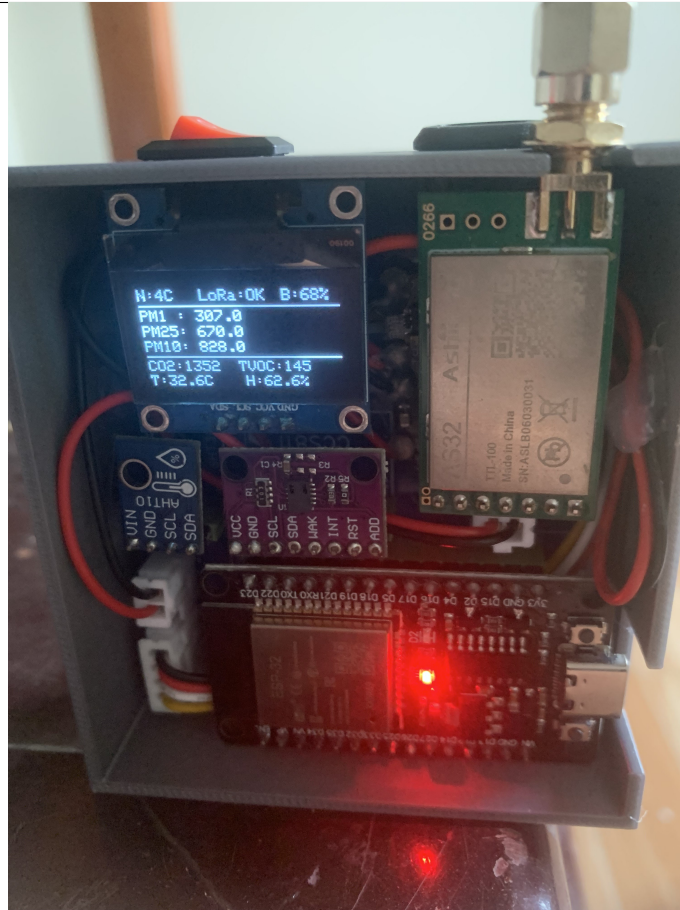
Hình 4.34: Dashboard pha Baseline (trước khi đốt hương)

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

09/06/2026 16:23	SN real 03061017 NODE_076	20.0 $\mu\text{g}/\text{m}^3$	21.0 $\mu\text{g}/\text{m}^3$	451 ppm	7 ppb	32.6 °C	62.4 %	68
09/06/2026 16:23	SN real 03061017 NODE_076	23.0 $\mu\text{g}/\text{m}^3$	26.0 $\mu\text{g}/\text{m}^3$	446 ppm	7 ppb	32.5 °C	62.4 %	74
09/06/2026 16:23	SN real 03061017 NODE_076	19.0 $\mu\text{g}/\text{m}^3$	20.0 $\mu\text{g}/\text{m}^3$	442 ppm	6 ppb	32.5 °C	62.4 %	66
09/06/2026 16:23	SN real 03061017 NODE_076	24.0 $\mu\text{g}/\text{m}^3$	27.0 $\mu\text{g}/\text{m}^3$	436 ppm	5 ppb	32.5 °C	62.4 %	76
09/06/2026 16:22	SN real 03061017 NODE_076	18.0 $\mu\text{g}/\text{m}^3$	19.0 $\mu\text{g}/\text{m}^3$	436 ppm	5 ppb	32.5 °C	62.5 %	63
09/06/2026 16:22	SN real 03061017 NODE_076	25.0 $\mu\text{g}/\text{m}^3$	27.0 $\mu\text{g}/\text{m}^3$	476 ppm	11 ppb	32.5 °C	62.5 %	78
09/06/2026 16:22	SN real 03061017 NODE_076	23.0 $\mu\text{g}/\text{m}^3$	25.0 $\mu\text{g}/\text{m}^3$	454 ppm	8 ppb	32.5 °C	62.3 %	74
09/06/2026 16:22	SN real 03061017 NODE_076	25.0 $\mu\text{g}/\text{m}^3$	28.0 $\mu\text{g}/\text{m}^3$	439 ppm	5 ppb	32.5 °C	62.2 %	78
09/06/2026 16:21	SN real 03061017 NODE_076	22.0 $\mu\text{g}/\text{m}^3$	24.0 $\mu\text{g}/\text{m}^3$	447 ppm	7 ppb	32.6 °C	62.3 %	72
09/06/2026 16:21	SN real 03061017 NODE_076	23.0 $\mu\text{g}/\text{m}^3$	26.0 $\mu\text{g}/\text{m}^3$	464 ppm	9 ppb	32.6 °C	62.2 %	74
09/06/2026 16:21	SN real 03061017 NODE_076	25.0 $\mu\text{g}/\text{m}^3$	28.0 $\mu\text{g}/\text{m}^3$	448 ppm	7 ppb	32.6 °C	62.2 %	78

Hình 4.35: Nhật ký giám sát trong pha Baseline

Ngay sau khi bắt đầu đốt hương tại thời điểm 16:25, các chỉ số ô nhiễm gia tăng đột biến, thể hiện rõ ở pha Ô nhiễm. Do khói hương tỏa ra trực tiếp chứa số lượng lớn hạt bụi mịn, nồng độ PM2.5 tăng vọt từ 22 $\mu\text{g}/\text{m}^3$ lên đỉnh điểm 416 $\mu\text{g}/\text{m}^3$ chỉ trong vòng một phút (tại 16:26), tức là tăng gần 20 lần. Giá trị này quy đổi ra chỉ số AQI vượt mức “Nguy hại” (màu nâu đỏ), cảnh báo khẩn cấp tới sức khỏe (Hình 4.36). Bên cạnh đó, quá trình đốt cháy cũng sinh ra các khí độc hại, khiến nồng độ eCO₂ và eTVOC bắt đầu tăng kể từ phút thứ hai sau khi đốt. Sự thay đổi đồng loạt của tất cả các thông số (AQI, PM2.5, PM10, CO₂, TVOC) được theo dõi sắc nét thông qua danh sách đo lường cùng cấu trúc tương ứng với pha trước (Hình 4.37).



Hình 4.36: Dashboard pha Đốt hương (AQI mức Nguy hại)

09/06/2026 16:27	SN real 03061017 NODE_076	670.0 $\mu\text{g}/\text{m}^3$	828.0 $\mu\text{g}/\text{m}^3$	1352 ppm	145 ppb	32.6 °C	62.6 %	500
09/06/2026 16:27	SN real 03061017 NODE_076	1235.0 $\mu\text{g}/\text{m}^3$	2230.0 $\mu\text{g}/\text{m}^3$	2130 ppm	984 ppb	32.6 °C	63.0 %	500
09/06/2026 16:26	SN real 03061017 NODE_076	1183.0 $\mu\text{g}/\text{m}^3$	2053.0 $\mu\text{g}/\text{m}^3$	1474 ppm	163 ppb	32.6 °C	63.4 %	500
09/06/2026 16:26	SN real 03061017 NODE_076	74.0 $\mu\text{g}/\text{m}^3$	86.0 $\mu\text{g}/\text{m}^3$	1436 ppm	157 ppb	32.5 °C	63.5 %	161
09/06/2026 16:26	SN real 03061017 NODE_076	416.0 $\mu\text{g}/\text{m}^3$	513.0 $\mu\text{g}/\text{m}^3$	543 ppm	21 ppb	32.6 °C	62.5 %	433
09/06/2026 16:25	SN real 03061017 NODE_076	19.0 $\mu\text{g}/\text{m}^3$	20.0 $\mu\text{g}/\text{m}^3$	448 ppm	7 ppb	32.6 °C	62.4 %	66

Hình 4.37: Nhật ký giám sát trong pha Ô nhiễm

Tại pha Phục hồi (bắt đầu từ 16:33), thao tác dập tắt hương và mở cửa sổ đã tạo luồng đối lưu không khí, giúp các thông số ô nhiễm nhanh chóng giảm xuống. Qua sự quan trắc trên dữ liệu nhật ký của hệ thống (Hình 4.38), ta có thể thấy các chỉ số đồng loạt khôi phục trạng thái an toàn. Nồng độ PM2.5 giảm mạnh từ 237 $\mu\text{g}/\text{m}^3$ xuống còn 41 $\mu\text{g}/\text{m}^3$ ở phút 16:37. Tương tự, eCO₂ cũng quay trở về sát mức nền là 405 ppm. Sự thay đổi nhạy bén và cập nhật liên tục này khẳng định hệ thống thu thập và hiển thị dữ liệu qua WebSocket chạy vô cùng ổn định với độ trễ cực thấp.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

09/06/2026 16:39	SN real 03061017 NODE_076	20.0 µg/m³	22.0 µg/m³	461 ppm	9 ppb	32.2 °C	63.2 %	68
09/06/2026 16:39	SN real 03061017 NODE_076	21.0 µg/m³	22.0 µg/m³	434 ppm	5 ppb	32.3 °C	61.7 %	70
09/06/2026 16:38	SN real 03061017 NODE_076	26.0 µg/m³	29.0 µg/m³	446 ppm	7 ppb	32.4 °C	61.9 %	80
09/06/2026 16:38	SN real 03061017 NODE_076	29.0 µg/m³	33.0 µg/m³	406 ppm	0 ppb	32.4 °C	61.2 %	87
09/06/2026 16:38	SN real 03061017 NODE_076	32.0 µg/m³	38.0 µg/m³	405 ppm	0 ppb	32.4 °C	61.1 %	93
09/06/2026 16:38	SN real 03061017 NODE_076	25.0 µg/m³	28.0 µg/m³	405 ppm	0 ppb	32.5 °C	61.2 %	78
09/06/2026 16:38	SN real 03061017 NODE_076	33.0 µg/m³	38.0 µg/m³	400 ppm	0 ppb	32.4 °C	61.2 %	95
09/06/2026 16:37	SN real 03061017 NODE_076	31.0 µg/m³	37.0 µg/m³	403 ppm	0 ppb	32.4 °C	61.2 %	91
09/06/2026 16:36	SN real 03061017 NODE_076	35.0 µg/m³	41.0 µg/m³	407 ppm	1 ppb	32.5 °C	61.2 %	99
09/06/2026 16:36	SN real 03061017 NODE_076	48.0 µg/m³	60.0 µg/m³	439 ppm	5 ppb	32.4 °C	61.3 %	132
09/06/2026 16:36	SN real 03061017 NODE_076	48.0 µg/m³	60.0 µg/m³	418 ppm	2 ppb	32.4 °C	61.2 %	132
09/06/2026 16:36	SN real 03061017 NODE_076	41.0 µg/m³	51.0 µg/m³	418 ppm	2 ppb	32.4 °C	61.1 %	115
09/06/2026 16:35	SN real 03061017 NODE_076	48.0 µg/m³	60.0 µg/m³	420 ppm	3 ppb	32.4 °C	61.3 %	132
09/06/2026 16:35	SN real 03061017 NODE_076	52.0 µg/m³	65.0 µg/m³	422 ppm	3 ppb	32.4 °C	61.3 %	142
09/06/2026 16:35	SN real 03061017 NODE_076	56.0 µg/m³	70.0 µg/m³	434 ppm	5 ppb	32.4 °C	61.2 %	151
09/06/2026 16:35	SN real 03061017 NODE_076	71.0 µg/m³	82.0 µg/m³	481 ppm	12 ppb	32.4 °C	61.3 %	159
09/06/2026 16:34	SN real 03061017 NODE_076	62.0 µg/m³	73.0 µg/m³	400 ppm	0 ppb	32.3 °C	61.4 %	154
09/06/2026 16:34	SN real 03061017 NODE_076	105.0 µg/m³	121.0 µg/m³	400 ppm	0 ppb	32.4 °C	61.7 %	177
09/06/2026 16:34	SN real 03061017 NODE_076	107.0 µg/m³	125.0 µg/m³	470 ppm	10 ppb	32.3 °C	61.8 %	178
09/06/2026 16:34	SN real 03061017 NODE_076	149.0 µg/m³	170.0 µg/m³	529 ppm	19 ppb	32.3 °C	62.0 %	199
09/06/2026 16:34	SN real 03061017 NODE_076	168.0 µg/m³	193.0 µg/m³	548 ppm	22 ppb	32.4 °C	62.1 %	218
09/06/2026 16:33	SN real 03061017 NODE_076	131.0 µg/m³	151.0 µg/m³	565 ppm	25 ppb	32.4 °C	62.4 %	190
09/06/2026 16:33	SN real 03061017 NODE_076	237.0 µg/m³	275.0 µg/m³	683 ppm	43 ppb	32.5 °C	62.3 %	287
09/06/2026 16:33	SN real 03061017 NODE_076	261.0 µg/m³	303.0 µg/m³	778 ppm	57 ppb	32.5 °C	62.3 %	309
09/06/2026 16:33	SN real 03061017 NODE_076	268.0 µg/m³	313.0 µg/m³	706 ppm	46 ppb	32.5 °C	62.5 %	315
09/06/2026 16:32	SN real 03061017 NODE_076	232.0 µg/m³	270.0 µg/m³	832 ppm	65 ppb	32.5 °C	62.7 %	282
09/06/2026 16:32	SN real 03061017 NODE_076	262.0 µg/m³	306.0 µg/m³	902 ppm	76 ppb	32.5 °C	62.7 %	310
09/06/2026 16:32	SN real 03061017 NODE_076	264.0 µg/m³	310.0 µg/m³	945 ppm	83 ppb	32.6 °C	62.6 %	312
09/06/2026 16:32	SN real 03061017 NODE_076	276.0 µg/m³	328.0 µg/m³	1071 ppm	102 ppb	32.6 °C	62.6 %	321

Hình 4.38: Nhật ký giám sát trong pha Phục hồi

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Kịch bản thử nghiệm thời gian sử dụng pin tối đa của Sensor Node: Một trong những yêu cầu quan trọng khi triển khai Sensor Node ngoài trời là khả năng hoạt động bằng pin. Biền thể firmware `sensor-node-30pin-deepsleep` sử dụng chế độ deep-sleep của ESP32 để tối ưu năng lượng: sau mỗi chu kỳ đo và gửi dữ liệu, toàn bộ vi điều khiển chuyển sang trạng thái ngủ sâu (chỉ giữ RTC hoạt động), sau đó tự đánh thức theo bộ hẹn giờ để thực hiện chu kỳ tiếp theo.

Thiết lập thử nghiệm: Sensor Node sử dụng board ESP32 DOIT DevKit V1 (30 chân), pin 18650 Li-ion 3.7V dung lượng 1500 mAh, chu kỳ deep-sleep 30 phút. Mỗi chu kỳ thức, Sensor Node thực hiện tuần tự: khởi tạo phần cứng (~1.5s), đợi PMS7003 warm-up và đọc cảm biến (~30s), đọc pin và gửi gói LoRa (~1.5s) — tổng thời gian thức mỗi chu kỳ là ~33 giây. Pin được sạc đầy 100% trước khi bắt đầu thử nghiệm, Sensor Node hoạt động liên tục cho đến khi hết pin hoàn toàn (0%, vi điều khiển không thể khởi động lại).

Kết quả đo: Bảng 4.13 tóm tắt kết quả thử nghiệm.

Bảng 4.13: Kết quả thử nghiệm thời gian sử dụng pin tối đa

Thông số	Giá trị
Board	ESP32 DOIT DevKit V1 (30 chân)
Pin sử dụng	18650 Li-ion 3.7V / 1500 mAh
Chu kỳ deep-sleep	30 phút
Thời điểm bắt đầu	19/06/2026 09:30
Thời điểm hết pin	20/06/2026 06:00
Tổng thời gian hoạt động	20.5 giờ
Số lần đo (ước tính)	~41 lần (20.5 giờ ÷ 30 phút)

Nhận xét: Kết quả thử nghiệm cho thấy với board DevKit V1 và pin 18650 dung lượng 1500 mAh, Sensor Node deep-sleep hoạt động liên tục được 20.5 giờ. Thời gian sử dụng pin bị hạn chế chủ yếu do các linh kiện trên board phát triển (chip CP2102 USB-UART bridge, bộ ổn áp AMS1117, đèn LED nguồn) tiêu thụ ~26 mA ngay cả khi ESP32 ở chế độ deep-sleep. Để cải thiện, có thể sử dụng bare ESP32-WROOM module (giảm dòng deep-sleep xuống ~1.5 mA) hoặc nâng dung lượng pin lên 3000–3500 mAh.

Kịch bản thử nghiệm khoảng cách truyền LoRa tối đa: Để đánh giá phạm vi phủ sóng thực tế của mạng LoRa 433 MHz, một thử nghiệm đo khoảng cách truyền tối đa được thực hiện giữa Sensor Node và Gateway trong không gian thoáng (outdoor, line-of-sight).

Thiết lập thử nghiệm: Gateway đặt cố định, anten gắn ở độ cao ~1.5 mét so với mặt đất. Sensor Node được di chuyển dần ra xa trong không gian thoáng (không có

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

vật cản lớn giữa hai thiết bị). Cả hai thiết bị sử dụng module LoRa AS32-TTL-100, tần số 433 MHz, anten mặc định đi kèm module. Tại vị trí xa nhất mà Gateway vẫn nhận được gói tin, ghi nhận khoảng cách và quan sát dữ liệu hiển thị trên server.

Kết quả đo: Bảng 4.14 tóm tắt kết quả thử nghiệm.

Bảng 4.14: Kết quả thử nghiệm khoảng cách truyền LoRa tối đa

Thông số	Giá trị
Module LoRa	AS32-TTL-100 (433 MHz)
Môi trường	Không gian thoáng (line-of-sight)
Khoảng cách tối đa đạt được	800 mét
Tầm phủ công bố của module	3000 mét (không gian mở, điều kiện lý tưởng)

Nhận xét: Kết quả cho thấy trong điều kiện không gian thoáng, khoảng cách truyền LoRa tối đa đạt được là 800 mét, tương đương khoảng 27% tầm phủ công bố của module (3000 mét trong điều kiện lý tưởng). Sự chênh lệch này là hợp lý do tầm phủ công bố được đo trong điều kiện lý tưởng (không nhiễu sóng, anten chuyên dụng, độ cao lớn), trong khi thử nghiệm thực tế chịu ảnh hưởng từ môi trường đô thị. Kết quả này xác nhận tính khả thi của mô hình Hub-Spoke: một Gateway đặt tại vị trí trung tâm có thể phục vụ nhiều Sensor Node trong bán kính 500–800 mét mà không cần hạ tầng mạng bổ sung. Trong thực tế triển khai tại khu dân cư (có vật cản như tường, cây xanh), khoảng cách hiệu quả sẽ giảm so với kết quả thử nghiệm này.

Kịch bản thử nghiệm đo đạc đa môi trường: Ngoài thử nghiệm trong phòng kín (đốt hương), hệ thống còn được đem ra đo đạc thực tế tại nhiều môi trường khác nhau nhằm đánh giá khả năng phản ánh chính xác chất lượng không khí trong các điều kiện đa dạng. Ba môi trường được lựa chọn bao gồm: ngoài đường (ven đường giao thông), trong nhà (phòng khách thông thường) và trong bếp (khi nấu ăn). Tại mỗi môi trường, Sensor Node được đặt cố định và thu thập dữ liệu liên tục trong khoảng 10–15 phút.

Môi trường 1 — Ven đường giao thông: Sensor Node được đặt trên vỉa hè cạnh một tuyến đường giao thông (lưu lượng xe máy, ô tô trung bình–thấp) vào khung giờ cao điểm buổi chiều (17:16–17:30). Thiết bị được đặt ở độ cao ~ 1.2 mét so với mặt đất, cách mép đường khoảng 0.5 mét.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.15: Dữ liệu đo lường ven đường giao thông

Thời gian	PM2.5	PM10	eCO ₂	eTVOC	Nhiệt độ	Độ ẩm
17:16	28	34	451	7	32.5	69.6
17:17	31	37	546	22	32.6	69.4
17:18	26	30	810	62	32.5	69.7
17:19	27	31	1482	164	32.6	69.9
17:20	18	20	640	37	32.8	69.1
17:21	25	30	1150	115	32.5	69.9
17:22	27	32	637	36	32.6	69.9
17:23	18	21	627	34	32.6	69.3
17:24	13	15	485	12	32.4	69.3
17:25	18	20	772	56	32.2	70.6
17:26	15	17	1380	149	32.4	70.9
17:27	38	46	1585	232	32.6	70.5
17:28	20	23	653	38	32.7	69.2
17:29	11	13	1012	93	32.7	69.3
17:30	15	17	438	5	32.6	68.9
Trung bình	22.0	25.7	844.5	70.8	32.6	69.7

Nhận xét môi trường ven đường: Giá trị PM2.5 trung bình đạt $\sim 22 \mu\text{g}/\text{m}^3$, tương ứng mức AQI “Trung bình” (vàng). Tuy nồng độ bụi mịn không quá cao, điểm nổi bật là sự biến động mạnh của eCO₂ và eTVOC — hai chỉ thị đặc trưng cho khí thải giao thông. Nồng độ eCO₂ trung bình đạt ~ 845 ppm, dao động mạnh trong dải 438–1585 ppm; eTVOC trung bình đạt ~ 71 ppb với đỉnh lên tới 232 ppb. Các đỉnh ô nhiễm xuất hiện không đều, phản ánh đúng tính chất gián đoạn của luồng xe: tăng đột biến khi có nhóm xe đông đi qua (17:19, 17:21, 17:26–17:27) và giảm nhanh trong khoảng xe thưa (17:24, 17:30). Nhiệt độ ngoài trời ổn định ở mức $\sim 32.6^\circ\text{C}$, độ ẩm $\sim 69.7\%$.

Môi trường 2 — Phòng khách trong nhà: Sensor Node được đặt trong phòng khách có diện tích $\sim 20\text{m}^2$, cửa sổ mở, không bật điều hòa. Đo lường được thực hiện vào buổi chiều (16:47–16:56), không có nguồn ô nhiễm đáng kể nào trong phòng.

Bảng 4.16: Dữ liệu đo lường trong phòng khách

Thời gian	PM2.5	PM10	eCO ₂	eTVOC	Nhiệt độ	Độ ẩm
16:47	15	16	508	16	33.7	68.4
16:48	14	15	433	5	33.8	67.5
16:49	14	15	442	6	33.9	67.8
16:50	14	15	427	4	34.0	66.8
16:51	16	18	436	5	34.0	66.1
16:52	17	19	448	7	34.1	65.9
16:53	16	18	442	6	34.1	65.8
16:54	13	14	451	7	34.1	65.9
16:55	13	14	465	9	34.2	65.7
16:56	19	21	461	9	34.3	65.5
Trung bình	15.1	16.5	451.3	7.4	34.0	66.5

Nhận xét môi trường phòng khách: Giá trị PM2.5 trung bình $\sim 15.1 \mu\text{g}/\text{m}^3$, tương ứng mức AQI “Trung bình” (vàng), cho thấy không khí trong phòng tương đối sạch khi không có nguồn ô nhiễm trực tiếp. Nồng độ eCO₂ ổn định quanh mức ~ 451 ppm và eTVOC trung bình chỉ ~ 7.4 ppb — thấp hơn đáng kể so với môi trường ngoài đường, phản ánh đúng không gian trong nhà không chịu tác động từ khí thải giao thông. Nhiệt độ tăng nhẹ từ 33.7°C lên 34.3°C trong 10 phút do thời tiết buổi chiều, độ ẩm giảm dần tương ứng từ 68.4% xuống 65.5%. Biên độ dao động nhỏ của tất cả các thông số cho thấy cảm biến hoạt động ổn định trong điều kiện không khí sạch.

Môi trường 3 — Bếp khi nấu ăn: Sensor Node được đặt trong khu vực bếp (diện tích $\sim 20\text{m}^2$), cách bếp gas khoảng 0.5 mét. Phép đo được thực hiện trong quá trình nấu ăn bữa tối (18:17–18:32), bao gồm các hoạt động xào, chiên. Thời gian đo 16 phút chia thành ba pha: trước khi nấu (3 phút), đang nấu (10 phút) và sau khi tắt bếp (3 phút).

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.17: Dữ liệu đo lường trong bếp khi nấu ăn

Thời gian	Pha	PM2.5	PM10	eCO ₂	eTVOC	Nhiệt độ	Độ ẩm
18:17	Trước nấu	15	16	572	26	33.3	73.1
18:18	Trước nấu	15	18	508	16	33.4	72.0
18:19	Trước nấu	20	22	674	41	33.4	71.3
18:20	Đang nấu	18	20	1195	121	33.5	71.6
18:21	Đang nấu	16	18	889	74	33.8	71.8
18:22	Đang nấu	15	17	776	57	33.9	71.3
18:23	Đang nấu	15	16	1041	97	34.0	71.0
18:24	Đang nấu	18	20	774	59	34.1	70.5
18:25	Đang nấu	18	21	813	62	34.2	70.2
18:26	Đang nấu	13	14	806	61	34.2	70.0
18:27	Đang nấu	17	19	1372	148	34.4	70.1
18:28	Đang nấu	14	16	913	78	34.5	69.6
18:29	Đang nấu	17	18	1528	191	34.5	69.8
18:30	Sau nấu	17	19	529	19	34.6	69.9
18:31	Sau nấu	20	22	645	37	34.6	69.9
18:32	Sau nấu	18	21	631	35	34.7	69.7

Nhận xét môi trường bếp: Kết quả cho thấy hoạt động nấu ăn (xào, chiên) tạo ra nguồn ô nhiễm khí đáng kể trong không gian bếp nhỏ. Khác với thử nghiệm đốt hương (nơi bụi mịn tăng vọt), ở bếp nấu ăn nồng độ PM2.5 chỉ dao động nhẹ quanh mức nền $\sim 13\text{--}20\text{ }\mu\text{g}/\text{m}^3$, trong khi eCO₂ và eTVOC tăng mạnh — eCO₂ đạt đỉnh 1528 ppm (gấp ~ 3 lần mức nền) và eTVOC đạt đỉnh 191 ppb (gấp ~ 7 lần mức nền). Hiện tượng này phù hợp với đặc thù nấu ăn bằng bếp gas: quá trình đốt cháy khí gas sinh ra nhiều CO₂ và các hợp chất hữu cơ bay hơi, nhưng không tạo lượng lớn hạt bụi mịn như đốt nhang. Nhiệt độ tăng dần $\sim 1.3^\circ\text{C}$ (từ 33.3 lên 34.5°C) do nhiệt từ bếp gas, độ ẩm giảm tương ứng. Sau khi tắt bếp, nồng độ eCO₂ và eTVOC giảm nhưng chưa trở về mức nền ngay (eCO₂ vẫn ở mức ~ 602 ppm sau 3 phút), cho thấy không gian bếp nhỏ cần thời gian thông gió lâu hơn.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

So sánh tổng hợp ba môi trường: Bảng 4.18 tổng hợp giá trị trung bình của các thông số đo tại ba môi trường, giúp rút ra những nhận xét so sánh trực quan.

Bảng 4.18: So sánh giá trị trung bình giữa ba môi trường

Môi trường	PM2.5	PM10	eCO ₂	eTVOC	T (°C)	H (%)	AQI
Ven đường	22.0	25.7	844.5	70.8	32.6	69.7	72
Phòng khách	15.1	16.5	451.3	7.4	34.0	66.5	57
Bếp (đang nấu)	16.1	18.0	1010.7	94.8	34.1	70.6	59
Bếp (trước/sau nấu)	17.5	19.7	593.2	29.0	34.0	71.0	62

Kết quả đo đạc đa môi trường cho thấy hệ thống có khả năng phân biệt rõ ràng đặc trưng ô nhiễm riêng của từng môi trường. Ven đường giao thông, mặc dù nồng độ bụi mịn PM2.5 chỉ ở mức trung bình ($\sim 22 \mu\text{g}/\text{m}^3$), nhưng eCO₂ và eTVOC dao động mạnh với các đỉnh rất cao (eCO₂ tới 1585 ppm, eTVOC tới 232 ppb) do tác động trực tiếp của khí thải xe cộ. Phòng khách trong nhà có chất lượng không khí tốt nhất và ổn định nhất trên mọi thông số, với eCO₂ và eTVOC đều thấp nhất trong ba môi trường. Bếp khi nấu ăn thể hiện đặc trưng ô nhiễm kiểu đốt cháy: eCO₂ trung bình vượt 1000 ppm và eTVOC trung bình ~ 95 ppb — cao hơn đáng kể so với phòng khách (~ 451 ppm eCO₂, ~ 7 ppb eTVOC), nhưng PM2.5 không tăng nhiều. Những kết quả này khẳng định cảm biến đa thông số là cần thiết để đánh giá toàn diện chất lượng không khí: nếu chỉ đo PM2.5, sẽ bỏ sót ô nhiễm khí độc tại bếp và ven đường; ngược lại nếu chỉ đo CO₂ sẽ không phát hiện ô nhiễm bụi mịn. Đồng thời, kết quả gợi mở ứng dụng tiềm năng trong giám sát chất lượng không khí cả ngoài trời lẫn trong nhà.

Về đánh giá độ chính xác số liệu cảm biến: Các kịch bản thử nghiệm trên đã cho thấy cảm biến phản hồi nhanh và cho kết quả ổn định trước sự thay đổi môi trường, tuy nhiên đồ án chưa thực hiện đánh giá định lượng về độ chính xác của số liệu đo. Việc đánh giá này đòi hỏi phải đặt Sensor Node cạnh thiết bị đo chuẩn hoặc trạm quan trắc quốc gia để so sánh số liệu — điều kiện mà đồ án không tiếp cận được do hạn chế về thời gian và nguồn lực. Về mặt thông số kỹ thuật, nhà sản xuất công bố sai số danh định cho từng cảm biến như sau: PMS7003 có sai số $\pm 10 \mu\text{g}/\text{m}^3$ tại nồng độ 100–500 $\mu\text{g}/\text{m}^3$ và $\pm 10\%$ tại nồng độ dưới 100 $\mu\text{g}/\text{m}^3$ [11]; CCS811 có sai số eCO₂ trong khoảng $\pm 10\%$ hoặc ± 50 ppm (lấy giá trị lớn hơn) [12]; AHT10 có sai số $\pm 0.3^\circ\text{C}$ về nhiệt độ và $\pm 2\%$ RH về độ ẩm (đã trình bày ở Chương 3) [13]. Cần lưu ý rằng các giá trị sai số trên là điều kiện lý tưởng trong phòng thí nghiệm của nhà sản xuất; trong thực tế triển khai, sai số thực tế có thể lớn hơn do ảnh hưởng của nhiệt độ, độ ẩm cao, và sự lão hóa cảm biến theo thời gian. Đây là một hạn chế đáng kể của đồ án, và em đề xuất thực hiện hiệu chuẩn

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

bằng cách đặt Sensor Node cạnh trạm đo chuẩn trong một khoảng thời gian nhất định để so sánh và bù sai số, như một hướng phát triển trong tương lai (chi tiết tại Chương 6).

4.5.2 So sánh với các hệ thống hiện có

Bảng 4.19 đối sánh hệ thống đề xuất với bốn hệ thống đã khảo sát ở Chương 2 (Bảng 2.1), bổ sung cột kết quả thực tế của hệ thống đã xây dựng.

Bảng 4.19: So sánh hệ thống đề xuất với các hệ thống hiện có

Tiêu chí	PAM Air	IQAir	PurpleAir	VN-AQI	Hệ thống đề xuất
Cảm biến	PM2.5	PM2.5, CO ₂	PM2.5, PM10	Đầy đủ (FEM)	PM2.5, PM10, CO ₂ , TVOC, T, H
Truyền thông	WiFi/4G	WiFi	WiFi	Chuyên dụng	LoRa + WiFi
Chi phí/trạm	Trung bình	Cao	Trung bình	Rất cao	Thấp
Mã nguồn mở	Không	Không	Một phần	Không	Có (toàn bộ)
Tùy biến	Không	Không	Hạn chế	Không	Có
LoRa	Không	Không	Không	Không	Có (433 MHz)
Realtime web	Có	Có	Có	Hạn chế	Có (Socket.IO)
Admin panel	Không rõ	Không	Không	Có (nội bộ)	Có (web)

Kết quả so sánh cho thấy hệ thống đề xuất giải quyết được bốn hạn chế chính đã xác định ở Chương 2. Thứ nhất, hệ thống đo đa thông số (sáu thông số) trong khi PAM Air và PurpleAir chỉ đo bụi mịn. Thứ hai, truyền thông LoRa cho phép triển khai Sensor Node tại các vị trí không có WiFi, khác biệt so với tất cả bốn hệ thống khảo sát đều yêu cầu WiFi hoặc mạng di động cho mỗi trạm. Thứ ba, mã nguồn mở hoàn toàn cho phép tùy biến và mở rộng, trong khi các hệ thống thương mại không hỗ trợ. Thứ tư, chi phí mỗi trạm Sensor Node thấp nhờ sử dụng linh kiện phổ thông và module LoRa giá rẻ. Tuy nhiên, hệ thống đề xuất có hai hạn chế so với VN-AQI: độ chính xác của cảm biến chi phí thấp không đạt tiêu chuẩn FEM, và quy mô triển khai hiện tại còn nhỏ (vài trạm thử nghiệm).

Bảng 4.20 trình bày chi phí linh kiện chi tiết cho một bộ Sensor Node và một bộ Gateway.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.20: Chi phí linh kiện (BOM) cho Sensor Node và Gateway

STT	Linh kiện	Số lượng	Đơn giá (VNĐ)	Thành tiền
<i>Sensor Node</i>				
1	ESP32 DevKit V1 (30 chân)	1	100.000	100.000
2	Module LoRa AS32-TTL-100	1	120.000	120.000
3	Cảm biến bụi PMS7003	1	350.000	350.000
4	Cảm biến CCS811 (eCO ₂ , eTVOC)	1	150.000	150.000
5	Cảm biến AHT10 (nhiệt độ, độ ẩm)	1	35.000	35.000
6	Màn hình OLED SSD1306 0.96"	1	40.000	40.000
7	Pin 18650 Li-ion 3.7V	1	30.000	30.000
8	Dây nối, mạch in, điện trở	1	50.000	50.000
Tổng chi phí Sensor Node				875.000
<i>Gateway</i>				
9	ESP32 DevKit V1 (30 chân)	1	100.000	100.000
10	Module LoRa AS32-TTL-100	1	120.000	120.000
11	Màn hình OLED SSD1306 0.96"	1	40.000	40.000
12	Dây nối, mạch in, nguồn 5V	1	50.000	50.000
Tổng chi phí Gateway				310.000

Tổng chi phí linh kiện cho một trạm đo hoàn chỉnh (1 Sensor Node + 1 Gateway) là khoảng 1.185.000 VNĐ (~48 USD), thấp hơn đáng kể so với các giải pháp thương mại như IQAir 10.000.000 VNĐ (~400 USD). Nếu một Gateway phục vụ nhiều Sensor Node (mô hình Hub-Spoke), chi phí trung bình mỗi điểm đo còn giảm thêm.

Chương này đã trình bày toàn bộ quá trình thiết kế, xây dựng, kiểm thử và phân tích kết quả vận hành của hệ thống giám sát chất lượng không khí. Các kết quả chính đạt được bao gồm: (i) về thiết kế, hoàn thiện kiến trúc bốn tầng kết hợp mô hình Hub-Spoke với mã nguồn được tổ chức theo nguyên tắc phân lớp rõ ràng; (ii) về xây dựng, hiện thực hóa đầy đủ chín chức năng và năm yêu cầu phi chức năng (Bảng 4.4); (iii) về kiểm thử, hoàn thành 44 kịch bản kiểm thử (25 thủ công và 19 tự động) với tỷ lệ đạt 100%; (iv) về hiệu năng và độ tin cậy, hệ thống đạt thời gian phản hồi API dưới 3s, duy trì uptime tuyệt đối trong 7 ngày liên tục và phát hiện thiết bị mất kết nối chuẩn xác; (v) về thực nghiệm, cảm biến phản hồi nhạy bén trong môi trường ô nhiễm, Sensor Node duy trì hoạt động liên tục 20.5 giờ (Bảng 4.13), truyền thông LoRa đạt khoảng cách truyền tối đa 800 mét (Bảng 4.14), và thiết bị hoạt động tin cậy trên nhiều môi trường thực tế — ven đường giao thông (eCO₂ trung bình ~845 ppm, eTVOC ~71 ppb), phòng khách trong nhà (eCO₂ ~451 ppm, eTVOC ~7 ppb) và bếp nấu ăn (eCO₂ đỉnh 1528 ppm, eTVOC đỉnh 191 ppb), khẳng định tính cần thiết của đo đa thông số để đánh giá toàn diện chất lượng không khí (Bảng 4.18) — đồng thời tối ưu chi phí phần cứng ở

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

mức cạnh tranh khoảng 875.000 VNĐ cho mỗi trạm đo. Những kết quả này chứng minh rõ nét tính khả thi của cấu trúc giám sát đa thông số chi phí thấp sử dụng mạng thông tin vô tuyến LoRa. Thừa hưởng các đánh giá thực chứng đó, Chương 5 sẽ phân tích tổng hợp các đóng góp nổi bật và định hướng phát triển của hệ thống trong tương lai.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày toàn diện quá trình thiết kế, triển khai, kiểm thử và vận hành hệ thống giám sát chất lượng không khí. Trong quá trình đó, một số quyết định kỹ thuật đòi hỏi phân tích sâu và giải quyết những thách thức không tầm thường. Chương 5 tập trung phân tích bốn đóng góp kỹ thuật nổi bật nhất, mỗi đóng góp được trình bày theo cấu trúc ba phần: (i) bài toán và thách thức đặt ra, (ii) giải pháp được lựa chọn và thiết kế, (iii) kết quả đạt được. Bốn đóng góp bao gồm: xây dựng hệ thống IoT đa tầng end-to-end (phần 5.1), thiết kế giao thức LoRa binary và cơ chế chống trùng lặp dữ liệu (phần 5.2), kiến trúc Backend phân lớp và pipeline xử lý telemetry (phần 5.3), và hệ thống chi phí thấp với mã nguồn mở (phần 5.4).

5.1 Xây dựng hệ thống IoT đa tầng end-to-end

5.1.1 Bài toán và thách thức

Hệ thống giám sát chất lượng không khí không phải là một ứng dụng đơn lớp mà là sự tích hợp của bốn miền kỹ thuật hoàn toàn khác biệt: thiết kế và lắp ráp phần cứng, lập trình nhúng trên vi điều khiển (C/C++ với PlatformIO), phát triển backend phía máy chủ (Node.js/Express), và phát triển giao diện web phía người dùng (React/Vite). Mỗi miền yêu cầu bộ công cụ, ngôn ngữ lập trình, và phương pháp debug riêng biệt, tạo ra thách thức tích hợp đặc thù: các lỗi ở ranh giới giữa hai tầng thường chỉ biểu hiện khi toàn bộ hệ thống chạy end-to-end, khó tái hiện trong môi trường kiểm thử riêng lẻ từng tầng.

Cụ thể, giao tiếp giữa các tầng yêu cầu tính đồng nhất tuyệt đối về format dữ liệu. Gói tin LoRa 18 byte do firmware Sensor Node tạo ra phải được Gateway giải mã chính xác theo cùng định nghĩa struct. Batch JSON mà Gateway gửi lên Backend phải khớp với schema validation của Controller. Sự kiện Socket.IO mà Backend phát đi phải được Frontend xử lý đúng theo cùng tên event và cấu trúc payload. Bất kỳ sự không nhất quán nào giữa hai tầng liên kế đều dẫn đến dữ liệu sai hoặc hệ thống ngừng hoạt động.

5.1.2 Giải pháp

Để kiểm soát độ phức tạp tích hợp, em áp dụng chiến lược *contract-first*: định nghĩa đầy đủ giao thức và schema giao tiếp giữa các tầng trước khi viết mã nguồn từng tầng. Giao thức LoRa binary 18 byte được định nghĩa dưới dạng struct C và tải liệu hóa đầy đủ trước khi triển khai cả firmware lẫn parser phía Backend. Schema JSON của HTTP POST từ Gateway được cố định ngay từ đầu để cả hai phía phát triển độc lập mà vẫn tương thích. Tên và payload của các Socket.IO event được

thống nhất giữa Backend và Frontend trước khi triển khai.

Chiến lược phát triển theo thứ tự từ dưới lên cũng giúp kiểm soát rủi ro: firmware Sensor Node được hoàn thiện và kiểm thử trước, tiếp đến là firmware Gateway, rồi Backend, và cuối cùng là Frontend. Mỗi tầng được kiểm thử độc lập trong phạm vi có thể (ví dụ, Backend nhận dữ liệu giả lập từ Postman để kiểm thử trước khi có Gateway thật), giúp khoanh vùng lỗi nhanh hơn. Docker Compose đảm bảo môi trường chạy Backend và cơ sở dữ liệu nhất quán giữa máy phát triển và máy chủ production, loại trừ sai sót do cấu hình môi trường.

5.1.3 Kết quả

Hệ thống hoàn chỉnh bốn tầng được đưa vào vận hành production trong khoảng bốn tháng phát triển, với tổng quy mô mã nguồn khoảng 14.036 dòng mã trải qua bốn nhóm công nghệ (1.262 LOC firmware Sensor Node C/C++, 1.608 LOC firmware Gateway C/C++, 2.732 LOC Backend Node.js, 8.050 LOC Frontend React, cùng 384 LOC SQL và cấu hình triển khai). Toàn bộ chín chức năng và năm yêu cầu phi chức năng được hiện thực đầy đủ (Bảng 4.4). Quy trình triển khai được rút gọn xuống còn một lệnh duy nhất (`docker compose up`), với tính năng tự đăng ký Gateway và Sensor Node không cần thao tác cấu hình thủ công trên máy chủ.

5.2 Thiết kế giao thức LoRa binary và cơ chế chống trùng lặp dữ liệu

5.2.1 Bài toán và thách thức

Truyền thông LoRa trong hệ thống đặt ra hai thách thức đặc thù. Thứ nhất, LoRa là giao thức truyền một chiều không có cơ chế xác nhận (ACK) ở tầng ứng dụng: Sensor Node phát gói tin nhưng không biết liệu Gateway có nhận được hay không. Cùng với đó, trong các kịch bản triển khai mật độ cao, vùng phủ sóng của hai Gateway có thể chồng lấp nhau, dẫn đến cùng một gói tin từ Sensor Node được nhiều Gateway thu nhận đồng thời và chuyển lên Backend — gây trùng lặp dữ liệu trong cơ sở dữ liệu nếu không có cơ chế lọc.

Thứ hai, LoRa là kênh truyền băng thông hẹp với air-time (thời gian chiếm dụng kênh truyền) tỷ lệ thuận với kích thước gói tin. Nếu sử dụng định dạng JSON cho gói tin LoRa (tương tự phần Gateway–Backend), mỗi gói tin sẽ có kích thước khoảng 150–200 byte, kéo dài đáng kể air-time và tăng nguy cơ va chạm gói tin trong môi trường nhiễu thiết bị.

5.2.2 Giải pháp

Để giải quyết vấn đề kích thước gói tin, giao thức LoRa được thiết kế dưới dạng nhị phân (binary protocol) với cấu trúc struct C packed (không có padding byte)

tổng kích thước 18 byte. Gói tin gồm ba trường tiêu đề: `nodeId` (1 byte, định danh node), `pktType` (1 byte, phân loại gói dữ liệu hoặc heartbeat), `msgId` (1 byte, bộ đếm tuần hoàn 0–255). Bảy trường payload lưu các giá trị cảm biến dưới dạng số nguyên: PM1, PM2.5, PM10 (mỗi trường 2 byte, nhân 10 để giữ một chữ số thập phân), CO₂ và TVOC (mỗi trường 2 byte), nhiệt độ (2 byte có dấu, nhân 10), độ ẩm (2 byte, nhân 10), và `battery` (1 byte, 0–100%). Biến đổi nhân 10 loại bỏ hoàn toàn số dấu phẩy động khỏi gói tin, đồng thời vẫn giữ đủ độ chính xác cho hiển thị. Giá trị sentinel lần lượt là 0xFFFF (unsigned) và 0x7FFF (signed) đánh dấu cảm biến bị lỗi đọc. Cấu trúc gói tin được minh họa tại Hình 4.5. So với JSON, kích thước 18 byte giảm air-time khoảng 11 lần.

Để giải quyết vấn đề trùng lặp dữ liệu, Backend triển khai cơ chế deduplication dựa trên trường `msgId`. Mỗi Sensor Node duy trì bộ đếm tăng dần tuần hoàn; khi Backend nhận gói tin, module `dedupCache` kiểm tra cặp khóa (`nodeId`, `msgId`) trong bộ nhớ đệm trước khi ghi vào cơ sở dữ liệu. Nếu cặp khóa đã tồn tại, gói tin được nhận diện là bản sao và bị loại bỏ. Chi tiết triển khai được trình bày tại phần 4.2.4.

5.2.3 Kết quả

Giao thức binary 18 byte giảm air-time LoRa xuống khoảng 11 lần so với phương án JSON tương đương, cải thiện đáng kể thông lượng kênh truyền và giảm tiêu thụ điện của module LoRa (thời gian phát sóng ngắn hơn). Cơ chế deduplication hoạt động chính xác trong toàn bộ các kịch bản kiểm thử: dữ liệu chỉ được ghi một lần duy nhất ngay cả khi nhiều Gateway chuyển tiếp cùng một gói tin. Thử nghiệm khoảng cách truyền LoRa trong không gian thoáng đạt 800 mét (Bảng 4.14), đủ để một Gateway đặt tại vị trí trung tâm phục vụ nhiều Sensor Node trong phạm vi bán kính triển khai thực tế.

5.3 Kiến trúc Backend phân lớp và pipeline xử lý telemetry

5.3.1 Bài toán và thách thức

Backend phải đáp ứng đồng thời bốn luồng xử lý có mức độ ưu tiên khác nhau: tiếp nhận và lưu dữ liệu telemetry từ Gateway (quan trọng nhất — không được mất dữ liệu), kiểm tra ngưỡng và tạo cảnh báo, phát sự kiện realtime qua Socket.IO, và phục vụ các REST API cho Frontend. Thách thức thiết kế then chốt là đảm bảo tính bền vững của dữ liệu: nếu bước kiểm tra cảnh báo xảy ra lỗi (ví dụ, lỗi truy vấn bảng cấu hình), logic lưu dữ liệu đo không được bị cuốn theo rollback. Trước khi thiết kế lại, cách tiếp cận đơn giản nhất là đặt toàn bộ xử lý vào một transaction — nhưng điều này vi phạm tính độc lập giữa hai nghiệp vụ.

Một thách thức khác là hiệu năng truy vấn dữ liệu lịch sử. Bảng `measure-`

ments tăng trưởng liên tục (mỗi Sensor Node ghi một bản ghi mỗi 30 phút); truy vấn thống kê theo giờ cho dải 30 ngày yêu cầu GROUP BY trên hàng chục nghìn bản ghi, dễ dẫn đến thời gian phản hồi vượt ngưỡng 3 giây yêu cầu phi chức năng.

5.3.2 Giải pháp

Pipeline xử lý telemetry được thiết kế gồm ba bước tách biệt. Bước một, module dedupCache lọc gói tin trùng lặp trước khi bất kỳ thao tác cơ sở dữ liệu nào diễn ra. Bước hai, một database transaction duy nhất cập nhật trạng thái Gateway, cập nhật trạng thái Sensor Node và ghi bản ghi đo vào bảng measurements — ba thao tác này được gộp chung trong một transaction để đảm bảo tính nhất quán. Bước ba, sau khi transaction kết thúc thành công, module alertService kiểm tra ngưỡng và tạo cảnh báo nằm ngoài transaction, đảm bảo lỗi cảnh báo không ảnh hưởng đến dữ liệu đo vừa được ghi. Biểu đồ trình tự mô tả pipeline này được trình bày tại Hình 4.6. Luồng phụ thuộc giữa các module tuân thủ nguyên tắc một chiều: routes → controllers → services → config, đảm bảo khả năng kiểm thử và bảo trì từng lớp độc lập.

Để giải quyết hiệu năng truy vấn lịch sử, hệ thống tận dụng tính năng Continuous Aggregate của TimescaleDB. View hourly_measurements được TimescaleDB tự động cập nhật gia tăng (incremental refresh) mỗi khi có dữ liệu mới ghi vào bảng measurements, lưu sẵn giá trị trung bình theo giờ cho từng Sensor Node. Khi Frontend yêu cầu dữ liệu lịch sử theo chế độ giờ, Backend truy vấn trực tiếp từ view tập hợp sẵn thay vì thực hiện GROUP BY tốn kém trên bảng gốc. Chi tiết thiết kế cơ sở dữ liệu được trình bày tại phần 4.2.5.

Ngoài ra, module alertService triển khai cơ chế cooldown 15 phút: nếu cùng một Sensor Node và cùng một thông số đã phát sinh cảnh báo trong 15 phút gần nhất, cảnh báo mới sẽ không được tạo, ngăn chặn tình trạng spam thông báo khi dữ liệu ô nhiễm liên tục vượt ngưỡng trong thời gian ngắn.

5.3.3 Kết quả

Pipeline ba bước đảm bảo zero data loss: trong toàn bộ quá trình kiểm thử và vận hành, không có bản ghi đo nào bị mất do lỗi ở bước kiểm tra cảnh báo. Module tính AQI (aqiService) — nghiệp vụ cốt lõi của hệ thống — được kiểm thử đơn vị với 19 test case tập trung vào giá trị biên, tất cả đều pass (phần 4.4.7). Tổng cộng 44/44 kịch bản kiểm thử (25 thủ công và 19 tự động) đạt kết quả pass (phần 4.4.8). Thời gian phản hồi API truy vấn dữ liệu 30 ngày nhờ Continuous Aggregate luôn dưới 3 giây, đáp ứng đúng yêu cầu phi chức năng. Hệ thống duy trì uptime 100% trong 7 ngày theo dõi liên tục trên môi trường production.

5.4 Hệ thống chi phí thấp với mã nguồn mở hoàn toàn

5.4.1 Bài toán và thách thức

Một hệ thống giám sát chất lượng không khí chỉ có giá trị thực tiễn khi có thể triển khai với mật độ đủ dày để phản ánh chính xác chất lượng không khí của một khu vực. Các giải pháp thương mại hiện có như IQAir (khoảng 10.000.000 VNĐ/thiết bị) hay các trạm đo chuẩn của VN-AQI (hàng chục đến hàng trăm triệu đồng/trạm) có chi phí quá cao để triển khai diện rộng, đặc biệt trong các khu dân cư hoặc trường học. Ngoài ra, tất cả các giải pháp thương mại khảo sát ở Chương 2 đều là hệ thống đóng (closed-source), không cho phép tùy biến logic nghiệp vụ hoặc tích hợp với hệ thống bên thứ ba.

5.4.2 Giải pháp

Hệ thống được xây dựng hoàn toàn từ linh kiện phổ thông, dễ mua, có tài liệu cộng đồng phong phú. Sensor Node sử dụng ESP32 DevKit V1 làm vi điều khiển trung tâm, kết hợp module LoRa AS32-TTL-100 và ba cảm biến: PMS7003 (bụi mịn PM1/PM2.5/PM10), CCS811 (CO₂ và TVOC), AHT10 (nhiệt độ và độ ẩm). Gateway sử dụng cùng board ESP32 với module LoRa tương tự. Đây đều là linh kiện có thể mua tại các cửa hàng điện tử trong nước, không phụ thuộc vào nhà cung cấp chuyên biệt.

Toàn bộ mã nguồn hệ thống — bao gồm firmware C/C++ cho Sensor Node và Gateway, Backend Node.js/Express, và Frontend React — được công khai dưới giấy phép mã nguồn mở, cho phép bên thứ ba tái sử dụng, tùy biến và mở rộng. Hệ thống hỗ trợ thêm Sensor Node và Gateway mới thông qua giao diện web captive portal (provisioning không dây) mà không cần truy cập vật lý vào thiết bị hay can thiệp vào cấu hình máy chủ. So sánh chi tiết với các hệ thống hiện có được trình bày tại Bảng 4.19.

5.4.3 Kết quả

Chi phí linh kiện cho một Sensor Node hoàn chỉnh là khoảng 875.000 VNĐ (36 USD), thấp hơn giải pháp IQAir khoảng 11 lần (Bảng 4.20). Áp dụng mô hình Hub-Spoke, một Gateway (310.000 VNĐ) có thể phục vụ nhiều Sensor Node trong bán kính 500–800 mét, khiến chi phí bình quân mỗi điểm đo tiếp tục giảm khi quy mô triển khai tăng lên. Hệ thống đề xuất là giải pháp duy nhất trong số các hệ thống khảo sát kết hợp đồng thời bốn điểm khác biệt: đo đa thông số (sáu thông số), truyền thông LoRa không cần WiFi tại từng trạm, mã nguồn mở hoàn toàn, và chi phí phần cứng thấp.

Chương 5 đã phân tích bốn đóng góp kỹ thuật nổi bật của hệ thống. Tổng hợp

lại, các đóng góp thể hiện ba ý nghĩa cốt lõi. Thứ nhất, về mặt tích hợp, việc phối hợp bốn miền kỹ thuật khác nhau thành một hệ thống vận hành ổn định chứng minh tính khả thi của quy trình phát triển contract-first kết hợp kiểm thử theo từng tầng. Thứ hai, về mặt kỹ thuật, hệ thống giải quyết thành công các thách thức đặc thù của IoT trong điều kiện thực tế: thiết kế giao thức truyền thông tinh gọn với khả năng chống lỗi dữ liệu, và kiến trúc Backend đảm bảo tính bền vững của dữ liệu đo. Thứ ba, về mặt thực tiễn, chi phí thấp và mã nguồn mở tạo ra điều kiện để hệ thống được nhân rộng và tùy biến cho các ngữ cảnh triển khai khác nhau. Chương 6 sẽ tổng kết toàn bộ đồ án và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương 5 đã phân tích bốn đóng góp kỹ thuật nổi bật, chứng minh rằng từng quyết định thiết kế trong hệ thống đều xuất phát từ thách thức cụ thể và được đánh giá qua kết quả đo lường thực tế. Chương 6 tổng kết toàn bộ đồ án, đánh giá mức độ hoàn thành mục tiêu, ghi nhận những hạn chế còn tồn tại, và đề xuất các hướng phát triển để nâng cao hệ thống trong tương lai.

6.1 Kết luận

Đồ án đặt ra mục tiêu xây dựng một hệ thống giám sát chất lượng không khí chi phí thấp, hoạt động ổn định trong điều kiện thực tế, có khả năng mở rộng quy mô triển khai và công khai toàn bộ mã nguồn. Hệ thống hoàn chỉnh bốn tầng — Sensor Node, Gateway, Backend, và Frontend — được đưa vào vận hành trong khoảng bốn tháng phát triển, với tổng quy mô 14.036 dòng mã trải qua bốn nhóm công nghệ. Toàn bộ chín chức năng và năm yêu cầu phi chức năng được hiện thực đầy đủ (Bảng 4.4). Kết quả kiểm thử đạt 44/44 kịch bản (Mục 4.4.8), trong đó 19 test case kiểm thử đơn vị module tính AQI đều pass. Hệ thống duy trì uptime 100% trong bảy ngày theo dõi liên tục trên môi trường production.

Khi đặt cạnh các giải pháp thương mại và nghiên cứu khảo sát ở Chương 2, hệ thống đề xuất là giải pháp duy nhất đồng thời đáp ứng bốn tiêu chí: đo đa thông số (sáu thông số: PM2.5, PM10, CO₂, TVOC, nhiệt độ và độ ẩm), truyền thông LoRa không yêu cầu WiFi tại từng điểm đo, mã nguồn mở hoàn toàn, và chi phí phần cứng thấp. Chi phí một Sensor Node hoàn chỉnh ở mức khoảng 875.000 VNĐ — thấp hơn giải pháp IQAir khoảng 11 lần (Bảng 4.20). Nhờ mô hình Hub-Spoke, một Gateway có thể phục vụ nhiều Sensor Node trong bán kính 500–800 mét (Bảng 4.14), khiến chi phí bình quân mỗi điểm đo tiếp tục giảm khi quy mô tăng lên.

Trong quá trình thực hiện, em rút ra ba bài học kinh nghiệm chính. Thứ nhất, chiến lược phát triển contract-first — định nghĩa đầy đủ giao thức và schema giao tiếp giữa các tầng trước khi viết mã nguồn từng tầng — là yếu tố then chốt giúp kiểm soát độ phức tạp tích hợp trong hệ thống đa tầng, đa công nghệ. Thứ hai, tính năng Continuous Aggregate của TimescaleDB là lựa chọn phù hợp để đảm bảo hiệu năng truy vấn dữ liệu lịch sử khối lượng lớn mà không cần thay đổi schema hay thêm tầng cache phức tạp. Thứ ba, kết hợp chế độ deep sleep với giao thức LoRa binary 18 byte là cặp giải pháp bổ trợ lẫn nhau: deep sleep giảm tiêu thụ điện ở phía phần cứng, trong khi giao thức binary giảm thời gian phát sóng — hai yếu tố cùng quyết định tuổi thọ pin của Sensor Node.

Bên cạnh những kết quả đạt được, hệ thống còn một số hạn chế cần thừa nhận. Thứ nhất, thời gian hoạt động pin trong dài hạn (trên một tháng) chưa được đo kiểm thực tế trong điều kiện triển khai liên tục; kết quả ước tính dựa trên công suất tiêu thụ lý thuyết và chưa tính đến suy giảm dung lượng pin theo thời gian. Thứ hai, hệ thống chưa tích hợp tính năng hiệu chuẩn cảm biến (calibration) theo chuẩn đo lường, khiến số liệu có thể lệch dần so với giá trị chuẩn sau một thời gian vận hành. Thứ ba, giao diện giám sát hiện chỉ hỗ trợ truy cập qua trình duyệt web; chưa có ứng dụng di động để theo dõi khi không có máy tính.

6.2 Hướng phát triển

Hướng phát triển thứ nhất tập trung vào phần cứng và firmware. Thiết kế lại mạch điện sử dụng chip ESP32 bare (không dùng board phát triển DevKit) cho phép tối ưu sâu hơn tiêu thụ điện, vì các board phát triển thường tích hợp thêm mạch nạp USB-UART và đèn LED tiêu thụ dòng điện ngay cả khi vi điều khiển đang ở chế độ deep sleep. Ngoài ra, việc bổ sung cảm biến khí độc như SO_2 , NO_2 , và O_3 sẽ mở rộng phạm vi đo, làm cho hệ thống phù hợp hơn với các kịch bản giám sát khu công nghiệp hoặc khu vực giao thông mật độ cao.

Hướng phát triển thứ hai liên quan đến hệ thống truyền thông. Giao thức LoRa binary hiện tại được thiết kế riêng cho hệ thống này và có hạn chế về khả năng tương tác với hạ tầng mạng của bên thứ ba. Tích hợp chuẩn Long Range Wide Area Network (LoRaWAN) (thông qua Chirpstack hoặc The Things Network) sẽ cho phép Sensor Node kết nối vào các hạ tầng mạng LoRaWAN sẵn có, mở rộng phạm vi triển khai mà không cần đặt thêm Gateway riêng. Đồng thời, cơ chế downlink của LoRaWAN cho phép Backend điều chỉnh chu kỳ đo của từng Sensor Node từ xa, linh hoạt hơn trong việc cân đối giữa độ chi tiết dữ liệu và tuổi thọ pin.

Hướng phát triển thứ ba nâng cấp tầng Backend và khả năng phân tích dữ liệu. Với lượng dữ liệu lịch sử tích lũy qua nhiều tháng vận hành, hệ thống có tiền đề để tích hợp mô hình dự báo AQI ngắn hạn (ví dụ, LSTM hoặc Prophet) nhằm cảnh báo trước các đợt ô nhiễm; đây là tính năng mà không giải pháp nào trong số các hệ thống đã khảo sát hỗ trợ. Bổ sung kênh push notification qua email hoặc Telegram sẽ làm cho cơ chế cảnh báo ngưỡng trở nên chủ động hơn so với cơ chế hiển thị thụ động trên giao diện web hiện tại. Hỗ trợ kiến trúc multi-tenant sẽ cho phép nhiều tổ chức — trường học, khu dân cư, hoặc nhà máy — quản lý mạng lưới Sensor Node của riêng mình trên cùng một nền tảng Backend.

Hướng phát triển thứ tư hướng đến triển khai thực tế quy mô lớn hơn. Hiệu chuẩn cảm biến bằng cách đặt Sensor Node cạnh trạm đo chuẩn của cơ quan quản lý môi trường trong một khoảng thời gian nhất định sẽ cho phép xác định mức sai

lệch và bù trừ cho từng lô cảm biến. Phát triển ứng dụng di động (React Native hoặc Flutter) sẽ bổ sung kênh truy cập thuận tiện hơn, đặc biệt cho người dùng cuối không ngồi trước máy tính. Thử nghiệm triển khai tại các kịch bản thực tế như trường học, bệnh viện, hoặc khu công nghiệp sẽ cung cấp dữ liệu thực địa để đánh giá và hoàn thiện hệ thống trước khi nhân rộng.

Tóm lại, hệ thống giám sát chất lượng không khí được xây dựng trong đồ án đã chứng minh tính khả thi của mô hình triển khai chi phí thấp, mã nguồn mở, trên nền tảng linh kiện phổ thông. Các hướng phát triển đề xuất tạo ra lộ trình rõ ràng để hệ thống đáp ứng các kịch bản triển khai thực tế quy mô lớn hơn, đóng góp vào bài toán giám sát môi trường đô thị tại Việt Nam.

TÀI LIỆU THAM KHẢO

- [1] World Health Organization, “Who global air quality guidelines: Particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide,” World Health Organization, Tech. Rep., 2021. [Online]. Available: <https://www.who.int/publications/i/item/9789240034228>
- [2] IQAir AG, “2024 world air quality report: Region & city PM_{2.5} ranking,” IQAir, Tech. Rep., 2025. Accessed: Jun. 15, 2026. [Online]. Available: <https://www.iqair.com/world-air-quality-report>
- [3] Bộ Tài nguyên và Môi trường, *Cổng thông tin quan trắc môi trường*, 2023. Accessed: May 1, 2026. [Online]. Available: <https://enviinfo.cem.gov.vn>
- [4] L. Morawska et al., “Applications of low-cost sensing technologies for air quality monitoring and exposure assessment: How far have they gone?” *Environment International*, vol. 116, pp. 286–299, 2018. DOI: 10.1016/j.envint.2018.04.018
- [5] PAM Air JSC, *Pam air – hệ thống quan trắc môi trường không khí*, 2024. Accessed: May 1, 2026. [Online]. Available: <https://pamair.org>
- [6] IQAir AG, *Iqair airvisual – world air quality*, 2024. Accessed: May 1, 2026. [Online]. Available: <https://www.iqair.com>
- [7] PurpleAir Inc., *Purpleair – real-time air quality monitoring*, 2024. Accessed: May 1, 2026. [Online]. Available: <https://www.purpleair.com>
- [8] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, “A study of LoRa: Long range & low power networks for the Internet of Things,” *Sensors*, vol. 16, no. 9, p. 1466, 2016. DOI: 10.3390/s16091466
- [9] United States Environmental Protection Agency, “Technical assistance document for the reporting of daily air quality – the air quality index (AQI),” US EPA, Tech. Rep. EPA-454/B-18-007, 2018.
- [10] Espressif Systems, *ESP32 technical reference manual*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [11] Plantower, *PMS7003 digital universal particle concentration sensor – datasheet*, 2023. Accessed: Jun. 1, 2026. [Online]. Available: https://www.plantower.com/en/products_33/74.html
- [12] ams AG, *CCS811 ultra-low power digital gas sensor for monitoring indoor air quality – datasheet*, 2023. Accessed: Jun. 1, 2026. [Online]. Available:

- https://cdn.sparkfun.com/assets/learn_tutorials/1/4/3/CCS811_Datasheet-DS000459.pdf
- [13] Aosong Electronics, *AHT10 integrated temperature and humidity sensor – datasheet*, 2023. Accessed: Jun. 1, 2026. [Online]. Available: <https://www.aosong.com/en/products-109.html>
 - [14] OpenJS Foundation, *Node.js – about*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://nodejs.org/en/about>
 - [15] The PostgreSQL Global Development Group, *PostgreSQL 15 documentation*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://www.postgresql.org/docs/15/>
 - [16] Timescale Inc., *TimescaleDB documentation – hypertables and continuous aggregates*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.timescale.com>
 - [17] PostGIS Project Steering Committee, *PostGIS – spatial and geographic objects for PostgreSQL*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://postgis.net/documentation/>
 - [18] Prisma Data Inc., *Prisma – next-generation ORM for Node.js and TypeScript*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://www.prisma.io/docs>
 - [19] Socket.IO Contributors, *Socket.IO documentation*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://socket.io/docs/v4/>
 - [20] Meta Platforms Inc., *React – a JavaScript library for building user interfaces*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://react.dev>
 - [21] E. You, *Vite – next generation frontend tooling*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://vite.dev>
 - [22] V. Agafonkin, *Leaflet – an open-source JavaScript library for interactive maps*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://leafletjs.com>
 - [23] Apache Software Foundation, *Apache ECharts – an open source JavaScript visualization library*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://echarts.apache.org>
 - [24] Docker Inc., *Docker documentation – overview*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.docker.com/get-started/overview/>